NANA

tu jardín de manga

Bitácora del proceso

De la idea a la versión final publicada, versión por versión

Materia	Desarrollo Frontend II · UADE · 2.º cuatrimestre 2026
Profesor	Joaquín Cerviño · Viernes, turno noche
Trabajo	Trabajo Práctico 2 · Versión final v1.2
Apellido y nombre	_____
L.U.	_____
App publicada	jardin-de-manga.vercel.app
Repositorio	github.com/serenaretrive02/jardin-de-manga
Fecha	13 de noviembre de 2026

Registro de cómo fue evolucionando la aplicación, versión por versión: qué se cambió, por qué, cómo se probó, cómo se usó la inteligencia artificial y qué problemas aparecieron.

> La app empezó con el nombre provisorio *Jardín de Manga*. Desde la v0.3 se llama **Nana** (ver la v0.3).

Índice

Versión	Fecha	Resumen
Etapa 0 — La idea	09/11	Lluvia de ideas y elección
v0.1 — MVP	09/11	CRUD con Firestore, GitHub y Vercel
v0.2 — Pixel art	10/11	Identidad visual y experiencia de uso
v0.3 — Nana + API	10/11	Nombre, búsqueda en Kitsu, edición
v0.4 — Comunidad	10/11	Clubes, misiones por rol, eventos
v0.4.1 — Ajustes	10/11	Flores únicas, guía, abeja, Nana IA
v1.0 — Final	10/11	Ramo, filtros, herbario, sonidos, auditoría
v1.1 — Pulido kawaii	10/11	Más espacio, esquinas pixeladas, adornos, revisión final
v1.2 — Puntuación	13/11	Puntuar con sakuras y reseña; Diario de lecturas, música de fondo, gatito y revisión final
Conclusión		Qué se aprendió

Etapa 0 — La idea (09/11/2026)

Punto de partida

Lo primero fue leer la consigna del TP2, la rúbrica, la consigna del examen final y las guías de teoría de la materia: Introducción a React, npm, módulos, props y hooks, APIs y arrays. El examen final consiste en **defender este mismo proyecto**, así que desde el inicio se buscó una app que use los temas vistos en clase y que se pueda seguir mejorando después.

Lluvia de ideas

Le pedí a la IA ideas que mezclaran mis gustos (Minecraft, League of Legends, manga/anime y flores) y que permitieran agregar, consultar, modificar y eliminar datos. Surgieron cinco propuestas:

Idea	De qué se trataba	Por qué sí / por qué no
Jardín de Manga ✅	Cada manga/anime es una flor que crece según lo que leés	Une tres de mis gustos y es muy visual. Es una "vuelta de tuerca" al registro de libros que sugiere la consigna
Bloque a Bloque	Planificador de construcciones de Minecraft con materiales	Buena idea de inventario, pero menos original visualmente
Champion Lab	Diario de práctica de League of Legends	Correcta técnicamente, pero más "típica"
Quest Log	Tareas convertidas en misiones de videojuego	Muy cercana a la lista de tareas común
Herbario Otaku	Catálogo de flores que aparecen en animes	Original, pero con pocas acciones posibles

Decisión

Elegí **Jardín de Manga**. Las acciones que pide la consigna se convierten en acciones de jardinería:

Lo que pide la consigna	En la app
Agregar un registro	**Plantar** un manga o anime
Consultar	Ver **mi jardín**
Modificar	**Regar** (+1 capítulo) y cambiar el estado
Eliminar	**Compostar** (quitar la planta)

¿Para quién es? Para lectoras y lectores de manga y anime que siguen muchas series a la vez y quieren recordar por dónde iban, de una forma más linda que una lista o una planilla.

Plan de versiones

Se planificaron 5 versiones, siguiendo la idea de **MVP** (producto mínimo viable) vista en clase: primero algo simple que funcione y después mejoras por etapas.

Versión	Objetivo
v0.1	Que funcione: guardar, ver, modificar y borrar en Firebase. Diseño básico a propósito
v0.2	Identidad visual de jardín, versión para celular, mensajes claros
v0.3	Buscar mangas/animes reales con una API (portadas y capítulos)
v0.4	Filtros, buscador, estadísticas y significado de cada flor
v1.0	Pulido final, pruebas completas y documentación

Versión 0.1 — MVP: que funcione (09/11/2026)

Qué se hizo

1. **Se creó el proyecto** en Visual Studio Code con el comando visto en clase: `npm create vite@latest jardin-de-manga -- --template react`
2. **Se instalaron las dependencias** con `npm install` y la librería de Firebase con `npm install firebase`.
3. **Se borraron los archivos de ejemplo** de Vite (logos e imágenes de muestra) que no se iban a usar.
4. **Se organizó el código en archivos chicos**, uno por tarea, como recomienda la guía de módulos:
   - `firebase.js`: la conexión con Firebase.
   - `services/plantas.js`: las 4 operaciones con la base de datos (consultar, agregar, modificar, eliminar).
   - `data/flores.js`: qué flor corresponde a cada género y cómo se ve la planta según el avance.
   - `components/`: el formulario, la lista y la tarjeta de cada planta.
5. **Se escribieron las reglas de seguridad** de la base de datos (`firestore.rules`).
6. **Se preparó el archivo** `.env.example`, con los nombres de las llaves de Firebase pero sin sus valores.
7. **Se escribió el README** con las instrucciones para instalar y ejecutar.

Cómo funciona esta versión (explicado simple)

- Cuando alguien abre la app, Firebase le da una **identidad anónima**: lo reconoce sin pedirle usuario ni contraseña.
- Cada persona tiene **su propio jardín** guardado en Firestore. Nadie más puede verlo ni modificarlo.
- El formulario permite **plantar** un manga o anime con título, tipo, género y cantidad de capítulos.
- Cada tarjeta muestra la planta con un emoji que cambia según el avance: 🫘 semilla → 🌱 brote → 🌿 creciendo → 🌷 en flor (o 🥀 si se abandona).
- Se puede **cambiar el estado**, **sumar un capítulo** y **eliminar**. Después de cada cambio, la app vuelve a pedirle los datos a Firebase, así lo que se ve en pantalla es lo que realmente quedó guardado.

Lo que esta versión todavía NO tiene (a propósito)

Se deja anotado para mejorarlo en las próximas versiones:

- El diseño es muy básico: letra común, bordes grises, sin colores ni identidad.
- En el celular se ve, pero no está pensado para pantallas chicas.
- No hay mensajes de "¡guardado!": solo aparecen los errores.
- Borrar no pide confirmación, así que un clic por error elimina la planta.
- Hay que escribir los títulos a mano y no hay portadas.
- No se puede buscar ni filtrar dentro del jardín.
- Los emojis de las flores dependen del sistema de cada dispositivo y no se pueden personalizar.

Uso de IA en esta etapa

- **Herramienta:** Claude Code (Claude, de Anthropic) dentro de la app de escritorio, trabajando sobre la carpeta del proyecto.
- **Ejemplos de lo que le pedí:**
  - *"Leé la teoría de todos los PDF y la consigna del TP2. Antes de crear cualquier aplicación, pensá ideas creativas y originales según lo que me gusta (Minecraft, LoL, manga, anime, flores) que permitan hacer todas las acciones que pide la consigna."*
  - *"Quiero avanzar con la idea del jardín de manga. Usamos Visual Studio Code, GitHub y lo que está en las teorías de los PDF, así que todo tiene que ser lo que vimos en clase."*
- **Qué revisé del resultado:** que el código use lo visto en clase (`useState` , `useEffect` , props, `map` , `try/catch/finally` como en la guía de props y hooks) y que la conexión con Firebase lea las llaves desde el `.env` con `import.meta.env` , como pide la consigna.

Dificultades y cómo se resolvieron

Problema	Cómo se detectó	Solución
El `.gitignore` que crea Vite **no incluía el archivo** `.env` . Si se subía así, las llaves de Firebase iban a quedar publicadas en GitHub	Revisando el archivo contra el paso 6 de la consigna	Se agregaron `.env` y sus variantes al `.gitignore`
Sin el `.env` , la app se rompía y mostraba la pantalla en blanco	Probando la app antes de tener Firebase configurado	La app ahora revisa si faltan llaves y muestra un aviso explicando qué hacer (captura 1)
Al compilar (`npm run build`) apareció la advertencia "*Some chunks are larger than 500 kB*"	Leyendo la terminal	No es un error, el sitio funciona igual. Se debe a que la librería de Firebase es pesada. Queda anotado como posible mejora

Pruebas realizadas

- `npm run dev` : la app abre en `http://localhost:5173/` ✅
- `npm run build` : compila sin errores ✅
- Sin `.env` : muestra el aviso de configuración en lugar de romperse ✅

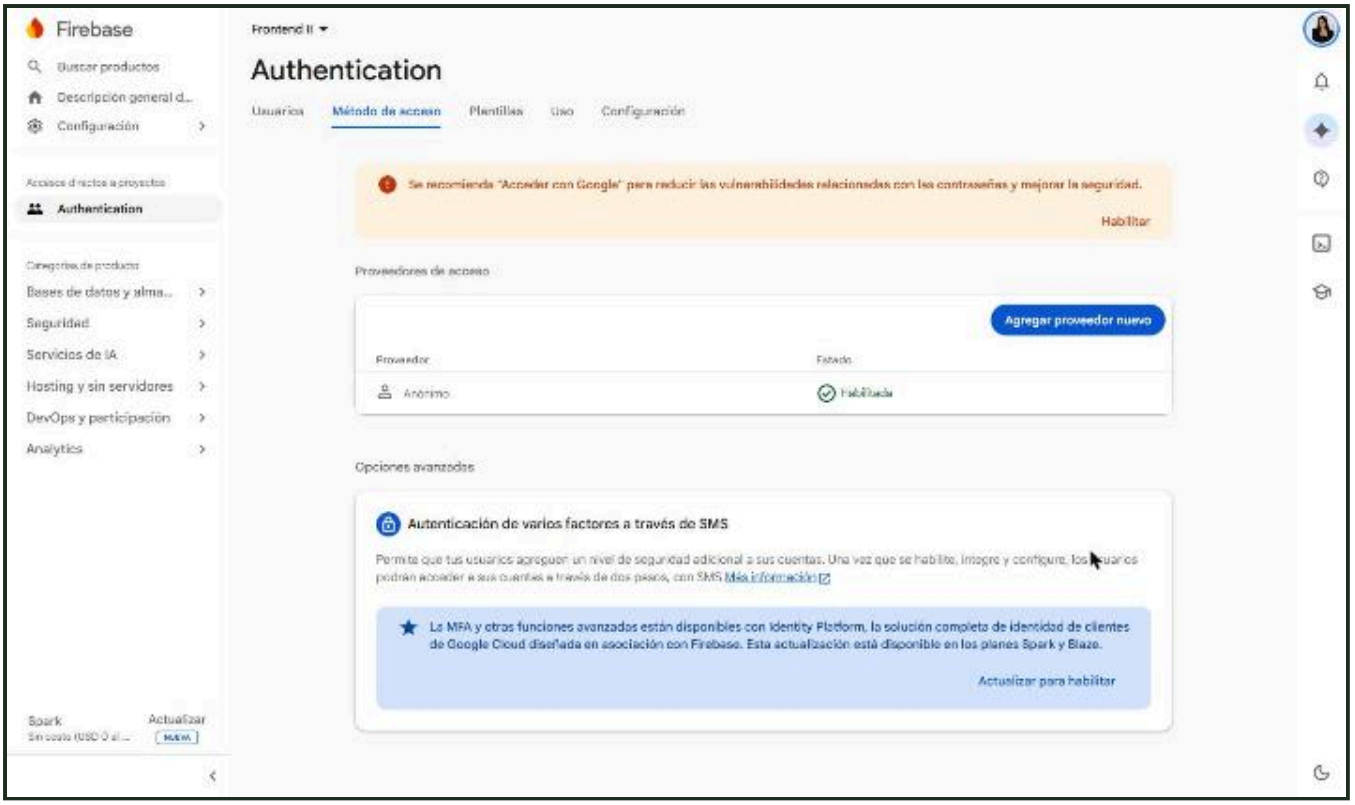


Captura 1: la app corriendo en local antes de conectar Firebase. Avisa qué falta en lugar de mostrar una pantalla en blanco.

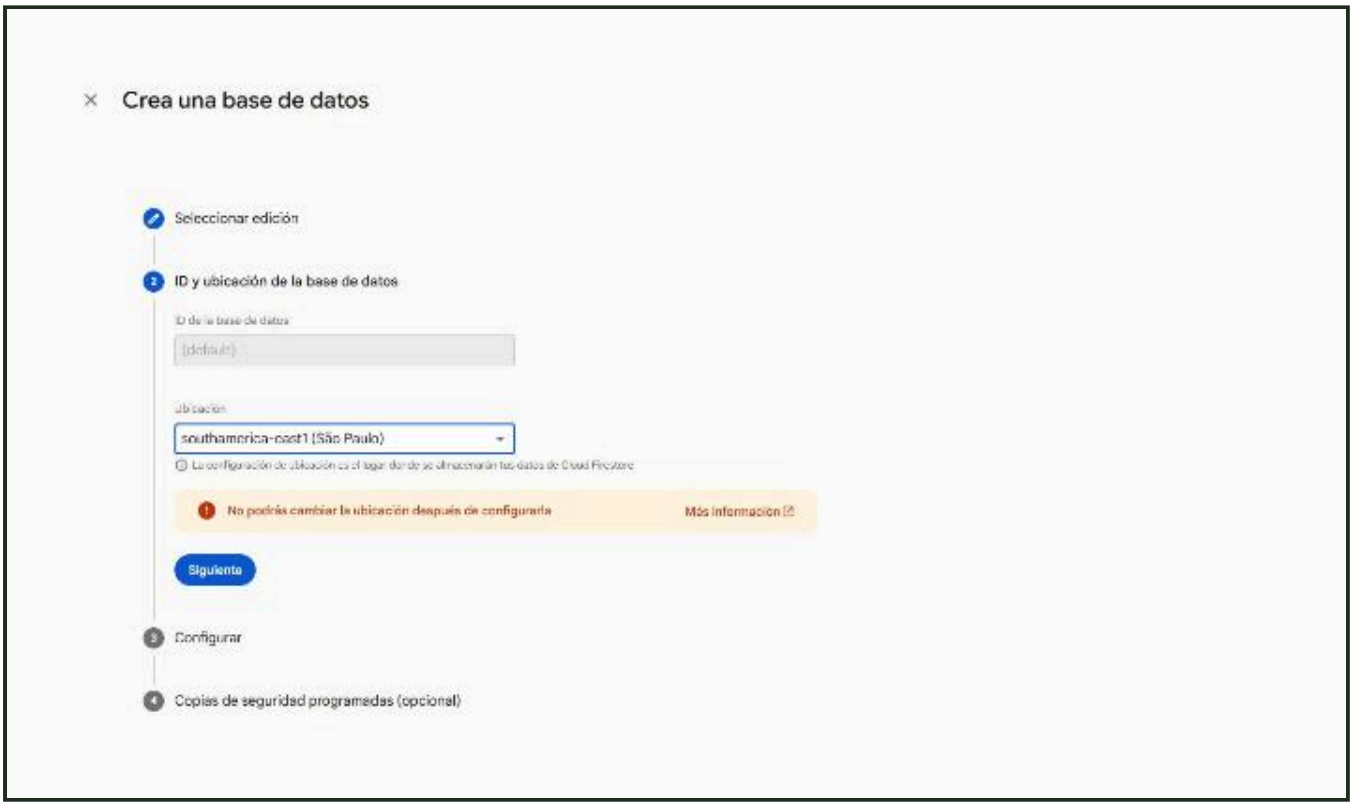
Configuración de Firebase (10/11/2026)

Se hizo desde la consola de Firebase, en el proyecto **Frontend II**, siguiendo las etapas 2 y 4 de la consigna:

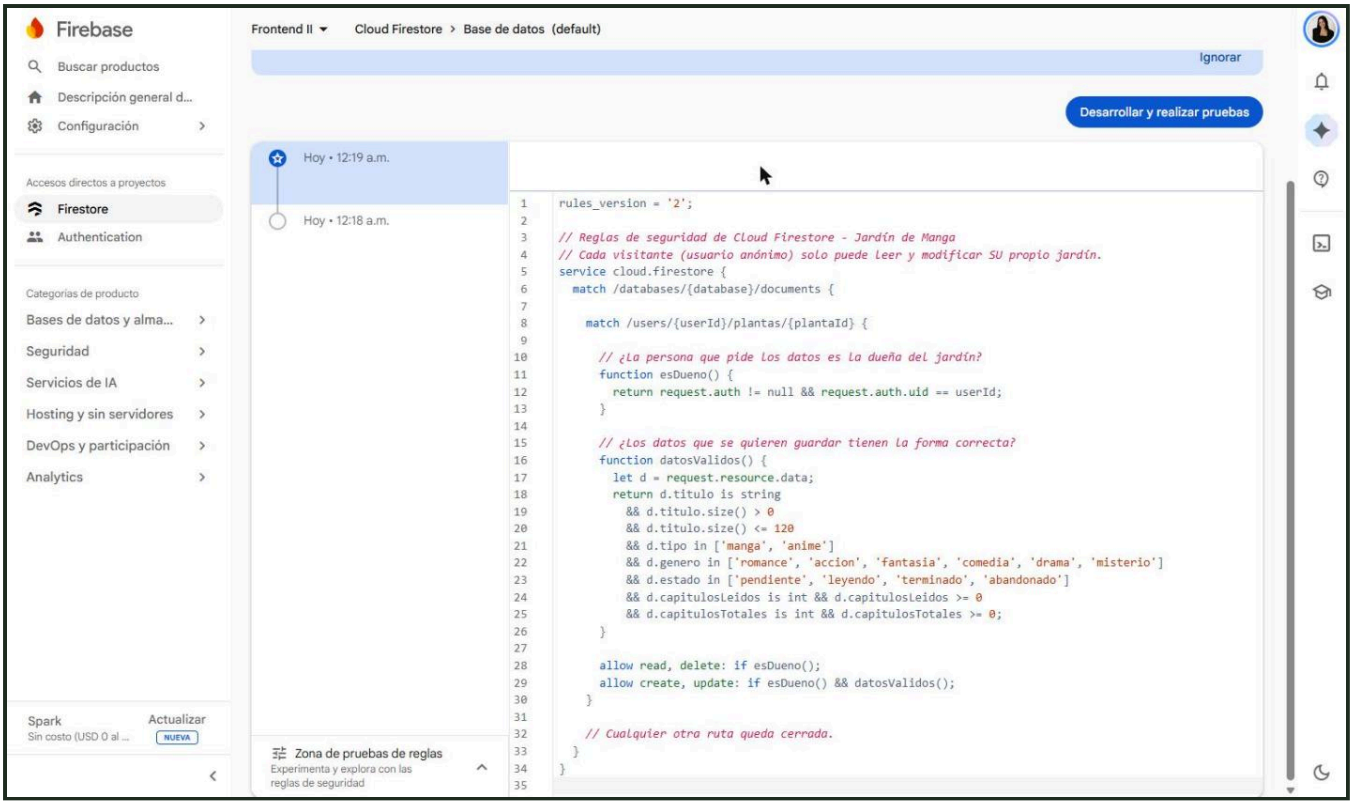
1. **Se registró la app web** (botón `</>`) con el nombre `jardin-de-manga`. Firebase entregó el objeto `firebaseConfig` con las llaves de conexión. No se activaron Hosting ni Analytics porque no hacen falta.
2. **Se habilitó el acceso Anónimo** en *Authentication → Método de acceso*.
3. **Se creó la base Cloud Firestore**: edición Standard, identificador `(default)`, región São Paulo (`southamerica-east1`) y modo producción. Las copias de seguridad programadas quedaron desactivadas porque son de pago.
4. **Se publicaron las reglas** copiando el archivo `firestore.rules` en la pestaña *Reglas*.
5. **Se copiaron las 4 llaves** (apiKey, authDomain, projectId, appId) al archivo `.env`, una por línea y sin comillas ni comas.



Captura 2: el proveedor "Anónimo" habilitado en Authentication.



Captura 3: creación de Firestore con la región São Paulo.



Captura 4: las reglas del proyecto publicadas. A la izquierda se ve el historial con la versión nueva.

Prueba de integración en local (10/11/2026)

Se siguió el paso 5 de la consigna ("Verificar la integración") con `npm run dev`:

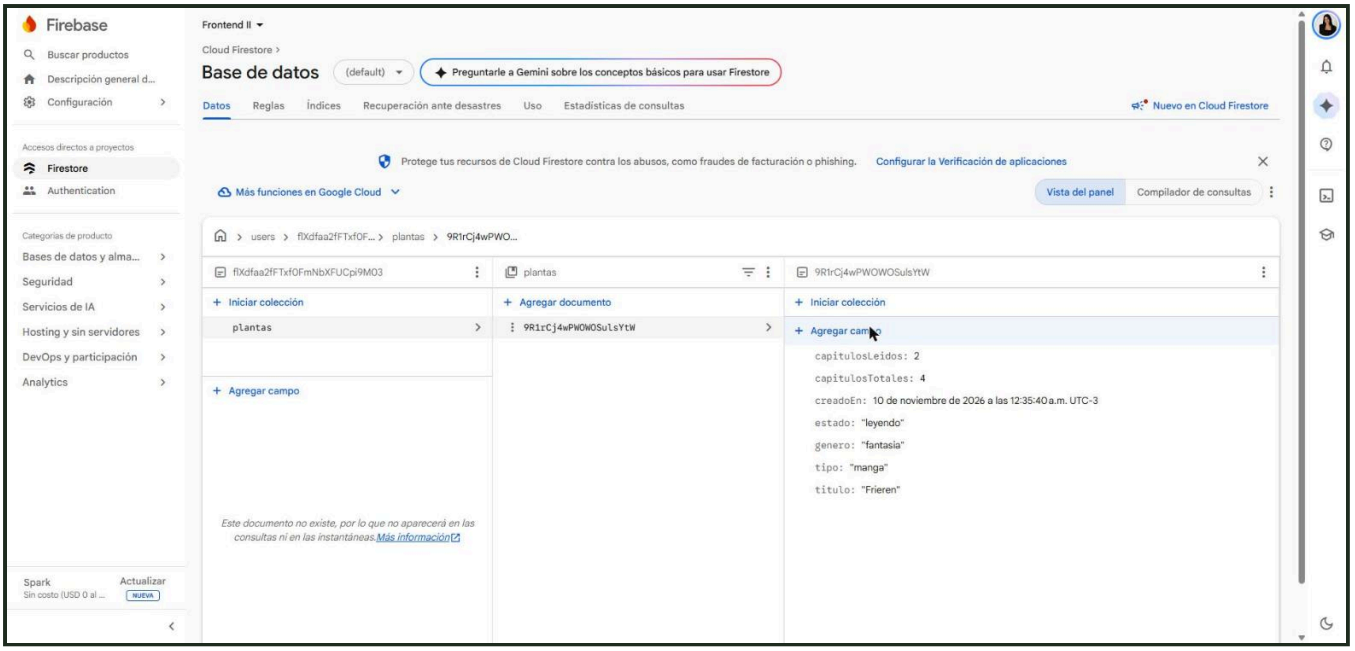
Prueba	Qué se hizo	Resultado
Agregar	Se plantó "Frieren" (manga, fantasía, 4 capítulos)	✅ Aparece como 🫘 semilla, 0/4
Ver en Firestore	Se buscó en *Firestore → Datos*	✅ Está en `users/(uid)/plantas/` con todos sus campos
Modificar	+1 capítulo dos veces y estado "Leyendo"	✅ Pasa a 🌿 "Creciendo", 2/4
Recargar	Se recargó la página (F5)	✅ Los cambios siguen ahí
Ver el cambio en Firestore	Se abrió el documento	✅ `capitulosLeidos: 2`, `estado: "leyendo"`
Eliminar	Se plantó "Prueba para borrar" y se eliminó	✅ Desaparece de la app y de Firestore



Captura 5: el primer manga plantado. El diseño todavía es el básico del MVP.



Captura 6: después de recargar, Frieren sigue con 2/4 capítulos y en "Leyendo / Viendo".



Captura 7: el mismo dato guardado en Firestore. Los valores coinciden con la pantalla.

Más dificultades de esta etapa

Problema	Cómo se detectó	Solución
`npm run dev` daba el error *"la ejecución de scripts está deshabilitada en este sistema"*	Al correr el comando por primera vez en la terminal de VS Code	Es una protección de Windows (PowerShell). Se aplicó el comando que indicó el profesor: `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`
Primero se escribió `run dev` en lugar de `npm run dev`	La terminal respondió que no reconocía "run"	El comando siempre empieza con `npm`, que es el programa que ejecuta los scripts del `package.json`
Al abrir `index.html` con doble clic se veía una página en blanco	Probando abrir el archivo directamente	Una app de React + Vite no se abre así: Vite tiene que "traducir" el código. Se abre con `npm run dev` en `http://localhost:5173`
En el `.env` los valores quedaron con un espacio después del `=`	Revisando el archivo antes de publicar	Se borraron los espacios. La consigna pide copiar solo los valores
Problema de diseño: cuando el texto de la tarjeta pasa a dos líneas (por ejemplo, "Creciendo"), los botones se mueven de lugar y es fácil tocar el equivocado	Haciendo clics seguidos en "+1 capítulo"	Se deja anotado para la v0.2: los botones van a tener un lugar fijo en la tarjeta

Subida a GitHub (10/11/2026)

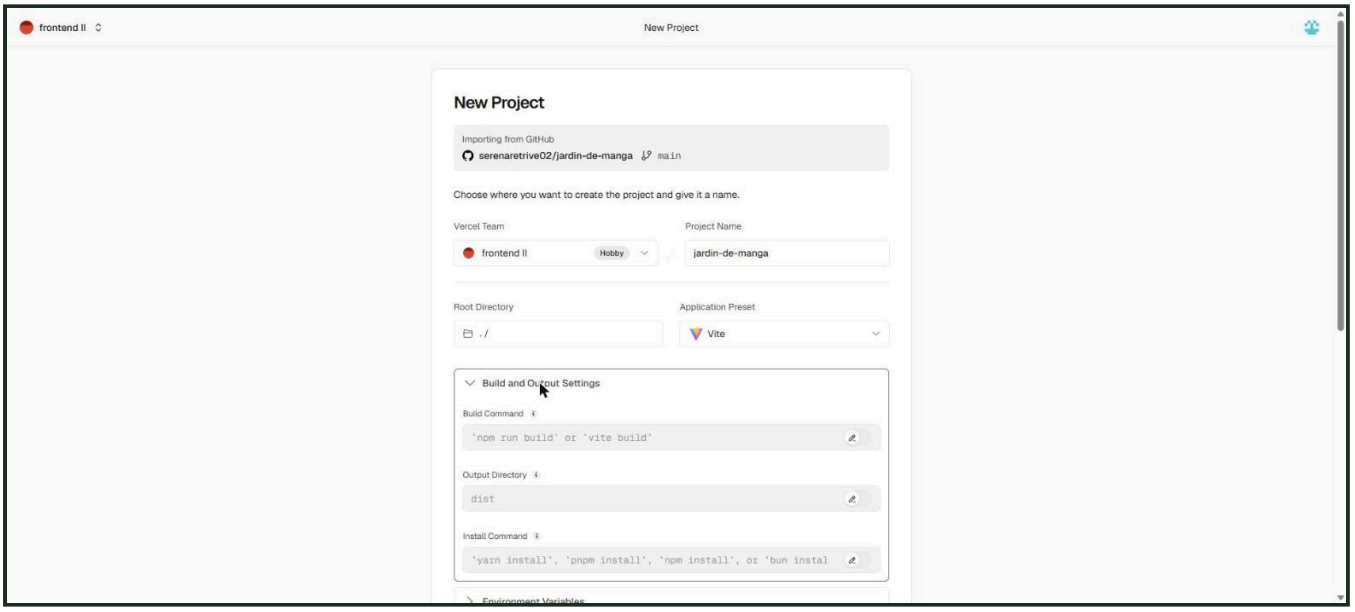
1. Se configuró git con el usuario de GitHub `serenaretrive02`. Como email se usó el email privado que da GitHub (`...@users.noreply.github.com`), para no publicar el email personal.
2. **Antes de subir se revisó qué archivos entraban en el commit** con `git status`, como pide la consigna. Entraron 27 archivos. Estaba `.env.example` y **no** estaban `.env`, `node_modules` ni `dist`.

3. Se creó el repositorio público **jardin-de-manga** en GitHub, sin README ni `.gitignore` automáticos, porque ya teníamos los nuestros.
4. Se subió el código con `git push`.
5. **Verificación:** se intentó abrir el `.env` directamente en GitHub y respondió "no existe" (error 404). Las llaves no quedaron publicadas ✅.

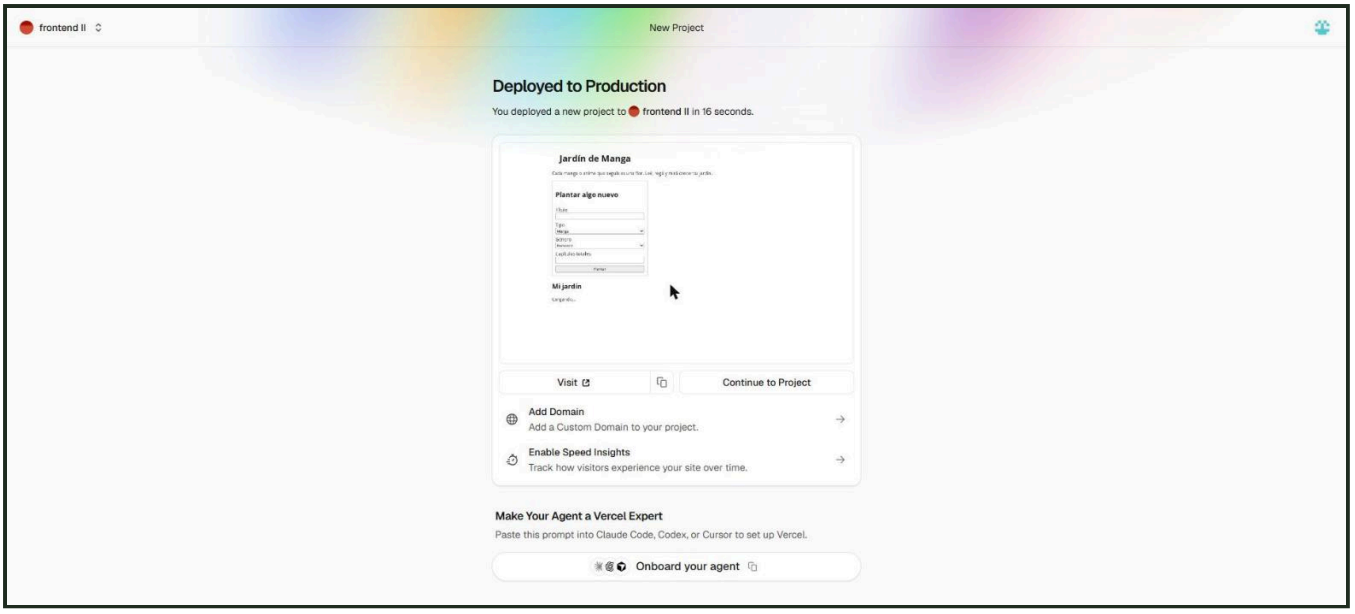
Repositorio: https://github.com/serenaretrive02/jardin-de-manga

Publicación en Vercel (10/11/2026)

1. Se inició sesión en Vercel con la cuenta de GitHub (plan Hobby, gratuito).
2. Se importó el repositorio `jardin-de-manga`. Vercel detectó solo que es un proyecto **Vite**.
3. Se revisó la configuración que pide la consigna: Root Directory `./` (donde está `package.json`), Build Command `npm run build` y Output Directory `dist`.
4. Se cargaron las 4 variables `VITE_FIREBASE_...` en **Environment Variables**, para Production y Preview y **sin** marcarlas como "Sensitive", porque son la configuración pública de Firebase.
5. Se hizo clic en **Deploy**. El build terminó en 16 segundos sin errores.



Captura 8: Vercel detecta Vite, `npm run build` y la carpeta `dist`.



Captura 9: "Deployed to Production". La vista previa ya muestra la app.

URL publicada: https://jardin-de-manga.vercel.app

Pruebas en la versión publicada (10/11/2026)

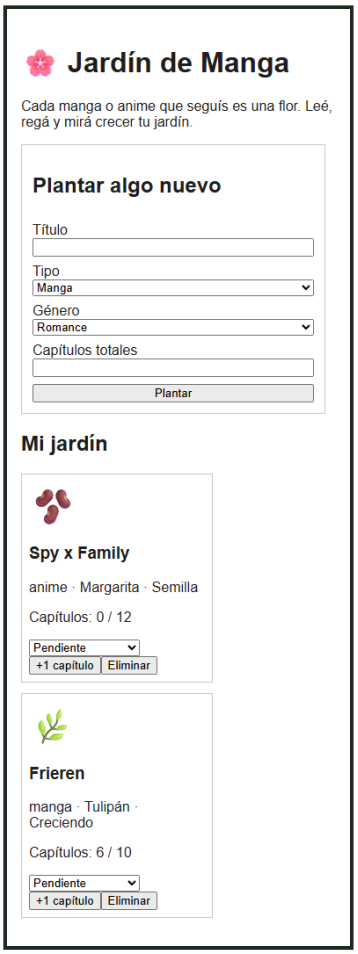
Se repitieron las pruebas desde la URL de Vercel (paso 8 de la consigna):

Prueba	Resultado
La app abre y se conecta a Firebase	✅ Muestra "Tu jardín está vacío" sin errores
Agregar "Spy x Family" (anime, comedia, 12 capítulos)	✅ Aparece como 🫘 semilla
+1 capítulo y estado "Terminado"	✅ Pasa a 🌸 "En flor"
Recargar la página	✅ Sigue en flor, 1/12, terminado
Agregar y eliminar "Prueba para borrar"	✅ Desaparece y no vuelve al recargar
Pantalla de compu (1280 px)	✅ Se usa bien (captura 10)
Pantalla de celular (390 px)	✅ Se puede usar, pero con detalles para mejorar (captura 11)

> **Algo que llamó la atención:** en la versión publicada el jardín empezó vacío, aunque en local ya había un manga plantado. No es un error: el acceso es anónimo, y Firebase toma a `localhost` y a `vercel.app` como dos visitantes distintos, así que cada uno tiene su propio jardín. La consigna lo explica en la etapa 4.



Captura 10: la versión 0.1 publicada en Vercel, vista en compu.



Captura 11: la misma versión en un celular (390 px de ancho).

Un error al actualizar el repositorio (y cómo se corrigió)

Al subir la documentación final de la v0.1, **el archivo `.env.example` se borró del repositorio**. Al preparar el `.env`, el archivo de ejemplo se había renombrado en lugar de copiarse. Git interpretó que se había eliminado y el commit subió esa eliminación.

- **Cómo se detectó:** revisando la lista de cambios del commit (`git status`), donde aparecía `D .env.example` (D = *deleted*, eliminado).
- **Cómo se resolvió:** se recuperó el archivo desde un commit anterior y se subió de nuevo con un commit nuevo ("fix: restaurar .env.example").
- **Verificación:** en GitHub `.env.example` vuelve a estar disponible y `.env` sigue sin publicarse.
- **Aprendizaje:** el archivo `.env` se crea **copiando** `.env.example`, no renombrándolo. Y conviene leer siempre la lista de cambios antes de cada commit, no solo la primera vez.

Conclusión de la v0.1

La v0.1 cumple el mínimo de la consigna: está hecha con React + Vite, guarda, consulta, modifica y elimina en Firestore, conserva los cambios al recargar, tiene reglas de seguridad, el código está en GitHub sin las llaves y está publicada en Vercel.

Mejoras detectadas para la v0.2

Lo que se observó al probar esta versión y que se va a trabajar en la próxima:

1. **Identidad visual:** hoy parece un formulario de los 2000. Hace falta una paleta, tipografía e ilustraciones que transmitan "jardín" y "manga".
2. **Celular:** las tarjetas miden 200 px fijos y dejan espacio vacío. Los botones son muy chicos para tocar con el dedo.
3. **Botones que se mueven:** cuando el texto pasa a dos líneas, los botones cambian de lugar.
4. **Mensajes:** no hay confirmación de "¡Plantado!" ni de "Guardado". Solo se ven los errores.
5. **Validaciones:** el título solo se controla como "obligatorio". Faltan avisos claros, por ejemplo para un título muy largo o capítulos negativos.
6. **Eliminar sin confirmar:** un clic por error borra la planta.
7. **Coherencia de datos:** se puede marcar "Terminado" con 1 de 12 capítulos, o sumar capítulos por encima del total.
8. **Emojis:** dependen del sistema de cada dispositivo. Ilustraciones propias darían un diseño más cuidado y coherente.

Versión 0.2 — Identidad visual pixel art y mejor experiencia de uso (10/11/2026)

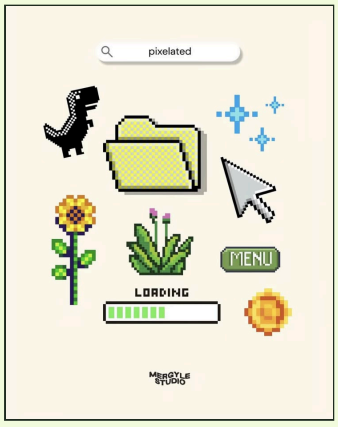
Objetivo de esta versión

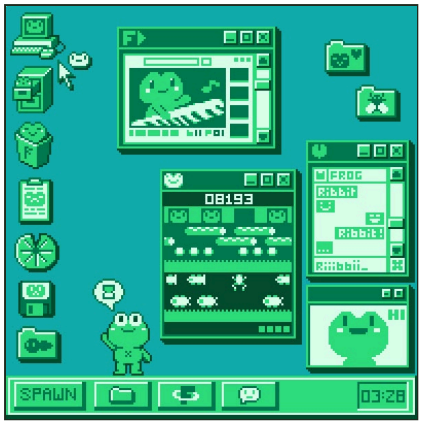
Resolver las mejoras que quedaron anotadas al probar la v0.1. Sobre todo, que la app **tenga identidad propia** y **se use cómodamente en el celular**. En esta versión no se agregan datos nuevos a Firebase: se mejora cómo se ven y cómo se usan los que ya existen.

Decisión de diseño: ¿por qué pixel art?

- **Conecta con el público y con mis gustos.** La estética de videojuego retro (Minecraft, juegos de 8 bits) es familiar para quienes leen manga y miran anime. Convierte "registrar capítulos" en algo parecido a cuidar plantas en un juego.
- **Refuerza la metáfora del jardín.** Cada planta es un pequeño personaje que cambia a medida que avanzás. El pixel art permite dibujar esas etapas de forma simple y reconocible.
- **Es coherente.** Todo (flores, íconos, botones, sombras, tipografía) sigue la misma regla: bordes oscuros, colores planos y sombras "duras" sin difuminar.

Referencias visuales que se usaron (imágenes de inspiración, no son propias):

Referencia	Qué se tomó
	Paleta **verde lima + rosa pastel** y flores en maceta pixeladas
	Íconos con contorno negro y barra de "LOADING" por bloques

Referencia	Qué se tomó
	**Ventanas de computadora retro** con barra de título y botones `_ □ ×`

Sistema visual definido

Elemento	Decisión	Por qué
Colores	Lima `#D7F59A` (principal), rosa `#F9C6CF` (acento), contorno `#1E2B1F`, fondo crema `#FFFBEF`	Pastel y alegre, con buen contraste gracias al contorno oscuro
Tipografías	**Press Start 2P** para títulos y **Pixelify Sans** para textos (Google Fonts)	La primera es muy "arcade", pero cuesta leerla en textos largos. Por eso se usa solo en títulos
Bordes y sombras	Bordes de 3 px y sombras sólidas de 4 px	Imitan los píxeles: nada difuminado
Ventanas	El formulario es `plantar.exe` y el jardín es `mi_jardin.exe`	Cada sección parece un "programa" de una compu retro
Fondo	Cuadrícula de puntitos	Recuerda al papel cuadriculado donde se dibuja pixel art

Las flores, dibujadas con código

Las flores **no son imágenes descargadas**: se dibujaron a mano como **grillas de letras**. Cada dibujo es un *array* de 16 textos de 16 letras, y cada letra es un píxel de un color. El componente `PixelSprite` recorre ese array con `map` (las filas) y otro `map` (las letras), y dibuja un cuadradito por cada letra. Así se aplica lo visto en la guía de *Arrays, map y filter*.

```
'.....KK..KK.....'     K = contorno
'....KPPKKPPK....'     P = pétalo (cambia según el género)
'...KPPWPPPPpK...'     C = centro de la flor
'...KPPPCCPPpK...'     G = tallo, H = hoja
```

La flor es **una sola grilla**, y cada género le cambia los colores de los pétalos y del centro (rosa, girasol, tulipán violeta, margarita, cerezo e hibisco). Así, con 5 dibujos (semilla, brote, creciendo, flor y marchita) se obtienen 10 apariencias distintas.

Iteración del dibujo: en el primer boceto la flor parecía una mancha redonda y el ícono de "pala" no se entendía. Se le marcaron pétalos con muescas y la pala se cambió por un tacho de compost.

Primer boceto	Versión final
	

Cambios en la experiencia de uso

Problema de la v0.1	Solución en la v0.2
Los botones cambiaban de lugar según el largo del texto	Los títulos ocupan como máximo 2 líneas y las acciones quedan siempre al final de la tarjeta
Botones chicos en el celular	Todos los botones miden al menos 44 px de alto, el tamaño recomendado para tocar con el dedo
No había mensajes de éxito	Aparece un **aviso flotante** ("Plantaste...", "Regaste... (3 cap.)", "...volvió a la tierra") que se cierra solo
Validación mínima	Mensajes debajo de cada campo: título vacío o muy largo (máx. 120), capítulos negativos o con decimales. Se muestra un contador "0 / 120"
Se borraba sin confirmar	El botón de compost pregunta **"¿Compostar esta planta?" Sí / No** dentro de la misma tarjeta
Se podían sumar capítulos sin límite	Al llegar al total, el botón cambia a **"Completo"** y se desactiva
"Cargando..." era un texto plano	**Barra de carga por bloques**, como en los videojuegos
No se veía el avance	**Barra de progreso** de 10 bloques en cada tarjeta
—	**Vista previa** en el formulario: muestra qué flor vas a obtener según el género elegido
—	En anime dice "episodios" en lugar de "capítulos"
—	Pie de página con el código de la "jardinera anónima", que ayuda a explicar el acceso anónimo

Antes y después

v0.1	v0.2
	



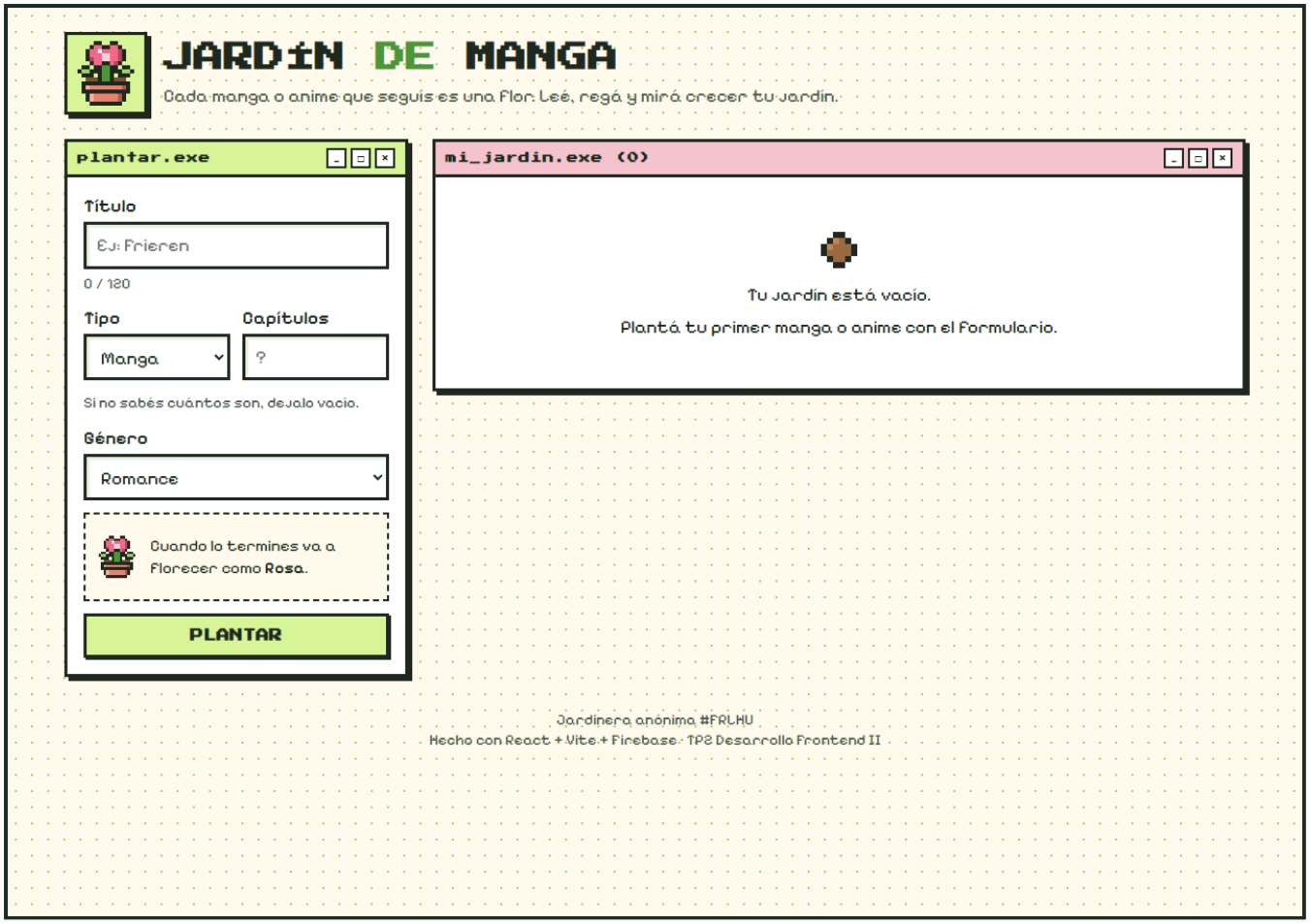
Captura: los errores del formulario se marcan en rojo, debajo de cada campo.



Captura: aviso de éxito después de plantar.



Captura: antes de eliminar, la tarjeta pide confirmación.



Captura: así se ve el jardín cuando todavía no hay plantas.

Diseño para celular: una iteración dentro de la versión

El primer intento para celular ponía **2 plantas por fila**. Al probarlo en una pantalla de 390 px, el texto del estado se cortaba ("Abandono...") y el botón de compost caía a otra línea. Se rediseñó: en el celular cada tarjeta es **horizontal**, con la maceta a la izquierda y los datos a la derecha, una planta por fila.

Primer intento (descartado)	Versión final
	

Uso de IA en esta etapa

- **Instrucción dada:** *"Vayamos con la 0.2. Me gustaría que el diseño sea tipo pixel art si se puede"*, junto con tres imágenes de referencia.
- **Qué hizo la IA:** propuso dibujar las flores como grillas de texto en lugar de usar imágenes, definió la paleta a partir de las referencias, escribió los componentes y probó la app con un navegador automático.
- **Qué revisé y corregí:** las capturas en compu y en celular. A partir de esa revisión se rehicieron el dibujo de la flor, el diseño para celular y el problema de la tipografía.

Dificultades y cómo se resolvieron

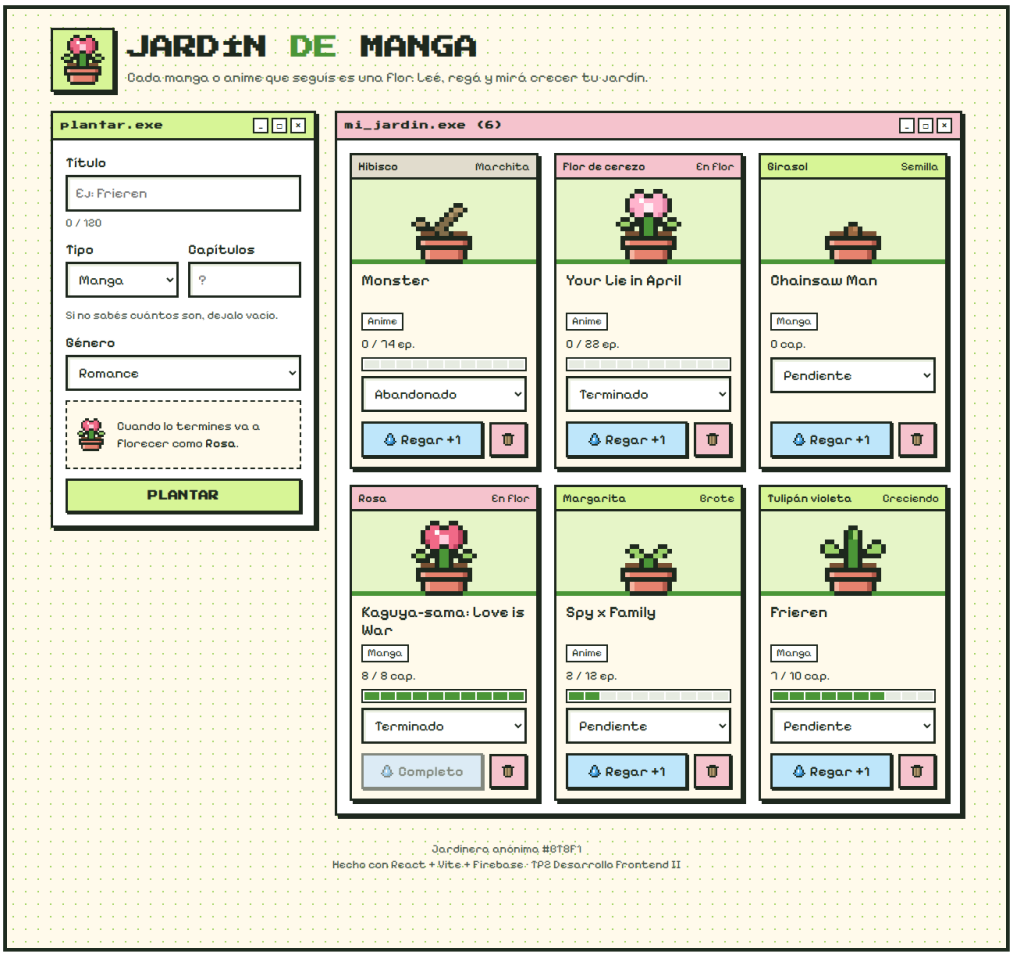
Problema	Cómo se detectó	Solución
La fuente Pixelify Sans dibuja "fl" como un solo símbolo y **"flor" se leía "Aor"**	Mirando la captura del formulario ("va a Aorecer")	Se desactivaron las ligaduras tipográficas con `font-variant-ligatures: none`
En el celular, con 2 columnas, los textos se cortaban	Captura automática a 390 px de ancho	Tarjetas horizontales de una columna (ver arriba)
El primer dibujo de la flor no se reconocía	Mirando la "hoja de sprites"	Pétalos con muescas y brillo blanco
El aviso flotante podía reiniciar su temporizador en cada render	Revisando el código del efecto	Se le pasa el *setter* `setAviso`, que no cambia entre renders. Así el `useEffect` solo se reinicia cuando llega un aviso nuevo (y limpia el anterior con `clearTimeout`)

Pruebas realizadas (en local, con un navegador automático)

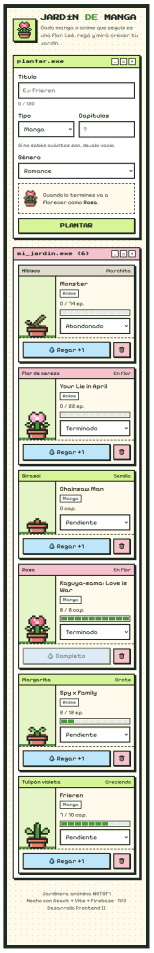
Prueba	Resultado
Enviar el formulario sin título y con capítulos en −3	✅ Aparecen los dos mensajes de error y no se guarda nada
Plantar 6 títulos de distintos géneros	✅ Aviso de éxito en cada uno
Regar hasta el total	✅ El botón pasa a "Completo" y se desactiva
Cambiar estados (terminado, abandonado)	✅ La flor florece o se marchita
Recargar la página	✅ Todo sigue igual: los datos están en Firestore
Pedir compostar	✅ Pregunta antes de borrar
Celular (390 px)	✅ Sin desplazamiento horizontal y con todo legible
Errores en la consola del navegador	✅ Ninguno

Publicación y verificación en Vercel

1. Se revisó la lista de cambios antes del commit (aprendizaje de la v0.1): no había archivos borrados ni `.env`.
2. Se subió a GitHub con la etiqueta **`v0.2`**.
3. **Vercel publicó la versión solo**, sin tocar nada: el deploy apareció como *Ready* a los 6 segundos.
4. Se repitieron **todas las pruebas sobre la URL pública** (validaciones, plantar, regar, estados, recargar, confirmar borrado y celular): todas pasaron y no hubo errores en la consola.



Captura: la v0.2 funcionando en https://jardin-de-manga.vercel.app



Captura: la v0.2 publicada, vista en celular.

Mejoras pendientes para la v0.3

- La fuente de títulos (*Press Start 2P*) no tiene la letra **"Í" mayúscula**, por eso el título se ve "JARDíN". Hay que resolverlo.
- Hoy se puede marcar "Terminado" con 0 capítulos leídos (por ejemplo, *Your Lie in April*: 0/22 y en flor). Habría que ofrecer completar los capítulos.
- Buscar mangas y animes reales con una **API** (portada, sinopsis y cantidad de capítulos automática), usando `fetch` como en la guía de APIs.
- Poder **editar** el título, el género o el total de capítulos de una planta ya plantada.
- Marcar automáticamente "Terminado" al llegar al último capítulo.

Versión 0.3 — Nombre nuevo, búsqueda con API y edición (10/11/2026)

Objetivo de esta versión

Sumar **funcionalidades nuevas**: buscar obras reales en internet, editar plantas y hacer que el estado de cada planta sea coherente con los capítulos. Además, la app pasa a llamarse **Nana**.

La app ahora se llama "Nana"

- "Jardín de Manga" era un nombre provisorio y descriptivo. **Nana** es corto, fácil de recordar y suena a ***hana*** (花), que en japonés significa "flor". Une las dos ideas de la app: el manga y las flores.
- "Tu jardín de manga" queda como **subtítulo**, para que se entienda de qué se trata.
- **Además resolvió un problema pendiente:** la fuente de títulos no tenía la "Í" mayúscula ("JARDíN"). "NANA" no lleva tilde, y los textos con tilde que usaban esa fuente (como "Cargando jardín...") pasaron a la fuente Pixelify Sans, que sí tiene todas las letras.
- El repositorio y la URL siguen siendo `jardin-de-manga`. Cambiarlos rompería los enlaces que ya funcionan.

Nueva función: buscar mangas y animes reales (API)

En el formulario ahora se elige **Manga** o **Anime**, se escribe un nombre y se toca **Buscar**. La app consulta una **API pública** y muestra hasta 6 resultados con portada, año, cantidad de capítulos y la flor sugerida. Al elegir uno, **se completan solos** el título, los capítulos y el género, y se pueden corregir a mano.

Cómo funciona (lo visto en la guía de APIs):

1. Con `fetch` se hace un pedido GET a una dirección como `https://kitsu.io/api/edge/manga?filter[text]=frieren`.
2. Se revisa `respuesta.ok` (fetch no falla solo con un error 404 o 500) y se lee el JSON con `respuesta.json()`.
3. Los resultados vienen en `datos.data`. Esta API usa el nombre `data`, no `message` como la Dog CEO de clase: hay que leer la documentación de cada API.
4. Se transforman con `map` en objetos simples. Con `filter` se descartan las categorías vacías, con `forEach` se arma un diccionario de categorías y con `find` se elige la flor que corresponde.
5. La consulta se hace al tocar **Buscar** (un evento) y no en cada letra, para no hacer pedidos de más ni chocar con el límite de consultas.
6. Hay estados de **carga** ("Buscando en Kitsu..."), **error** ("No se pudo consultar la API. Podés completar los datos a mano") y **sin resultados**. Si el error es el 429 (demasiadas consultas), se pide esperar unos segundos.

Cómo se elige la flor a partir de la API: Kitsu devuelve categorías en inglés ("Romance", "Fantasy", "Action"...). Una tabla las relaciona con nuestros géneros, en orden de prioridad: romance → fantasía →

acción → misterio → comedia → drama. Por ejemplo, *Spy x Family* (Action, Comedy…) se sugiere como **girasol** (acción).



Captura: resultados de la búsqueda "frieren", con portada, año, capítulos y la flor sugerida.



Captura: al elegir un resultado se completan los campos y se avisa que los datos vienen de Kitsu.

Dificultad: la API que habíamos planeado estaba caída

El plan original era usar **Jikan**, una API gratuita que toma los datos de MyAnimeList.

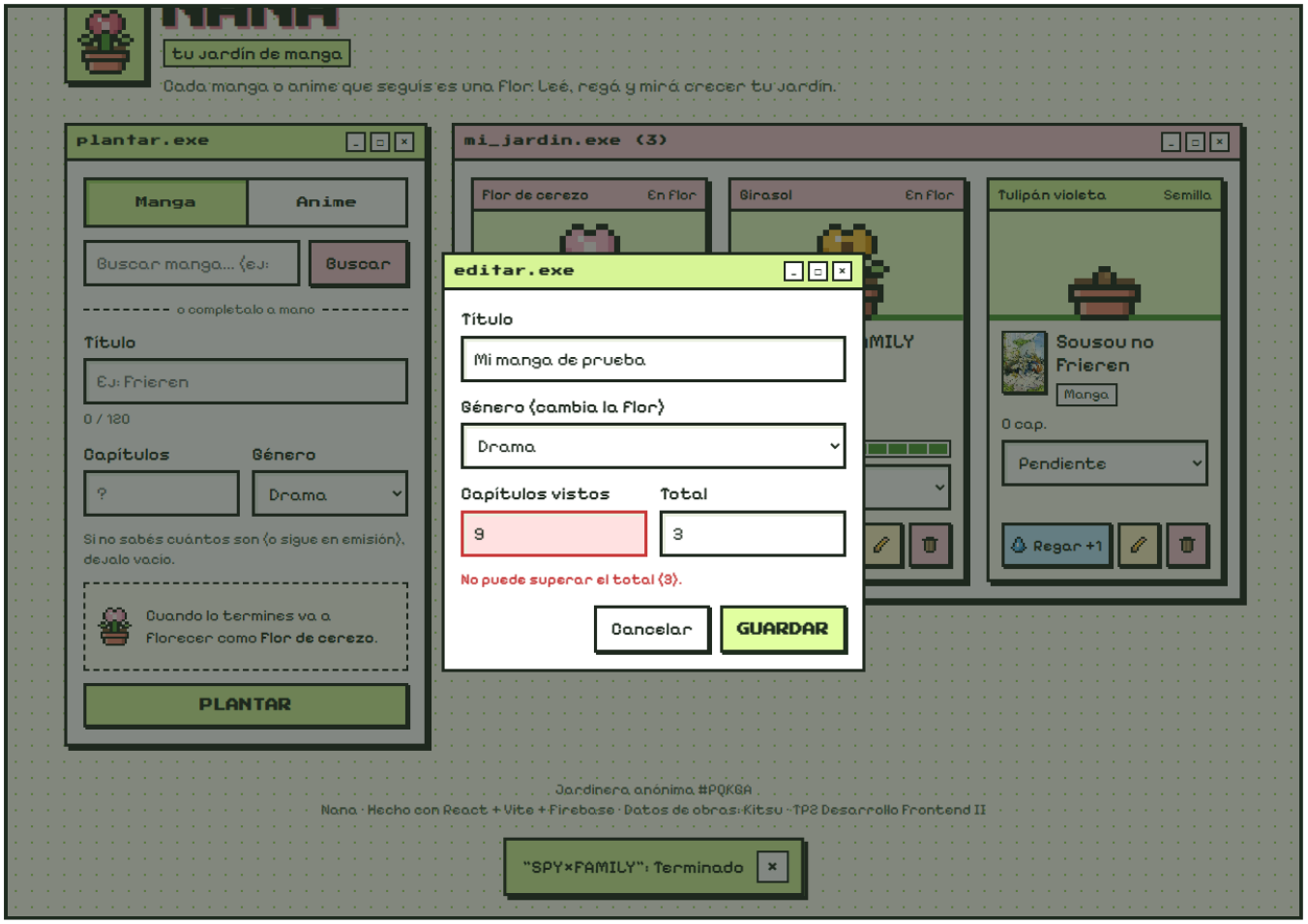
Paso	Qué se hizo	Resultado
1	Se consultó Jikan desde la terminal	*Connection timed out* (no respondía)
2	Se probó en el navegador con `fetch`	`TypeError: Failed to fetch`
3	Se comprobó que otras páginas sí respondían	GitHub respondía bien: el problema era Jikan, no la conexión
4	Se probaron dos alternativas gratuitas y sin clave: **Kitsu** y **AniList**	Las dos respondieron (código 200)
5	Se eligió **Kitsu**	Funciona igual que lo visto en clase: una URL con `fetch` (GET) que devuelve JSON. AniList usa GraphQL, un sistema que no vimos en la materia

Aprendizaje: una app que depende de un servicio externo tiene que funcionar aunque ese servicio falle. Por eso la búsqueda es **una ayuda opcional**: si la API no responde, se avisa y se puede completar todo a mano.

Nueva función: editar una planta

Cada tarjeta tiene un botón con un **lápiz pixelado** que abre la ventana `editar.exe`. Desde ahí se cambian el título, el género (y con él, la flor), los capítulos vistos y el total.

- Valida igual que el formulario de plantar y además **no deja poner más capítulos vistos que el total**.
- Se cierra con *Cancelar*, con la tecla *Escape* o tocando afuera.



Captura: la ventana de edición avisa que los capítulos vistos no pueden superar el total.

Estados coherentes con los capítulos

Situación	Antes (v0.2)	Ahora (v0.3)
Regar por primera vez una planta "Pendiente"	Seguía "Pendiente"	Pasa a **"Leyendo"**
Regar el último capítulo	Solo se bloqueaba el botón	**Florece sola** (Terminado) con el aviso "¡...floreció! Lo terminaste"
Marcar "Terminado" con capítulos pendientes	Quedaba "en flor" con 0/22	**Completa los capítulos** automáticamente
Editar y agregar capítulos a algo "Terminado"	—	Vuelve a **"Leyendo"**



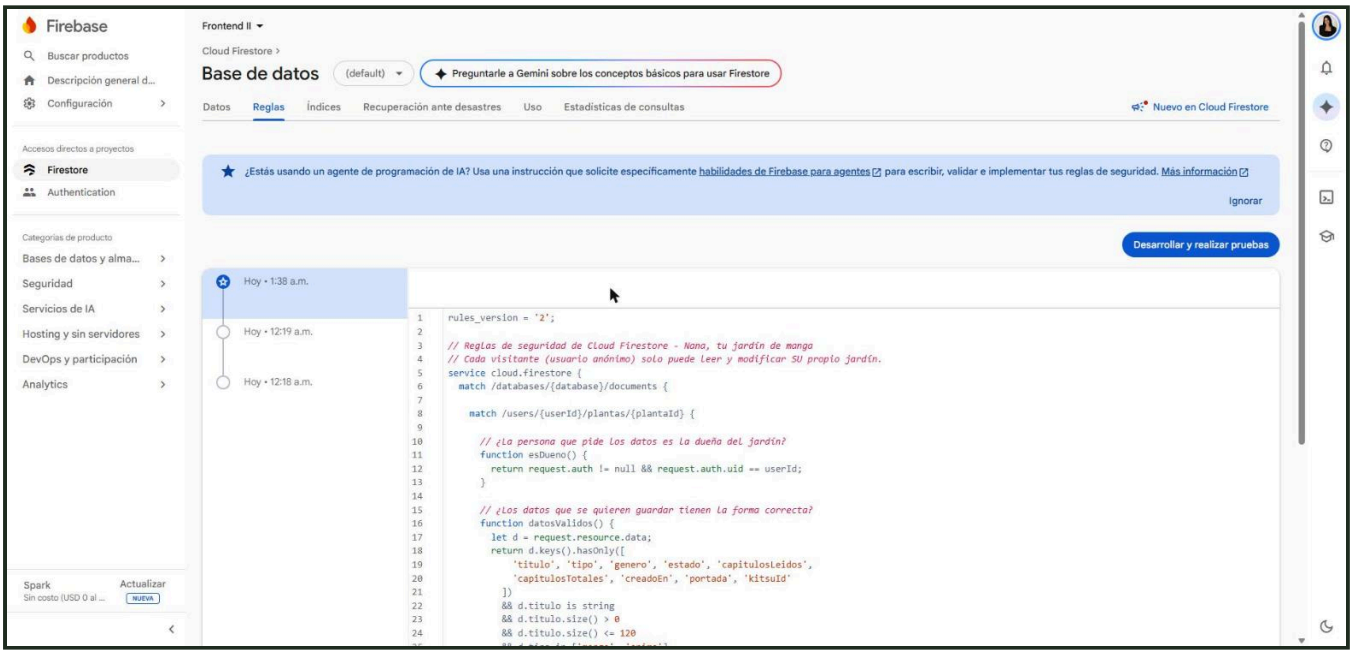
Base de datos: campos nuevos y reglas más estrictas

Cada planta puede guardar dos datos nuevos (solo si vino de la búsqueda):

- `portada`: la dirección de la imagen de tapa;
- `kitsuId`: el número de la obra en Kitsu.

Se actualizaron las **reglas de Firestore** y se publicaron de nuevo en la consola de Firebase (desplegar en Vercel no las actualiza):

- **Solo se aceptan los campos conocidos** (`hasOnly`). Si alguien intenta guardar un campo inventado, Firebase lo rechaza.
- La portada tiene que ser un enlace que empiece con `https://`.
- Los capítulos leídos **no pueden superar el total**. Ahora lo controla también la base de datos, no solo la interfaz.



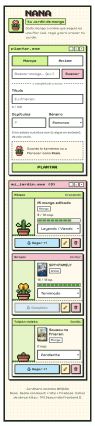
Captura: reglas publicadas. A la izquierda se ve la versión nueva (1:38) junto a las anteriores.

Uso de IA en esta etapa

- **Instrucción dada:** *"Sí [seguimos con la v0.3]. Cambiemos el título a Nana."*
- **Qué hizo la IA:** investigó por qué Jikan no respondía y comparó alternativas, escribió el servicio de búsqueda, la edición y las reglas nuevas, y armó una prueba automática de 17 pasos.
- **Qué se revisó:** las capturas de cada prueba. Gracias a eso se encontraron los errores de la tabla siguiente.

Dificultades y cómo se resolvieron

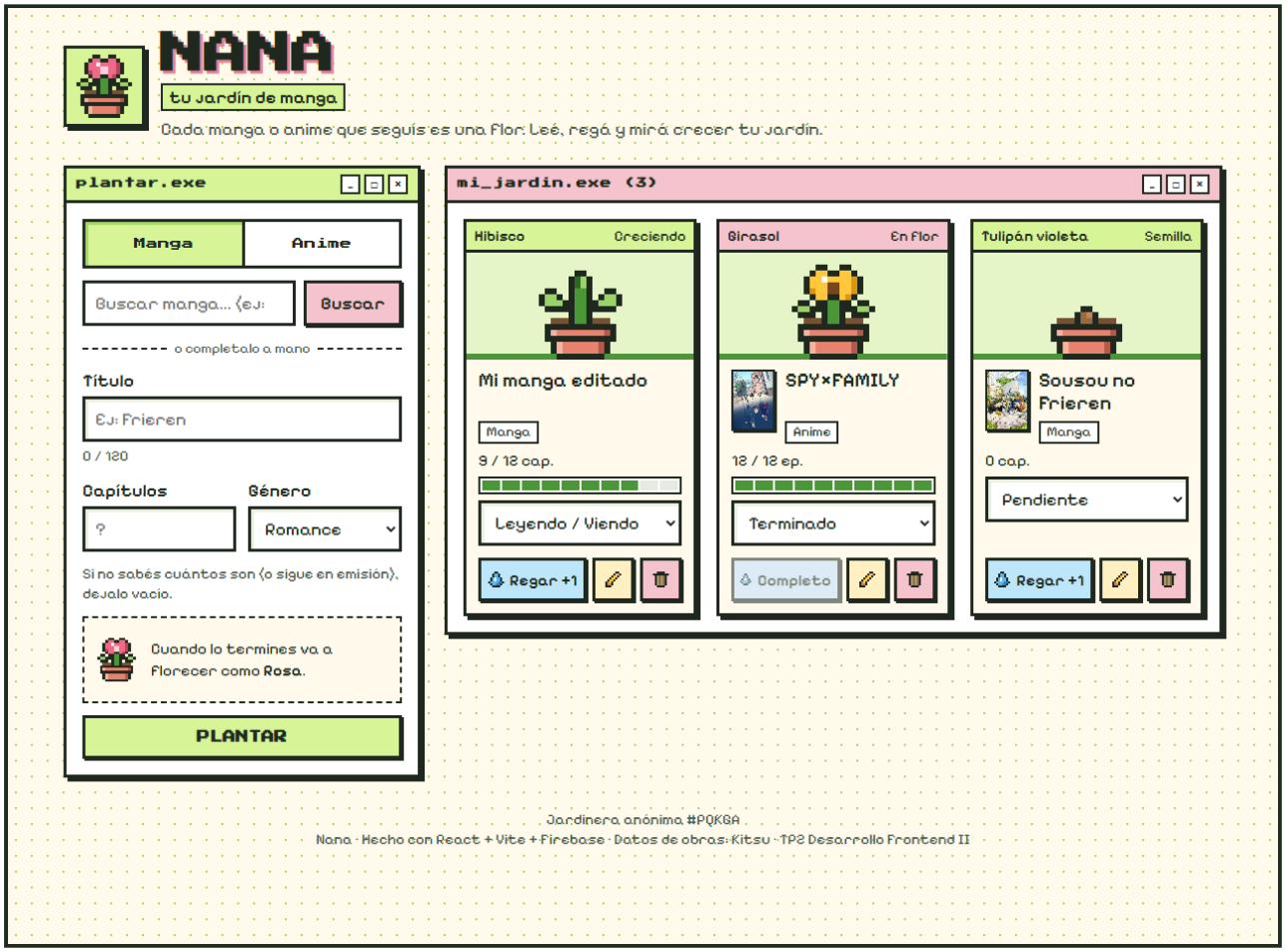
Problema	Cómo se detectó	Solución
La API Jikan no respondía	Pruebas desde la terminal y desde el navegador	Se cambió a Kitsu (ver arriba)
Si una obra de Kitsu no tiene portada, se mandaba un texto vacío y **la regla nueva lo rechazaba**	Releyendo las reglas mientras se publicaban	La portada solo se guarda si existe
En el celular, el formulario **se salía de la pantalla** (409 px en una pantalla de 390 px)	La prueba automática de "sin desplazamiento horizontal" falló. Un diagnóstico mostró que era la ventana `plantar.exe`	En CSS Grid, `1fr` no deja que una columna sea más angosta que su contenido. Se cambió por `minmax(0, 1fr)`
Al editar el total de una planta "en flor", seguía en flor con 9/12	Mirando la captura del jardín después de editar	Al guardar una edición se recalcula el estado
Con el botón de editar, el de compost bajaba a otra línea	Captura de la compu	Los tres botones quedan en una fila y "Regar" se adapta al espacio
El texto de ejemplo del buscador se cortaba en el celular	Captura del celular	Se acortó a "Ej: Frieren"
La prueba automática apretaba "Buscar" en vez de "Plantar"	La prueba se quedaba esperando	Había dos botones de envío. Se indicó cuál era el del formulario. Era un problema de la prueba, no de la app

Antes	Después
	

Pruebas realizadas (en local, prueba automática de 17 pasos)

Prueba	Resultado
El título dice NANA	✅
Buscar "frieren" en Kitsu devuelve resultados	✅
Al elegir, se completan el título y el aviso "Datos traídos de Kitsu"	✅

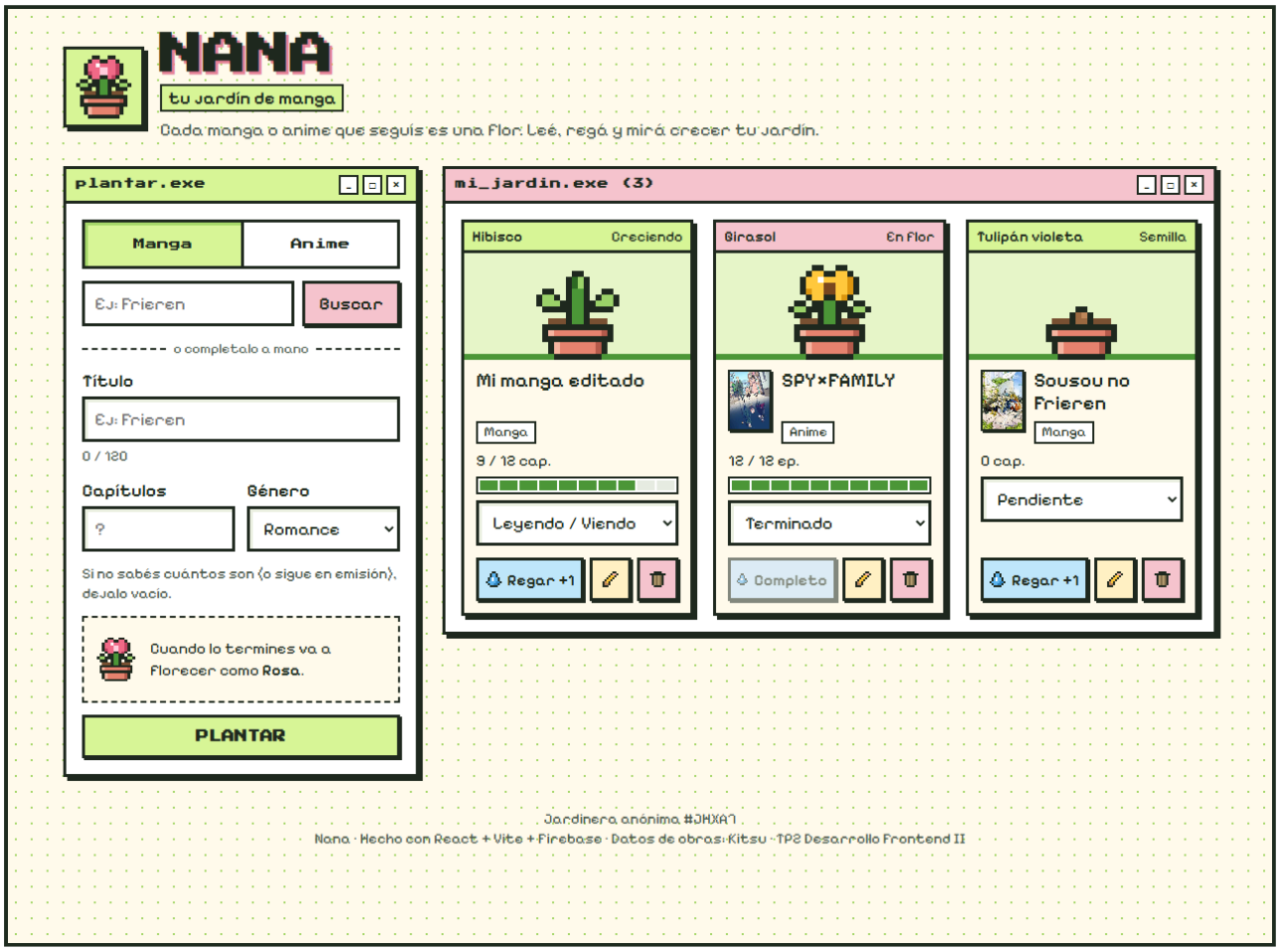
Prueba	Resultado
La tarjeta muestra la portada	✅
Buscar el anime "spy x family": completa 12 episodios y sugiere girasol (acción)	✅
Regar por primera vez pasa a "Leyendo"	✅
Regar el último capítulo hace florecer la planta	✅
Marcar "Terminado" completa los episodios (12/12)	✅
Editar no deja poner 9 capítulos vistos de un total de 3	✅
Editar guarda el título, el total y el género, y cierra la ventana	✅
Después de editar el total, la planta vuelve a "Leyendo"	✅
Después de recargar, todo sigue igual (incluidas las portadas)	✅
Celular de 390 px sin desplazamiento horizontal	✅
Errores en la consola	✅ Ninguno



Captura: la v0.3 en la compu, con portadas, botón de editar y estados coherentes.

Publicación y verificación en Vercel

1. **Primero se publicaron las reglas nuevas en Firebase y después el código.** En el orden inverso, la versión publicada habría intentado guardar portadas que las reglas viejas rechazaban.
2. Se revisó la lista de cambios (24 archivos, sin borrados ni `.env`) y se subió con la etiqueta `v0.3`.
3. Vercel publicó la versión solo, en 5 segundos (*Ready*).
4. Se repitió la **prueba automática de 17 pasos sobre la URL pública**: 17/17 correctas y ningún error en la consola.



Captura: la v0.3 funcionando en https://jardin-de-manga.vercel.app



Captura: la v0.3 publicada, vista en celular.

Ideas para la v0.4

- **Buscar y filtrar dentro del jardín** (por estado, género o nombre), aplicando `filter` como en la guía.
- **"Mi ramo"**: estadísticas del jardín (cuántas flores, capítulos leídos en total, género favorito).
- **Significado de cada flor** según el *hanakotoba* (el lenguaje japonés de las flores).
- Ordenar el jardín (más recientes, por avance, por nombre).

Versión 0.4 — Comunidad: clubes, misiones con roles y eventos (10/11/2026)

Revisión previa: ¿cumplimos la consigna y la rúbrica?

Antes de empezar se revisó el proyecto contra cada punto de la consigna y de la rúbrica:

Criterio	Estado
A · React + Vite con interacciones	✅
A · Firebase: consultar, agregar, modificar, eliminar y persistir	✅
A · GitHub con los archivos para ejecutar	✅ 31 archivos. `.gitignore` excluye `.env`, `node_modules` y `dist`. Ninguna llave en el código
A · Vercel con conexión a Firebase	✅
B · README, `.env.example`, reglas, informe con IA y dificultades	✅
B · Entrega organizada con enlaces	⚠️ Faltaba un documento de entrega → se creó `entrega.md`
C · Originalidad y complejidad	Se refuerza en esta versión con la comunidad
Al menos 3 versiones	✅ v0.1, v0.2, v0.3 (con etiquetas en GitHub)
Mini video de demostración	⏳ Pendiente (lo graba la autora)

Objetivo de esta versión

La app era solo un registro personal: plantar, regar y nada más. Para darle una **vuelta de tuerca** y **sumar acciones con sentido**, Nana pasa a ser también una **comunidad de lectura**. Siguiendo lo trabajado en clase (diagrama de casos de uso y requisitos), se definieron:

- **CRUD de grupos:** clubes de lectura.
- **CRUD de tareas de distintos tipos:** misiones (leer, ver episodios, reseña, fan art, teoría, recomendar).
- **Requisito de rol:** la administradora es el único rol que puede asignar tareas.
- **CRUD de eventos:** lecturas conjuntas, watch parties, debates, intercambios.
- **Compartir** tareas, eventos e invitaciones.

El detalle (requisitos, criterios de aceptación y diagrama de casos de uso) está en `especificacion.md`.

Cómo funciona (explicado simple)

1. **Perfil:** la primera vez que se entra a **Comunidad**, se elige un **apodo** y una **flor-avatar**.

2. **Crear un club:** quien lo crea es su **administradora**. Cada club tiene un **código de 6 letras** (por ejemplo `DAAUN4`).
3. **Invitar:** el botón *Invitar* ↗ abre el menú de compartir del celular (o copia el texto en la compu) con un enlace como `…vercel.app/?club=DAAUN4`. Al abrirlo, la app va directo a Comunidad con el código ya escrito.
4. **Misiones (tareas):** la administradora crea una misión, elige el **tipo**, se la **asigna** a un miembro y puede ponerle fecha límite. Se ven en un **tablero Kanban** con tres columnas: *Por hacer · Haciendo · Hecha*. Cada persona mueve **solo sus** misiones. Las vencidas se marcan en rojo.
5. **Eventos:** cualquier miembro propone uno (fecha, lugar o enlace, descripción). Los demás tocan **¡Voy!** y se ve quiénes van. Los eventos pasados se muestran aparte y atenuados.
6. **Salir o eliminar:** un miembro puede salir del club. La administradora puede quitar miembros o eliminar el club entero, junto con sus misiones y eventos.

El requisito más importante: solo la administradora asigna tareas

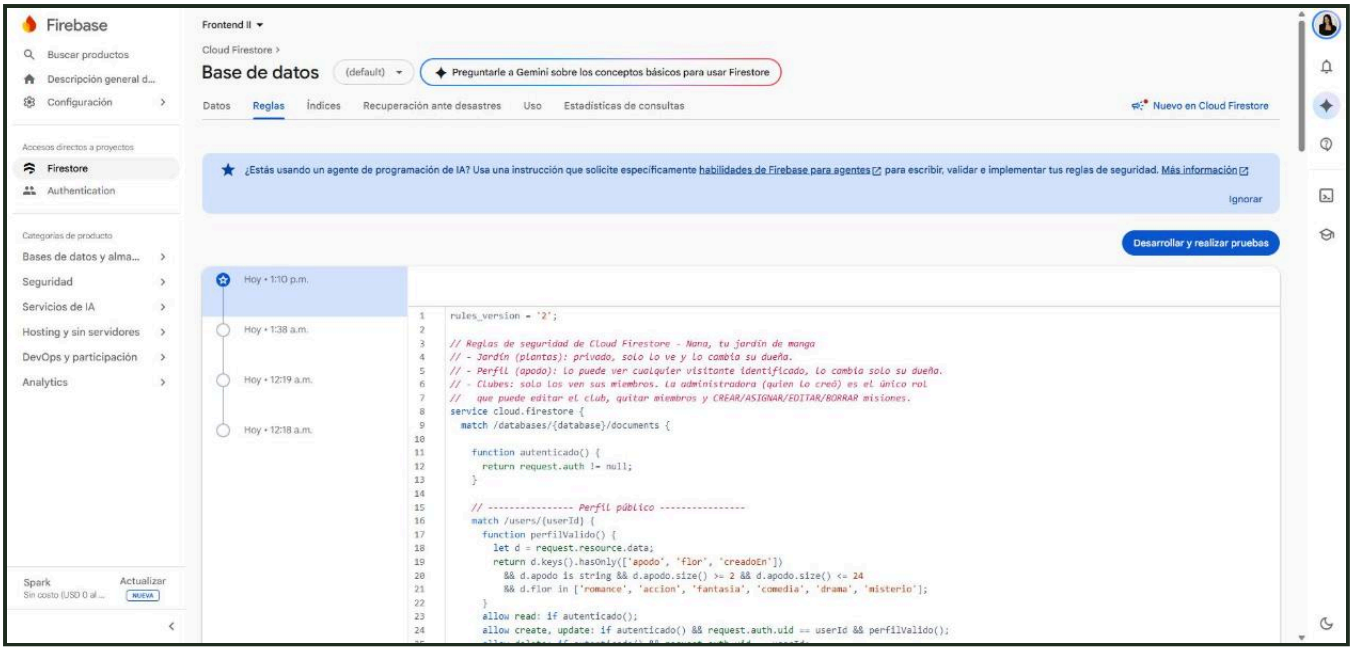
No alcanza con **esconder el botón**: alguien con conocimientos podría mandar el pedido directo a Firebase. Por eso la regla se aplica **en dos lugares**:

- **En la interfaz:** los miembros no ven "+ Nueva misión", y en su lugar dice "Solo la administradora asigna misiones".
- **En las reglas de Firestore:** solo quien figura como `adminUid` del club puede crear, editar o borrar misiones. La persona asignada solo puede cambiar el campo `estado` de **su** misión.

Prueba de seguridad: se simuló a una usuaria (Bea, miembro) que intenta crear una misión y cambiar el nombre del club **llamando directamente a la API de Firestore**, sin pasar por la app. Firebase respondió **403 (prohibido)** en los dos casos ✅.

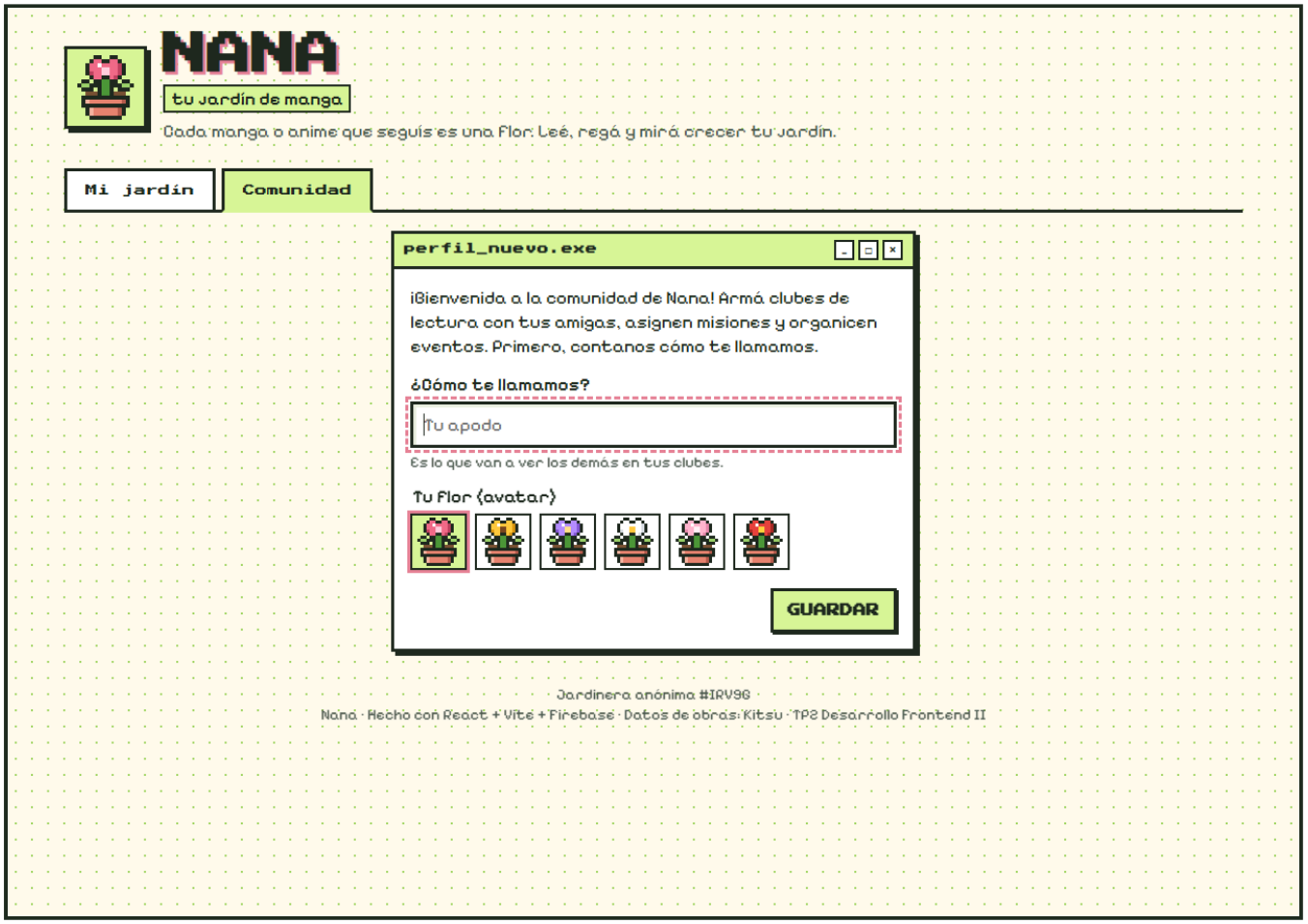
Las reglas también controlan que:

- para unirse, solo se pueda **agregar el propio** usuario a la lista de miembros, y para salir, solo **sacarse a una misma**;
- para anotarse a un evento, solo se pueda tocar **el propio nombre** en la lista de asistentes;
- los clubes solo los puedan ver **sus miembros** (máximo 30), y todos los datos tengan la forma correcta.

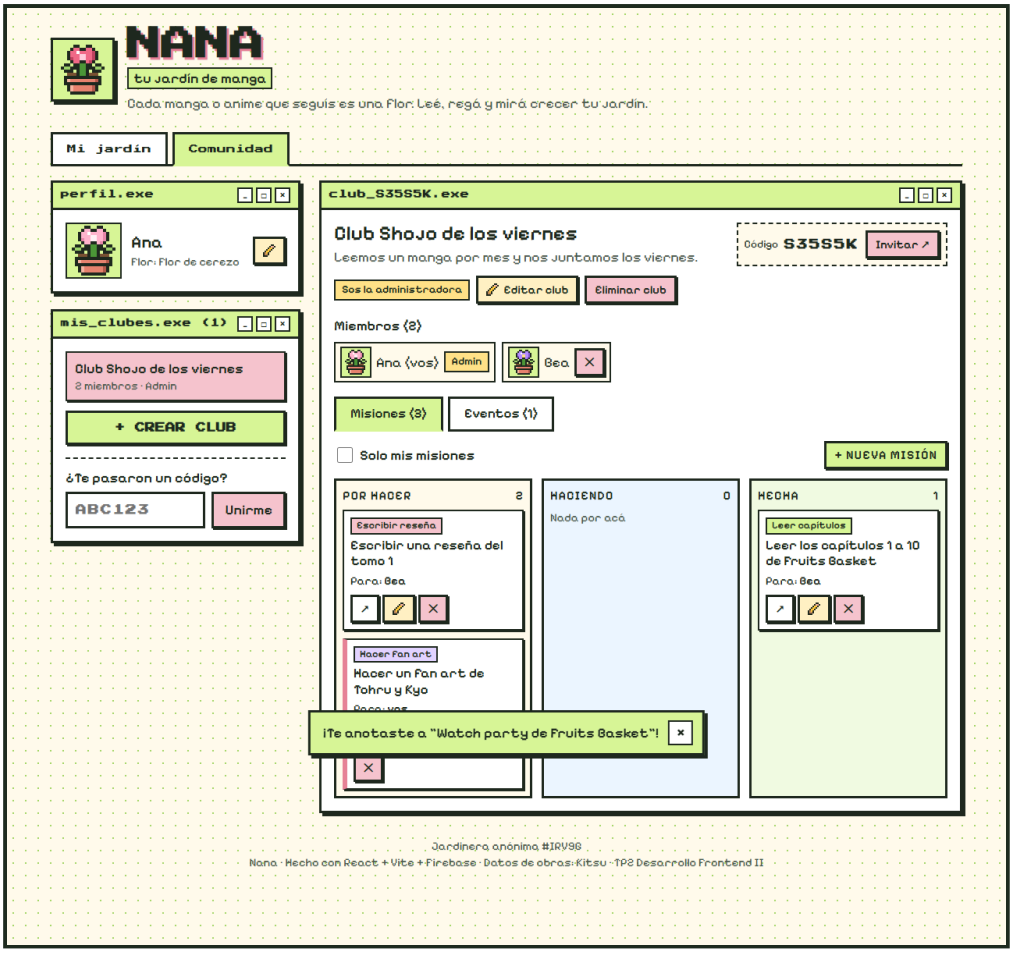


Captura: reglas publicadas en Firebase (versión de las 13:10).

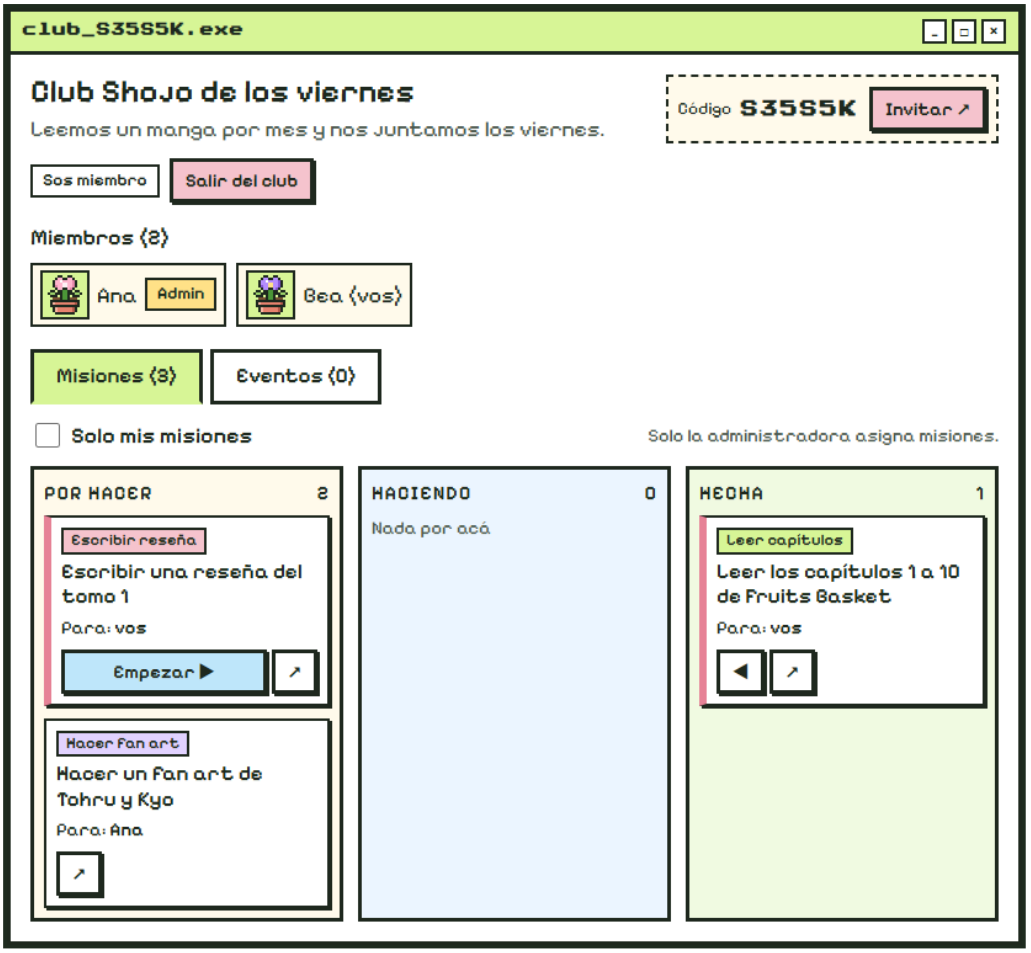
Capturas



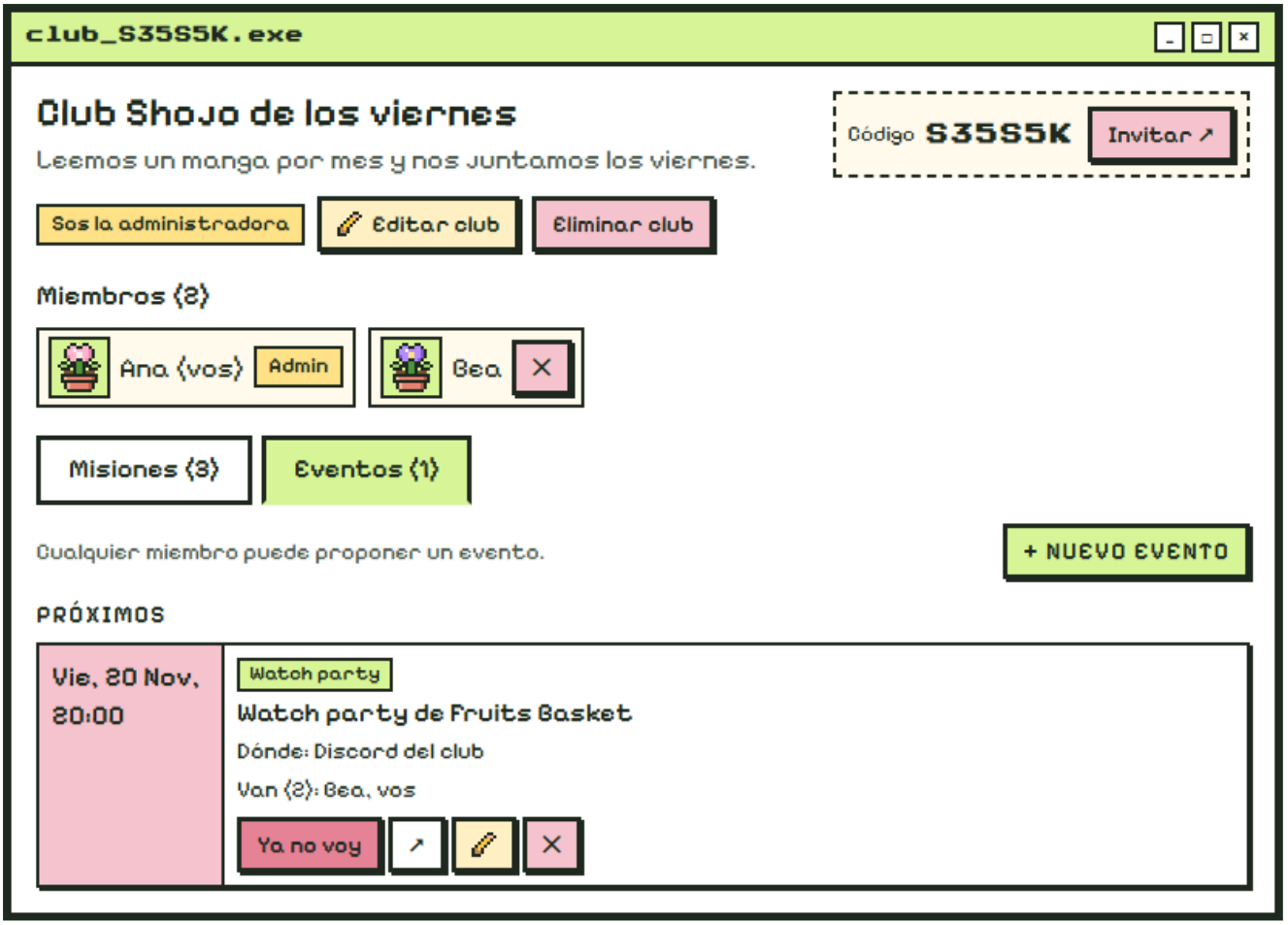
La primera vez se elige un apodo y una flor-avatar.



El club visto por la administradora: código e invitación, miembros con su insignia y tablero de misiones de distintos tipos.



El mismo club visto por Bea (miembro): no puede asignar misiones, pero mueve las suyas (marcadas con un borde rosa).



Eventos: Bea propuso una watch party y Ana se anotó.



La comunidad en el celular: todo en una columna y el tablero apilado.

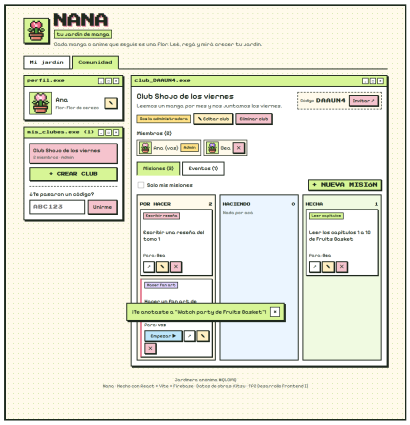
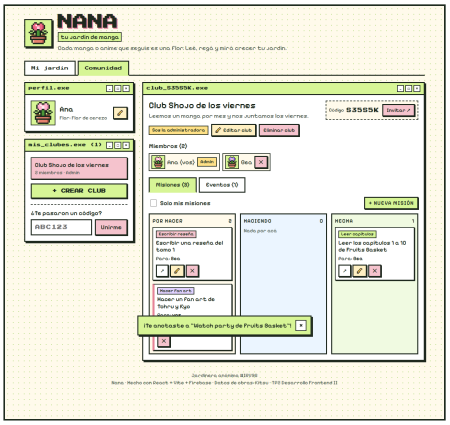
Uso de IA en esta etapa

- **Instrucción dada (resumen):** *"Chequeá que se cumpla todo lo que pide la consigna y la rúbrica para sacarnos un 10. Quiero sumar más acciones: una comunidad, añadir amigos o grupos de lectura. Que el usuario pueda crear, modificar, eliminar y compartir tareas; CRUD de eventos y de grupos; tareas de distintos tipos; el administrador es el único rol que puede asignar tareas; distintos tipos de iteraciones ágiles."*
- **Qué hizo la IA:** auditó el repositorio contra la rúbrica, diseñó el modelo de datos y las reglas por rol, programó 4 servicios y 12 componentes nuevos, y armó una prueba automática con **dos usuarias simultáneas**.
- **Qué se revisó y se corrigió** a partir de las capturas y las pruebas: los 7 problemas de la tabla siguiente.
- **Decisión propia:** en lugar de una lista de "amigos" sueltos, la comunidad se organiza en **clubes**. Cumple la misma necesidad (leer con amigas), es más fácil de entender y permite roles claros.

Dificultades y cómo se resolvieron

Problema	Cómo se detectó	Solución
Cómo unirse a un club sin poder verlo antes. Las reglas solo dejan ver un club a sus miembros	Al diseñar las reglas	El **código es el id del documento**: para unirse alcanza con conocerlo. Las reglas permiten un solo cambio: agregarse a una misma

Problema	Cómo se detectó	Solución
El aviso "Misión actualizada" aparecía **antes** de que el tablero mostrara el cambio	La prueba automática leía el tablero viejo	Ahora primero se vuelven a consultar las misiones y eventos, y **recién después** se avisa
Un evento de esta noche podía figurar como "pasado" 3 horas antes	Revisando el código de fechas	`toISOString()` da la hora UTC (Londres). Se calcula la hora **local** de Argentina
"+ NUEVA MISIÓN" y "PRÓXIMOS" se veían con la Ó minúscula	Captura del primer diseño	La fuente arcade no tiene vocales con tilde en mayúscula. Esos textos usan Pixelify Sans
Mucho espacio vacío dentro de cada misión	Captura del primer diseño	Los subtítulos `h4` y `h5` traían márgenes por defecto. Se quitaron
El lápiz "✎" se veía como una barra ""	Captura	Se reemplazó por el lápiz pixelado del jardín
El navegador de pruebas (Edge) dejó de abrir después de una actualización	Error "Failed to launch the browser"	Se usó Chrome. Para simular dos usuarias se usaron **dos ventanas de incógnito**, que Firebase ve como dos visitantes distintos

Primer diseño	Diseño corregido
	

Pruebas realizadas (dos usuarias a la vez: Ana y Bea)

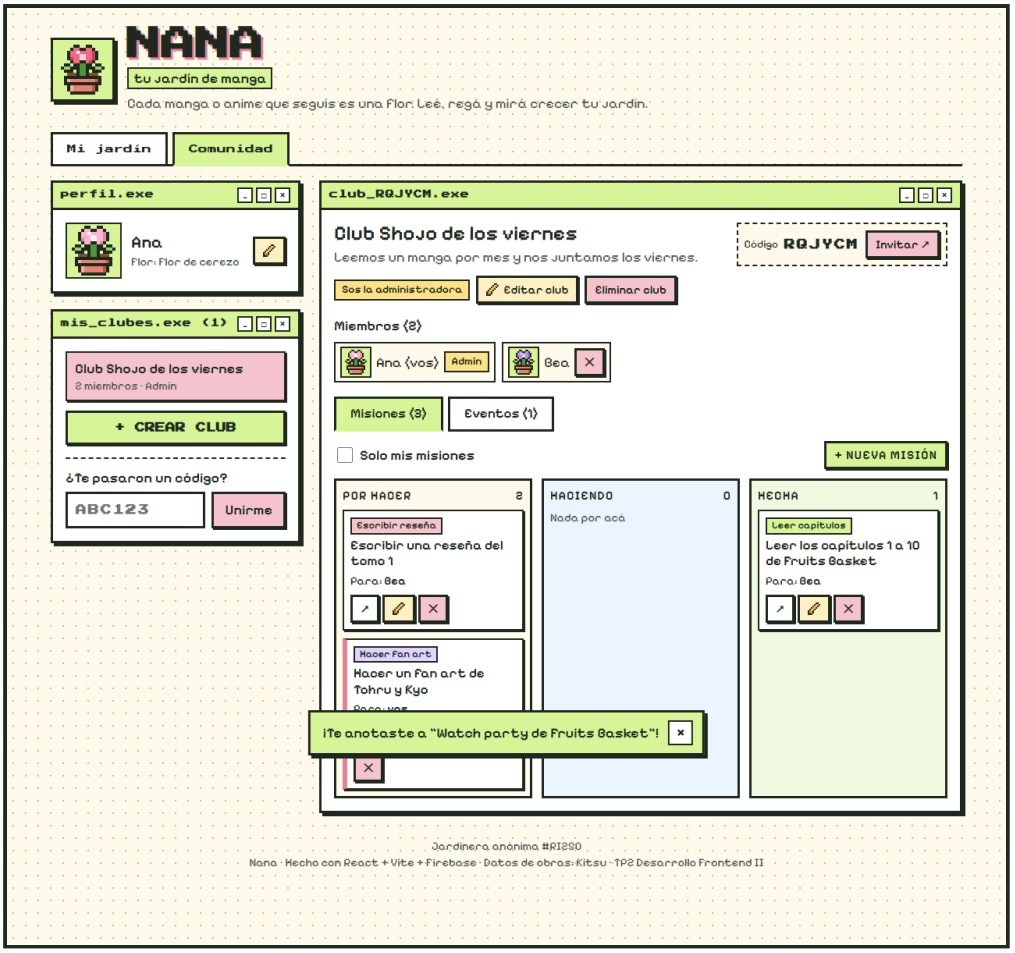
Prueba	Resultado
Ana crea su perfil y un club, y queda como administradora	✅
Una misión vacía muestra errores de validación	✅
Bea abre el enlace de invitación y el código se completa solo	✅
Bea se une y ve "Sos miembro"	✅
Bea **no** ve el botón "Nueva misión"	✅
Ana ve a Bea en la lista de miembros	✅
Ana asigna 3 misiones de distintos tipos y edita una	✅
Bea ve sus 2 misiones marcadas y las mueve: Por hacer → Haciendo → Hecha	✅

Prueba	Resultado
Firebase rechaza (403) que Bea cree una misión	✅
Firebase rechaza (403) que Bea cambie el nombre del club	✅
Bea crea un evento y Ana se anota (van 2)	✅
Celular sin desplazamiento horizontal	✅
Bea sale del club	✅
Ana elimina el club con sus misiones y eventos	✅
Errores en la consola	✅ Ninguno

Resultado: 19/19 pruebas correctas.

Publicación y verificación en Vercel

1. Primero se publicaron las **reglas nuevas** en Firebase y después el código.
2. Se revisó la lista de cambios (37 archivos, sin borrados ni `.env`) y se subió con la etiqueta `v0.4`.
3. Vercel la publicó solo, en 4 segundos (*Ready*).
4. **Sobre la URL pública** se repitió la prueba de dos usuarias (**19/19** ✅, incluidos los dos rechazos 403) y además la prueba completa del jardín de la v0.3 (**17/17** ✅), para confirmar que las reglas nuevas no rompieron nada de lo anterior. Esto es una **prueba de regresión**.



La v0.4 funcionando en https://jardin-de-manga.vercel.app



Ideas para la versión final (v1.0)

- **"Mi ramo":** estadísticas del jardín (flores, capítulos leídos, género favorito).
- **Filtros y buscador** dentro del jardín.
- **Significado de cada flor** (*hanakotoba*).
- **(Opcional)** Guardar la cuenta con Google para recuperar el jardín y los clubes en otro dispositivo, que es la "ampliación opcional" que menciona la consigna.
- Grabar el **video de demostración**.

Versión 0.4.1 — Ajustes de la versión 4: flores únicas, guía, abeja y Nana IA (10/11/2026)

Por qué una "0.4.1"

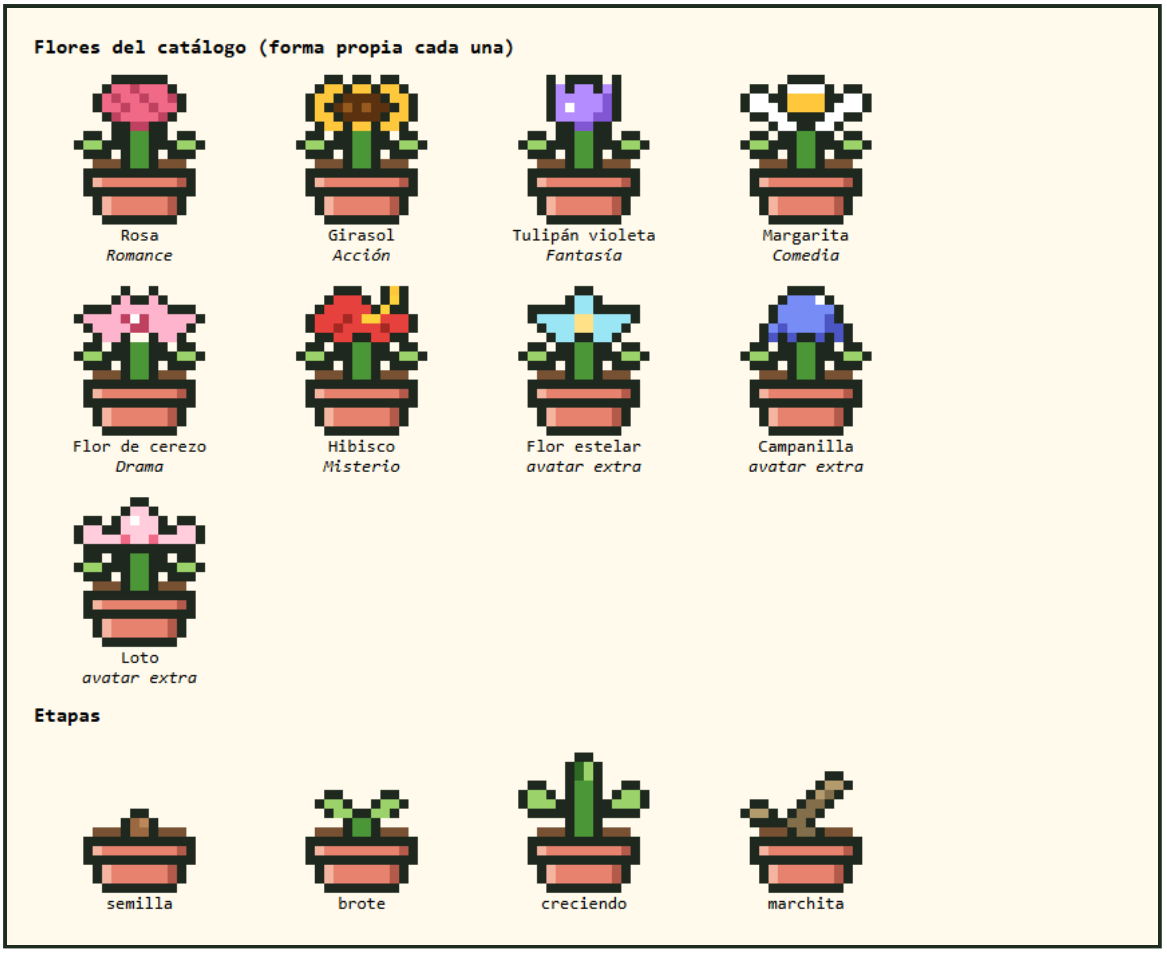
Después de probar la v0.4 surgieron mejoras de **diseño y ayuda** que completan esa misma versión. No agregan un módulo nuevo, así que se publicaron como una **versión intermedia** (0.4.1) antes de la final.

1. Cada flor con su propia forma

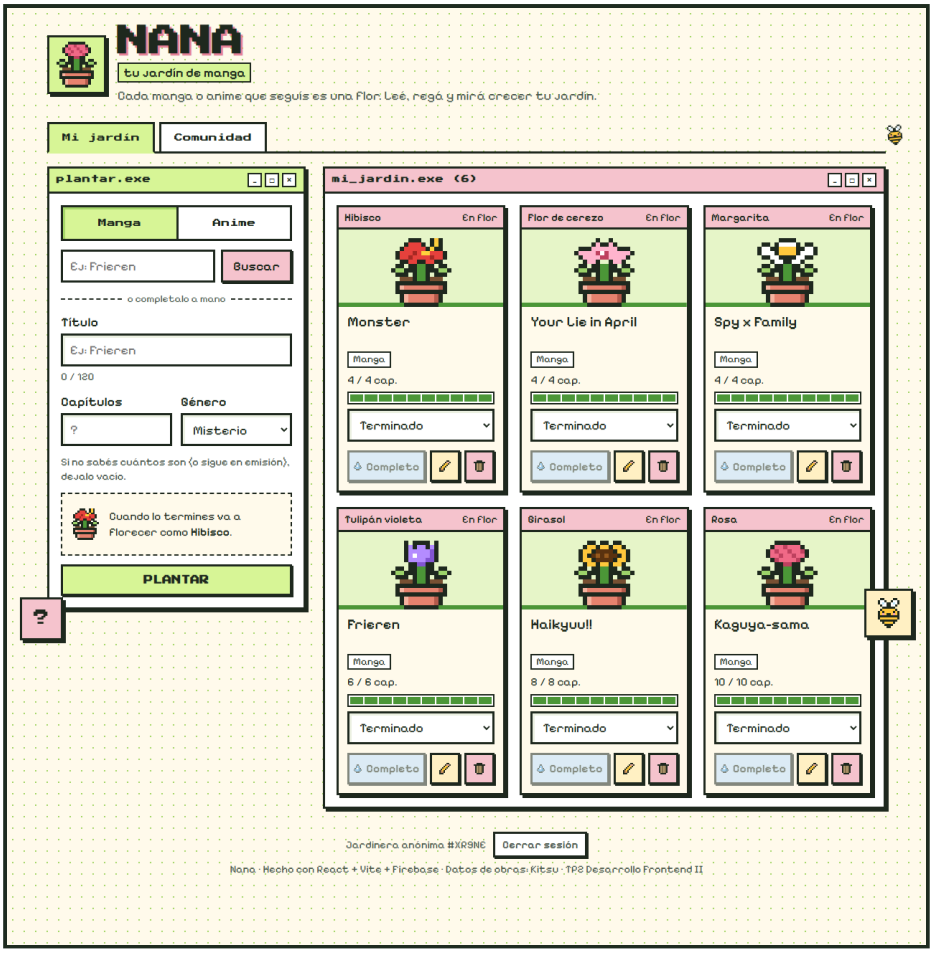
Antes: todas las flores eran el mismo dibujo pintado de distinto color. **Ahora:** hay **9 flores dibujadas con formas distintas**:

Flor	Forma	Uso	Significado (hanakotoba)
Rosa (*bara*)	Pétalos en espiral	Romance	Amor
Girasol (*himawari*)	Pétalos alrededor de un centro grande	Acción	Admiración y energía
Tulipán violeta (*chūrippu*)	Copa con tres puntas	Fantasía	Amor eterno y nobleza
Margarita (*deiji*)	Rueda de pétalos blancos	Comedia	Inocencia y alegría
Flor de cerezo (*sakura*)	Cinco pétalos con muesca	Drama	La belleza de lo que dura poco
Hibisco (*haibisukasu*)	Pétalos grandes y estambre largo	Misterio	Belleza delicada
Flor estelar	Estrella	Solo avatar	**Inventada por Nana**: guarda deseos
Campanilla (*kikyō*)	Campana	Solo avatar	Amor que no cambia
Loto (*hasu*)	Pétalos abiertos en capas	Solo avatar	Pureza y volver a empezar

- Técnicamente, cada flor tiene su "cabeza" (6 filas de píxeles) y comparten el mismo tallo y la misma maceta. Un componente nuevo, `SpriteFlor`, dibuja cualquier flor en cualquier etapa.
- El perfil ahora ofrece **9 avatares** y muestra el significado de la flor elegida. Se actualizó la regla de Firestore del perfil para aceptar las 3 flores nuevas, y **se publicó antes del código**.



Hoja con las 9 flores y las etapas de crecimiento.



Un jardín con un manga de cada género: cada uno florece con su forma.

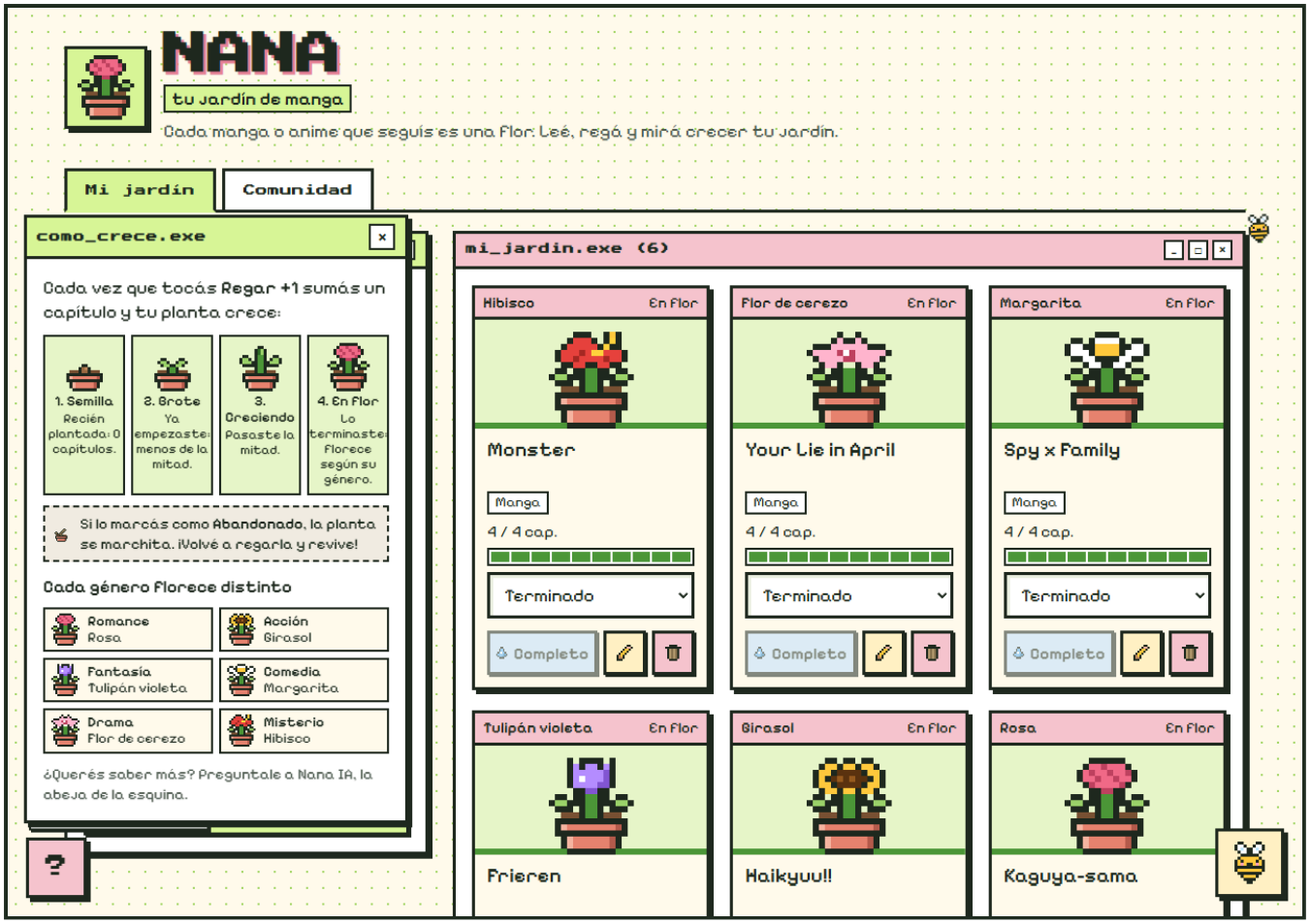
2. Guía flotante "¿Cómo crece tu flor?"

Un botón **"?"** fijo abajo a la izquierda. Al pasar el mouse muestra su etiqueta, y al tocarlo abre la ventana `como_crece.exe`:

- las 4 etapas (semilla → brote → creciendo → en flor) y qué hace falta para llegar a cada una;
- qué pasa si se abandona (se marchita) y cómo revivirla;
- qué flor da cada género.

No ocupa lugar todo el tiempo: se abre solo cuando se necesita.

Cambio de funcionamiento: la guía dice que una planta marchita **revive si la regás**. Antes no era así: una planta abandonada seguía marchita aunque se regara. Se corrigió para que la app haga lo que promete: regar una planta abandonada la pasa a "Leyendo".



3. La abeja que revolotea

Una **abeja pixelada** recorre los bordes de la pantalla, aleteando y parpadeando.

- Es **solo decoración**: no se puede tocar, no tapa botones y los lectores de pantalla la ignoran.
- **Accesibilidad:** si la persona configuró su sistema para "reducir el movimiento", la abeja no aparece.
- El parpadeo usa `useEffect` con `setInterval`, y su **limpieza** (`clearInterval`) detiene el temporizador.

4. Nana IA, la asistente virtual

Una abeja con ojitos y cachetes, en la esquina inferior derecha. Al tocarla se abre el chat `nana_ia.exe`, que **explica paso a paso** cómo usar la app. Algunas preguntas que responde:

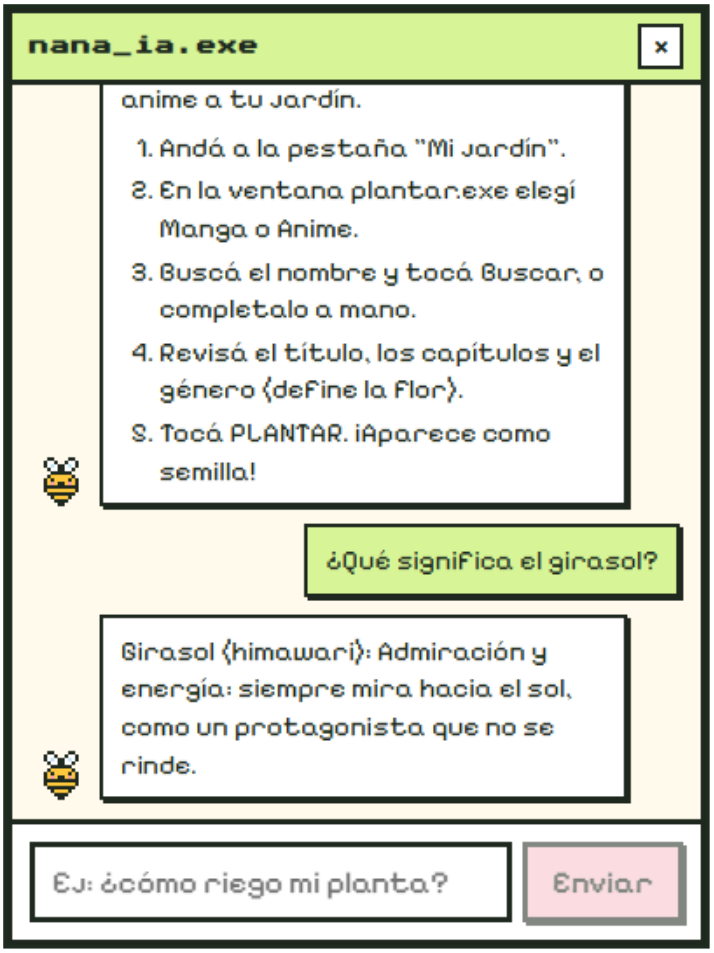
- "¿Cómo planto un manga?", "¿Cómo crece mi flor?", "¿Qué significa el girasol?";
- "¿Quién asigna las misiones?", "¿Cómo cierro sesión?";
- "¿Cuántas plantas tengo?": esta respuesta usa **tus datos reales** del jardín.

Decisión de diseño importante: ¿por qué no ChatGPT u otra IA externa? Conectar un modelo de IA externo exige una **clave privada**, y la consigna dice que nunca se deben incorporar claves privadas en el código del navegador (con el prefijo `VITE_` quedarían a la vista de cualquiera). Por eso Nana IA funciona con una **base de conocimiento propia** (`src/data/conocimiento.js`):

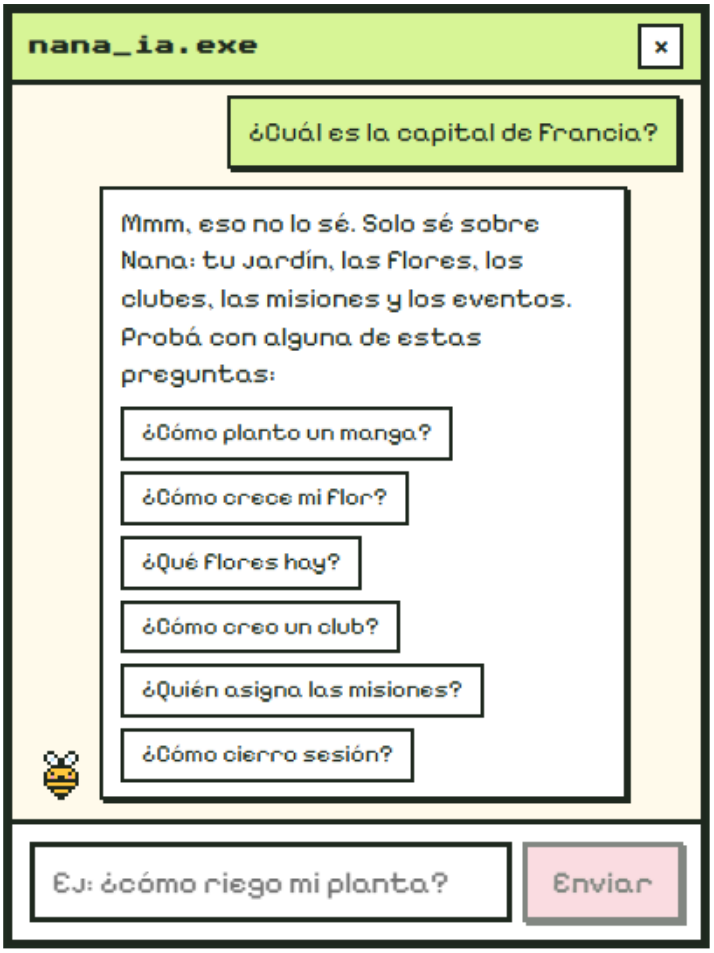
- 27 temas, cada uno con palabras clave, una respuesta y, si corresponde, los pasos;

- la pregunta se **normaliza** (minúsculas, sin tildes ni signos) y se compara con las palabras clave. Gana el tema con más coincidencias;
- algunas respuestas se arman en el momento: la lista de flores (con `map`), el significado de una flor (con `find`) y el resumen de tu jardín (con `filter` y `reduce`);
- si la pregunta no es sobre Nana, lo dice y ofrece **preguntas sugeridas** que se pueden tocar.

Así cumple lo pedido ("que tenga información de la aplicación y el sitio únicamente"), funciona sin internet extra, no cuesta nada y no expone ninguna clave.



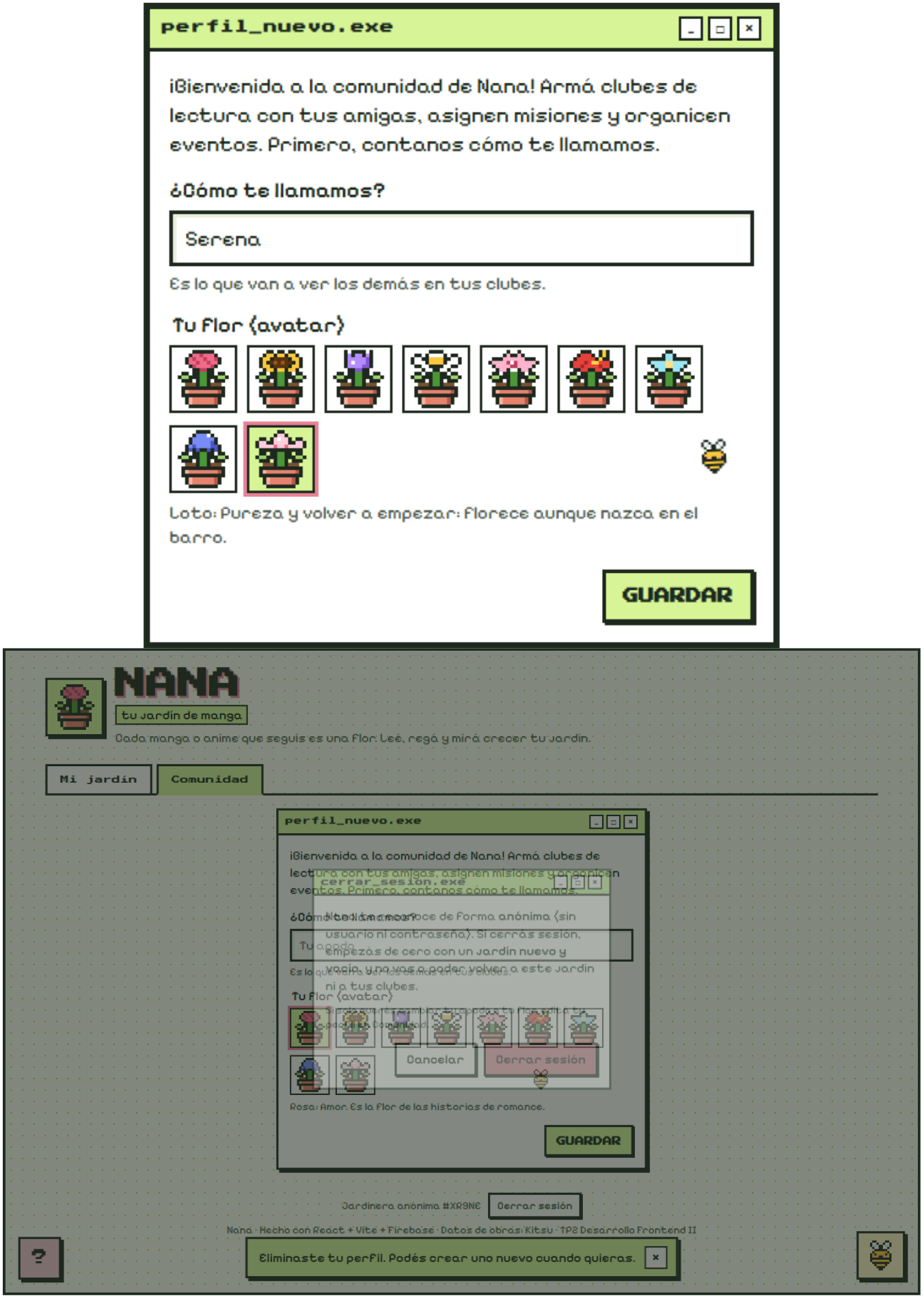
Nana IA explicando paso a paso cómo plantar y el significado del girasol.



Si le preguntan algo que no es de la app, lo aclara y sugiere preguntas.

5. Perfil con CRUD completo y "Cerrar sesión"

- El perfil ya se podía **crear, ver y editar**. Faltaba **eliminarlo**: ahora en `perfil.exe` está "Eliminar perfil", que pide confirmación y vuelve a la bienvenida, como al principio.
- **Cerrar sesión**, al pie de la página. Como el acceso es **anónimo**, cerrar sesión significa **empezar de cero**: Firebase crea una identidad nueva con un jardín vacío. Por eso la app lo **advierte antes** en la ventana `cerrar_sesion.exe`, y sugiere editar el perfil si solo se quiere cambiar el apodo.
- Al cambiar de identidad, la Comunidad se reinicia por completo: el componente lleva `key={uid}`, así React lo vuelve a crear de cero.

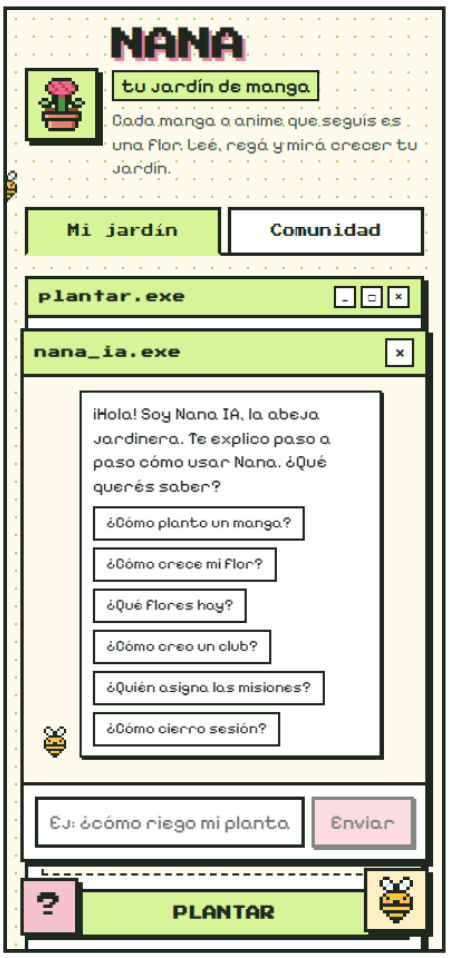

perfil_nuevo.exe
¡Bienvenida a la comunidad de Nana! Armá clubes de lectura con tus amigas, asignen misiones y organicen eventos. Primero, contanos cómo te llamamos.
¿Cómo te llamamos?
Serena
Es lo que van a ver los demás en tus clubes.
Tu Flor (avatar)
Loto: Pureza y volver a empezar: Florece aunque nazca en el barro.
GUARDAR
NANA
tu jardín de manga
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Mi jardín
Comunidad
cerrar_sesion.exe
Tu apodo
Cancelar
Cerrar sesión
Rosa: Amor. Es la flor de las historias de romance.
GUARDAR
Jardinera anónima #XR9NE
Cerrar sesión
Nana · Hecho con React + Vite + Firebase · Datos de obras: Kitsu · TP2 Desarrollo Frontend II
Eliminaste tu perfil. Podés crear uno nuevo cuando quieras.

Dificultades y cómo se resolvieron

Problema	Cómo se detectó	Solución
Nana IA no entendía "¿Cómo **planto**...?" ni "¿Cómo **cierro** sesión?"	Prueba automática con preguntas reales	Las palabras clave ahora incluyen raíces ("plant", "rieg", "elimin") y conjugaciones
A "¿Cuál es la capital de Francia?" respondía sobre la **API**	La prueba esperaba "no sé"	"c**api**tal" contenía "api". Ahora las claves solo coinciden con el **comienzo de una palabra**
A "¿Qué significa la flor de loto?" respondía con la **flor de cerezo**	Revisando el código antes de probar	Se buscaba la primera palabra del nombre ("flor"). Ahora se usan palabras distintivas: loto, hasu, sakura...
La guía prometía que la planta revive al regarla, pero no pasaba	Al escribir el texto de la guía	Se cambió la lógica: regar una planta abandonada la pasa a "Leyendo"

Pruebas (21 de 21 correctas)

La abeja aparece y es decorativa · las 6 flores distintas en el jardín · la etiqueta del "?" al pasar el mouse · la guía con etapas y géneros · 9 preguntas a Nana IA con su respuesta correcta (incluida una fuera de tema) · las sugerencias se pueden tocar · 9 avatares · el significado del avatar · perfil creado con la flor de loto (la regla nueva lo acepta) · eliminar perfil · cerrar sesión crea una identidad nueva con el jardín vacío · celular sin desplazamiento horizontal con Nana IA abierta · ningún error en la consola.



Versión 1.0 — Versión final (quinta versión) (10/11/2026)

Objetivo

Cerrar el producto: sumar las herramientas para **encontrar y entender** lo que hay en el jardín (filtros, "Mi ramo", herbario), darle **sonido**, **revisar todo contra la consigna** y dejar la documentación lista para la entrega y para el final.

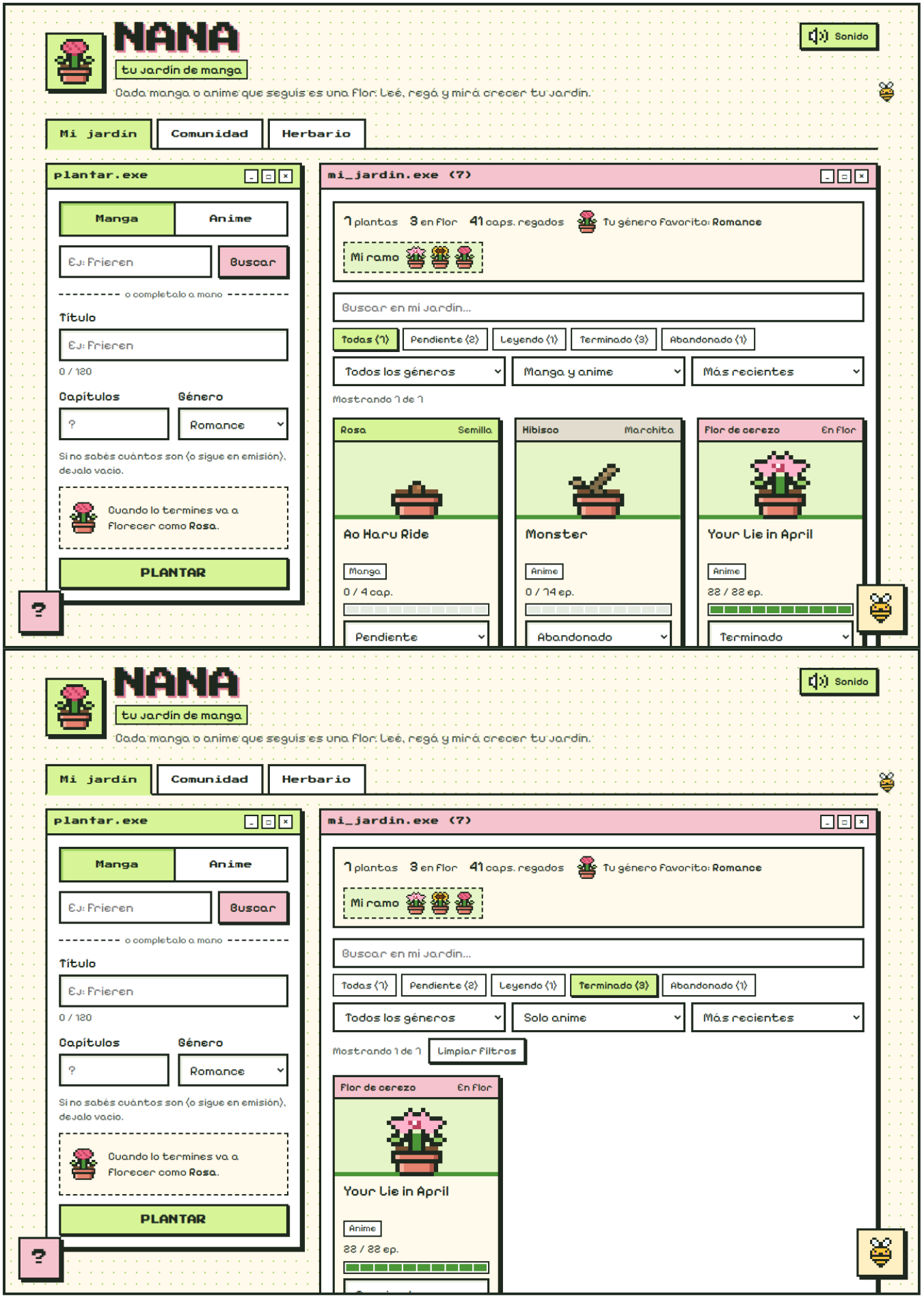
Qué se agregó

1. "Mi ramo", arriba del jardín Un resumen con cantidad de plantas, cuántas están en flor, capítulos regados en total y **género favorito**. Debajo, **el ramo**: una flor chiquita por cada manga terminado. Se calcula con `filter`, `reduce` y `map` sobre las plantas que ya están cargadas, así que no hace ninguna consulta extra a la base.

2. Filtros del jardín

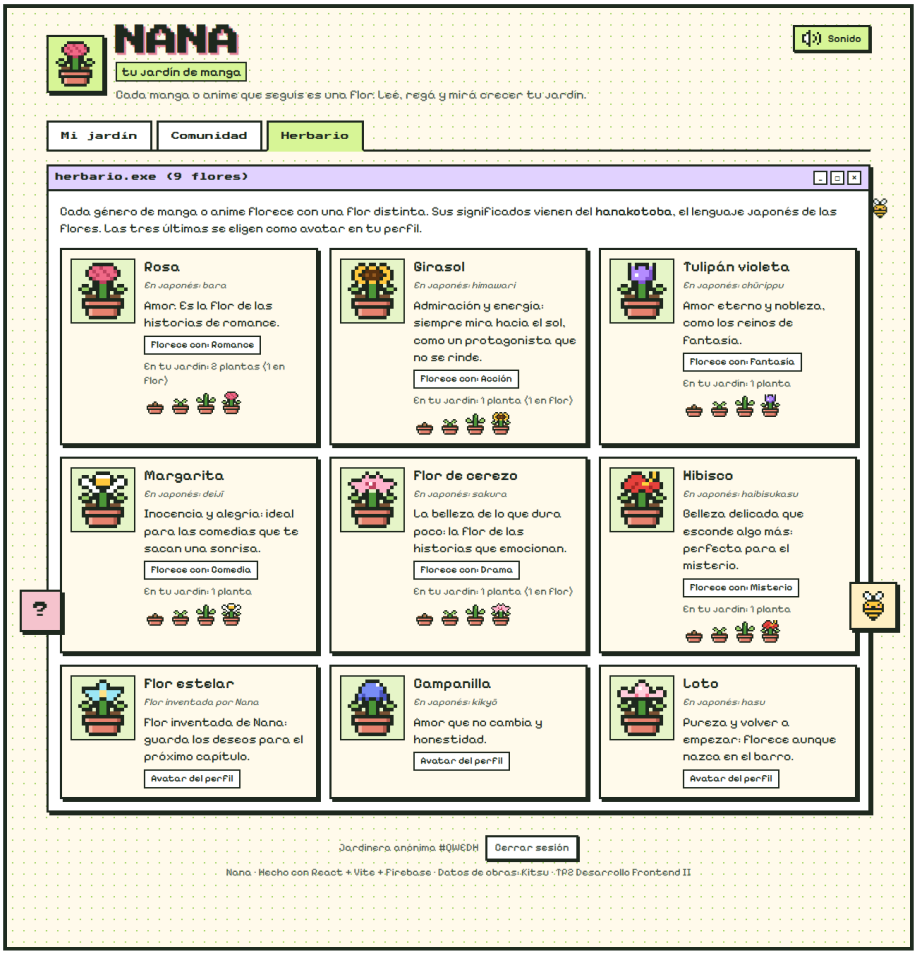
- buscador por título (sin importar tildes ni mayúsculas);
- chips por estado, con la cantidad de cada uno: *Todas (7) · Pendiente (2) · Terminado (3)…*;
- género, tipo (manga o anime) y **orden** (recientes, A-Z, más avanzadas, más regadas);
- "Mostrando X de Y" y un botón "Limpiar filtros".

Siguiendo la guía de clase, los filtros **no usan** `useEffect`: se calculan durante el renderizado encadenando `filter` y `sort`.



Filtro "Terminado" + "Solo anime": queda una sola planta.

3. Herbario, la tercera pestaña El catálogo de las 9 flores, con su dibujo, su nombre en japonés, su significado según el *hanakotoba*, con qué género florece (o si es avatar), **cuántas tenés en tu jardín** y las 4 etapas de crecimiento de esa flor. Además, en cada tarjeta del jardín, al pasar el mouse por el nombre de la flor aparece su significado.



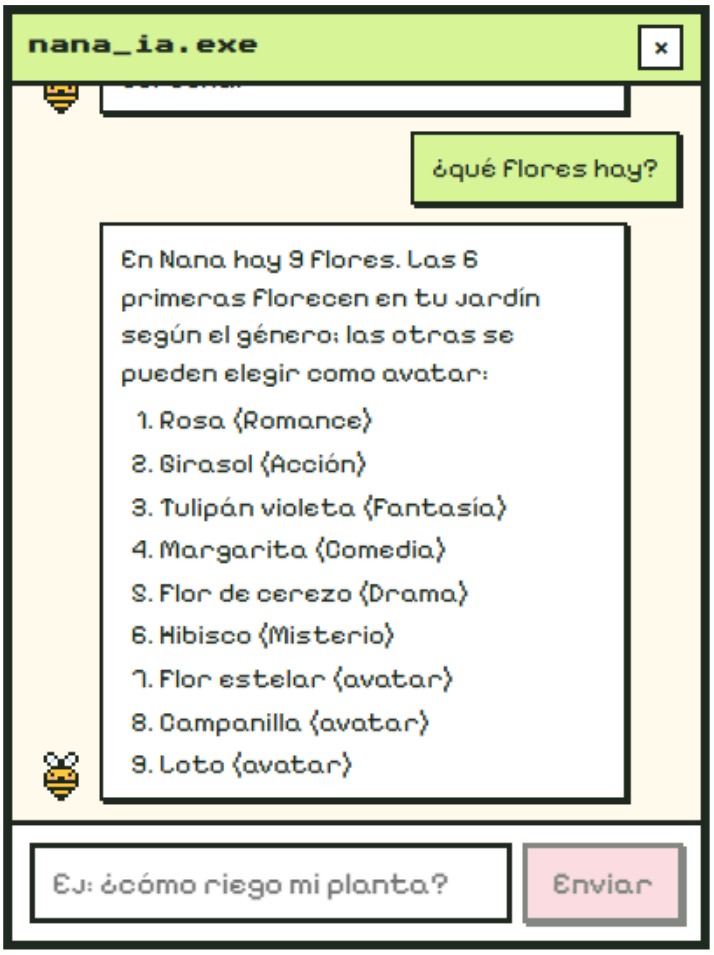
4. Sonidos retro Se generan con la **Web Audio API** del navegador, como un chip de consola: no hay archivos de audio para descargar.

Acción	Sonido
Plantar o asignar una misión	Tres notas que suben
Regar	Dos gotitas
Florecer o revivir	Arpegio con brillo
Compostar o eliminar	Notas que bajan
Cumplir una misión	Fanfarria
Anotarse a un evento	Dos notas alegres
Error	Zumbido grave
Abrir Nana IA	Zumbido de abeja

El botón con el **parlante pixelado** (arriba a la derecha) los apaga o prende. Esa preferencia se recuerda en `localStorage`: es una **preferencia del navegador**, no un dato de la app. Los datos siguen todos en Firebase,

como pide la consigna.

5. Nana IA aprendió los temas nuevos: filtros, herbario, "Mi ramo" y sonidos.



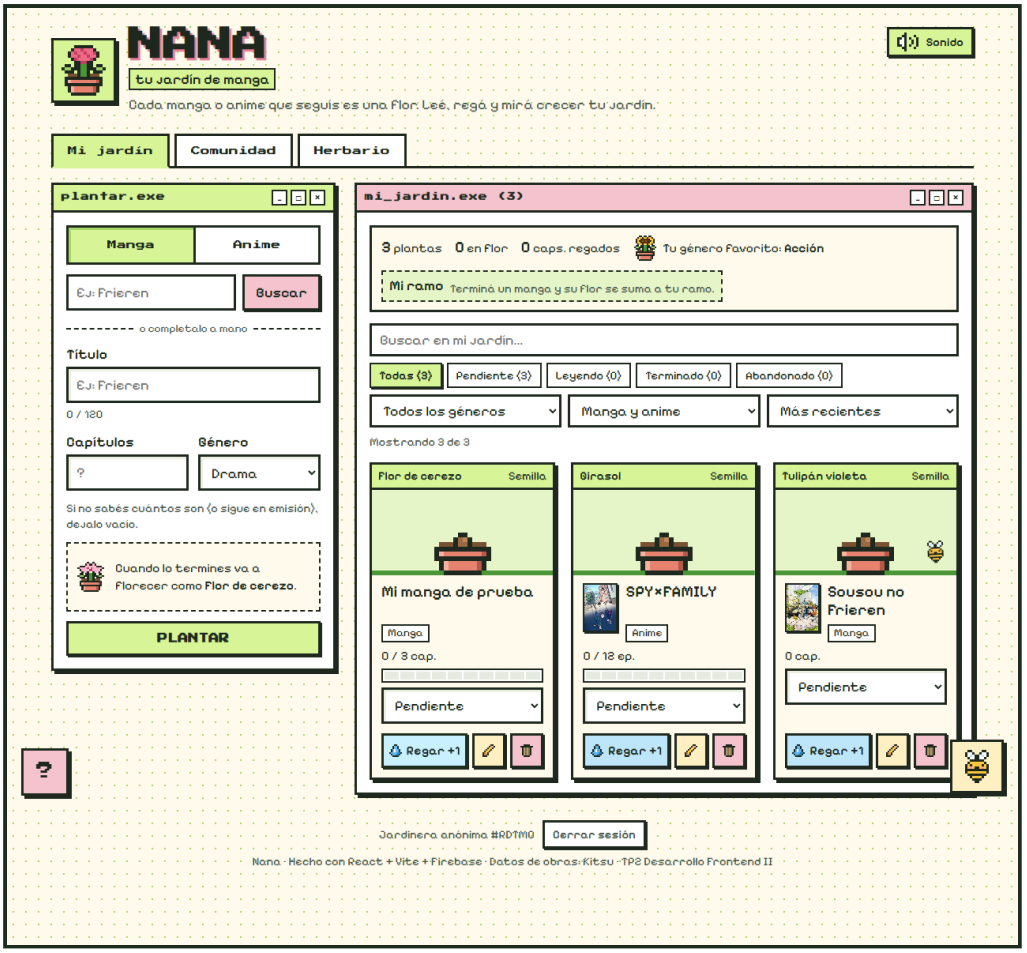
Auditoría final contra la consigna y la rúbrica

Requisito	Cómo se cumple	✓
React + Vite: componentes, eventos y estado	30 componentes. `useState` y `useEffect` en toda la app	✅
Firestore: consultar, agregar, modificar, eliminar	**5 CRUD completos**: plantas, perfil, clubes, misiones y eventos	✅
Persistencia al recargar	Todo en Firestore (`localStorage` solo para la preferencia de sonido)	✅
GitHub con instrucciones	README con pasos, `.env.example` , `firestore.rules` , 6 etiquetas de versión	✅
Vercel con URL funcional	Despliegue automático en cada `push` , verificado en cada versión	✅
Validaciones, carga, éxito y error; compu y celular	En todos los formularios. Pruebas a 1280 px y 390 px	✅
Reglas que no dejen la base abierta	Reglas por dueña y por rol. Probado con 403	✅
`.env` fuera de GitHub	Verificado: no está publicado y no hay ninguna llave en el código	✅

Requisito	Cómo se cumple	✓
Al menos 3 versiones con cambios identificables	6: v0.1, v0.2, v0.3, v0.4, v0.4.1 y v1.0	✅
Informe (idea, público, funciones, datos en Firebase, IA e instrucciones)	`docs/informe.md`, reescrito **breve** como pide la consigna	✅
Bitácora con evolución, pruebas, IA, dificultades y capturas	Este documento	✅
Mini video de demostración	⏳ Lo graba la autora (guion en `entrega.md`)	⏳
¿Hay algo que no cumpla y haya que sacar?	Se revisó: Nana IA no usa claves privadas, los sonidos no usan archivos externos y `localStorage` no guarda datos de la app. **No hubo que quitar nada**	✅

Dificultades de esta versión

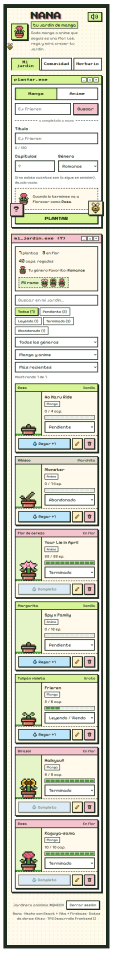
Problema	Cómo se detectó	Solución
El aviso flotante tapaba los botones que quedaban debajo. Con "Mi ramo" y los filtros, la primera tarjeta bajó justo a la altura del aviso, y tocar "Regar" en esos 3,5 segundos no hacía nada	La **prueba de regresión** de la v0.3 empezó a fallar	El aviso deja pasar los clics (`pointer-events: none`), salvo su botón de cerrar
Nana IA confundía "¿Qué es el herbario?" con "¿Qué flores hay?", y "¿Cómo filtro mi jardín?" con el resumen del jardín	Revisando las palabras clave antes de probar	Se separaron las claves que compartían dos temas
El 7 de la fuente pixelada parece un 1 ("Todas (7)")	Captura	Se verificó con zoom que el número es correcto: es la forma del 7 en esa fuente



Antes de la corrección: después de tocar "Regar" en la primera tarjeta no pasaba nada, porque el clic caía sobre el aviso.

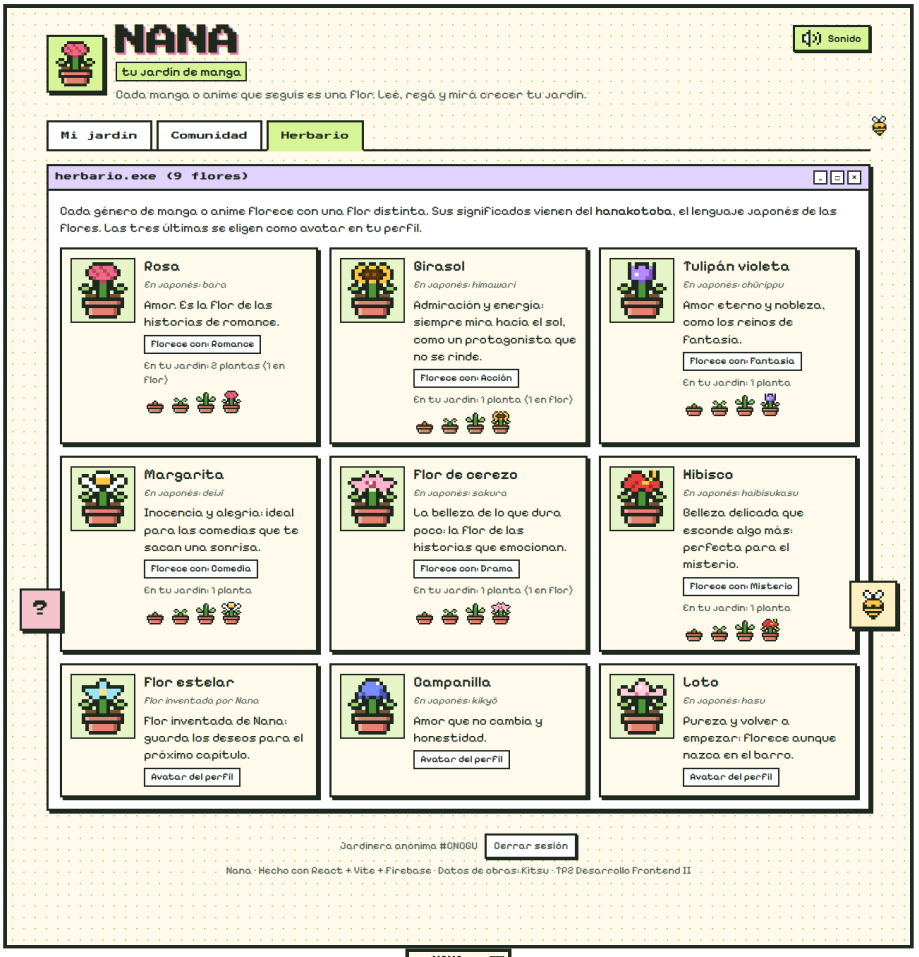
Pruebas finales

Prueba automática	Resultado
v1.0: Mi ramo, filtros, herbario, sonidos (se cuentan las notas), silencio que se recuerda, Nana IA, celular	**26/26** ✅
Regresión v0.3: jardín, API, edición, estados	**17/17** ✅
Regresión v0.4: comunidad con dos usuarias, roles y seguridad (403)	**19/19** ✅
Regresión v0.4.1: flores, guía, Nana IA, perfil, cerrar sesión	**21/21** ✅
Errores en la consola	Ninguno



Publicación y verificación final en Vercel

1. Se revisó la lista de cambios (25 archivos, sin borrados ni `.env`) y se buscaron llaves en el código: no había ninguna.
2. Se subió con la etiqueta `v1.0`. Vercel la publicó en 4 segundos (*Ready*).
3. **Sobre la URL pública** se corrieron **todas** las pruebas: v1.0 (26/26), jardín (17/17), comunidad con dos usuarias (19/19) y v0.4.1 (21/21). En total, **83 de 83 correctas** y ningún error en la consola.


NANA
tu jardín de manga
Sonido
Cada manga o anime que seguís es una Flor: Leé, regá y mirá crecer tu jardín.
Mi jardín
Comunidad
Herbario
herbario.exe (9 flores)
Cada género de manga o anime florece con una flor distinta. Sus significados vienen del hanakotoba, el lenguaje japonés de las flores. Las tres últimas se eligen como avatar en tu perfil.
Rosa
En japonés: bara
Amor. Es la flor de las historias de romance.
Florece con: Romance
En tu jardín: 8 plantas (1 en flor)
Girasol
En japonés: himawari
Admiración y energía: siempre mira hacia el sol, como un protagonista que no se rinde.
Florece con: Acción
En tu jardín: 1 planta (1 en flor)
Tulipán violeta
En japonés: chūrippu
Amor eterno y nobleza, como los reinos de fantasía.
Florece con: Fantasía
En tu jardín: 1 planta
Margarita
En japonés: deijī
Inocencia y alegría: ideal para las comedias que te sacan una sonrisa.
Florece con: Comedia
En tu jardín: 1 planta
Flor de cerezo
En japonés: sakura
La belleza de lo que dura poco: la flor de las historias que emocionan.
Florece con: Drama
En tu jardín: 1 planta (1 en flor)
Hibisco
En japonés: haibisukasu
Belleza delicada que esconde algo más: perfecta para el misterio.
Florece con: Misterio
En tu jardín: 1 planta
Flor estelar
Flor inventada por Nana
Flor inventada de Nana: guarda los deseos para el próximo capítulo.
Avatar del perfil
Campanilla
En japonés: kikyō
Amor que no cambia y honestidad.
Avatar del perfil
Loto
En japonés: hasu
Pureza y volver a empezar: florece aunque nazca en el barro.
Avatar del perfil
Jardinera anónima #CNOGU
Cerrar sesión
Nana · Hecho con React + Vite + Firebase · Datos de obras: Kitsu · TP2 Desarrollo Frontend II



Versión 1.1 — Pulido de diseño kawaii (10/11/2026)

Objetivo

Revisar por última vez todo contra la consigna y la rúbrica, y **pulir la interfaz**: los textos estaban "apelmazados", con poco espacio alrededor. La idea fue darle aire, hacerla más **kawaii** (tierna) y que se vea profesional, **sin cambiar la paleta de colores** (lima, rosa, crema y tinta oscura).

Referencias visuales

La autora sumó nuevas imágenes de referencia: enredaderas de cerezo pixeladas en las esquinas de un panel, un planificador kawaii en tonos rosa y crema, macetas pixeladas y una baldosa de pasto con margaritas. De ahí salieron tres decisiones:

1. **Flores que abrazan las esquinas** de las ventanas principales.
2. **Fondo de cuaderno cuadriculado** rosa pálido, como el del planificador.
3. **Una franja de pasto con margaritas** al pie de la página, como si la app fuera un jardín.

Qué cambió

1. Más aire en toda la app

- Se armó una **escala de espacios** (4, 8, 12, 16, 24 y 32 píxeles) y se usó en todos lados, así todo "respira" igual.
- Interlineado más alto (de 1,35 a 1,5) y más margen interior en ventanas, tarjetas, formularios, filtros, el chat de Nana IA, la guía, el herbario y la comunidad.
- La guía "¿Cómo crece tu flor?" pasó de 4 columnas apretadas a **2 columnas**, y se lee de corrido.

2. Esquinas "redondeadas en píxeles" En pixel art no se usan bordes redondeados (se verían borrosos). Lo que se hace es **cortar el píxel de cada esquina**. Se logró dibujando el marco con 4 sombras duras, una por lado, que dejan las esquinas vacías. Así se ven las ventanas, las tarjetas, los botones, el aviso y los botones flotantes.

3. Sombras rosadas suaves en las ventanas grandes (antes eran negras), y los tres botones de cada ventana ahora son **cuadraditos pastel** (amarillo, celeste y rosa).

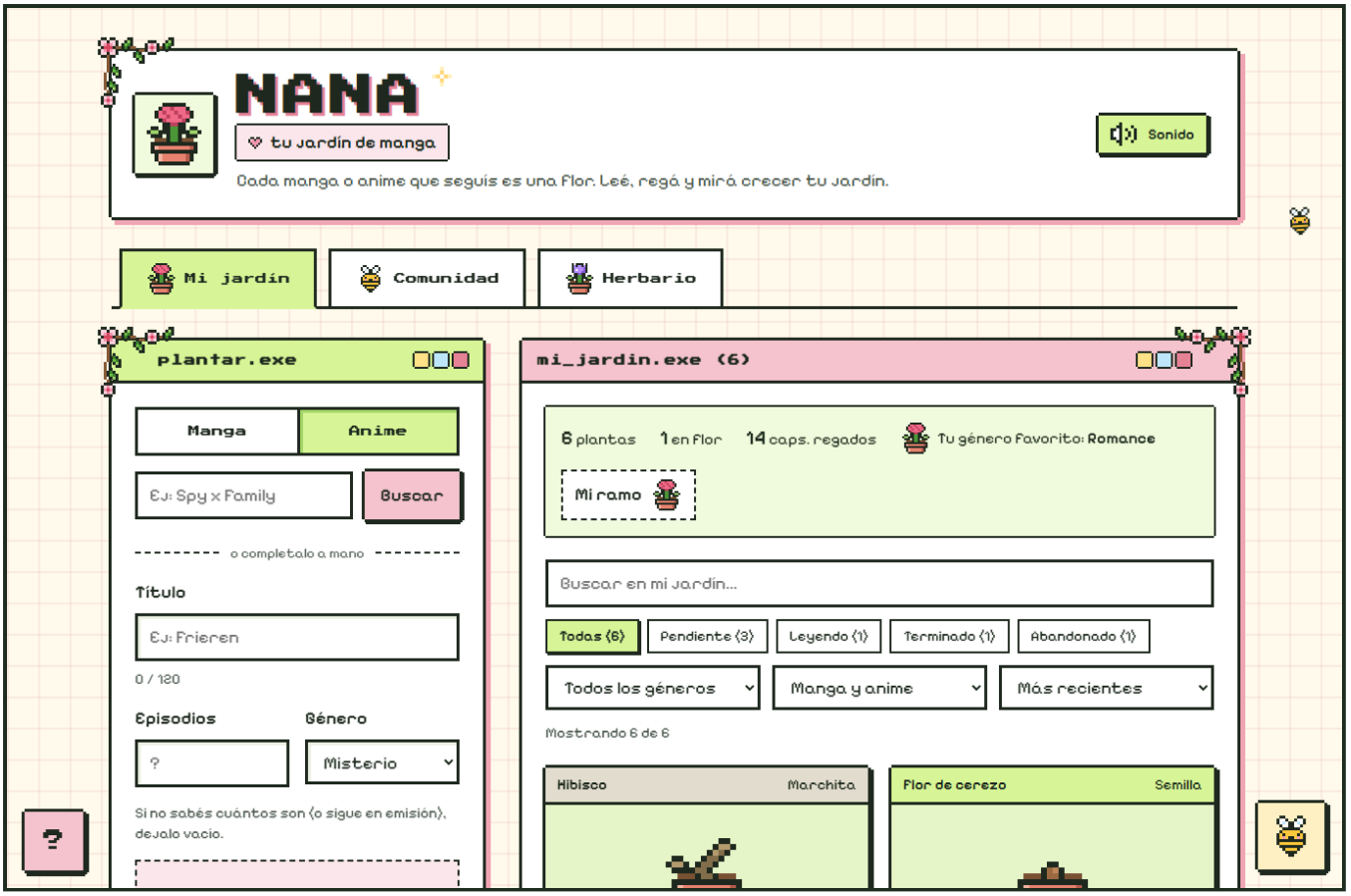
4. Adornos pixelados dibujados con código, igual que las flores (archivo nuevo `src/data/decoracion.js` y componente `Adorno.jsx`):

- **Enredadera de cerezo** en una esquina de las ventanas principales. Es la misma imagen dada vuelta como en un espejo para la esquina derecha.
- **Destello** titilante al lado de "NANA" y un **corazón** en "tu jardín de manga".
- **Franja de pasto con margaritas** al pie. Es una "baldosa" pixelada que se convierte en imagen con la función `spriteComoFondo` (recorre el dibujo con `map`, igual que `PixelSprite`) y se repite a lo ancho.
- Todos los adornos son solo decoración: los lectores de pantalla los ignoran y no reciben clics.

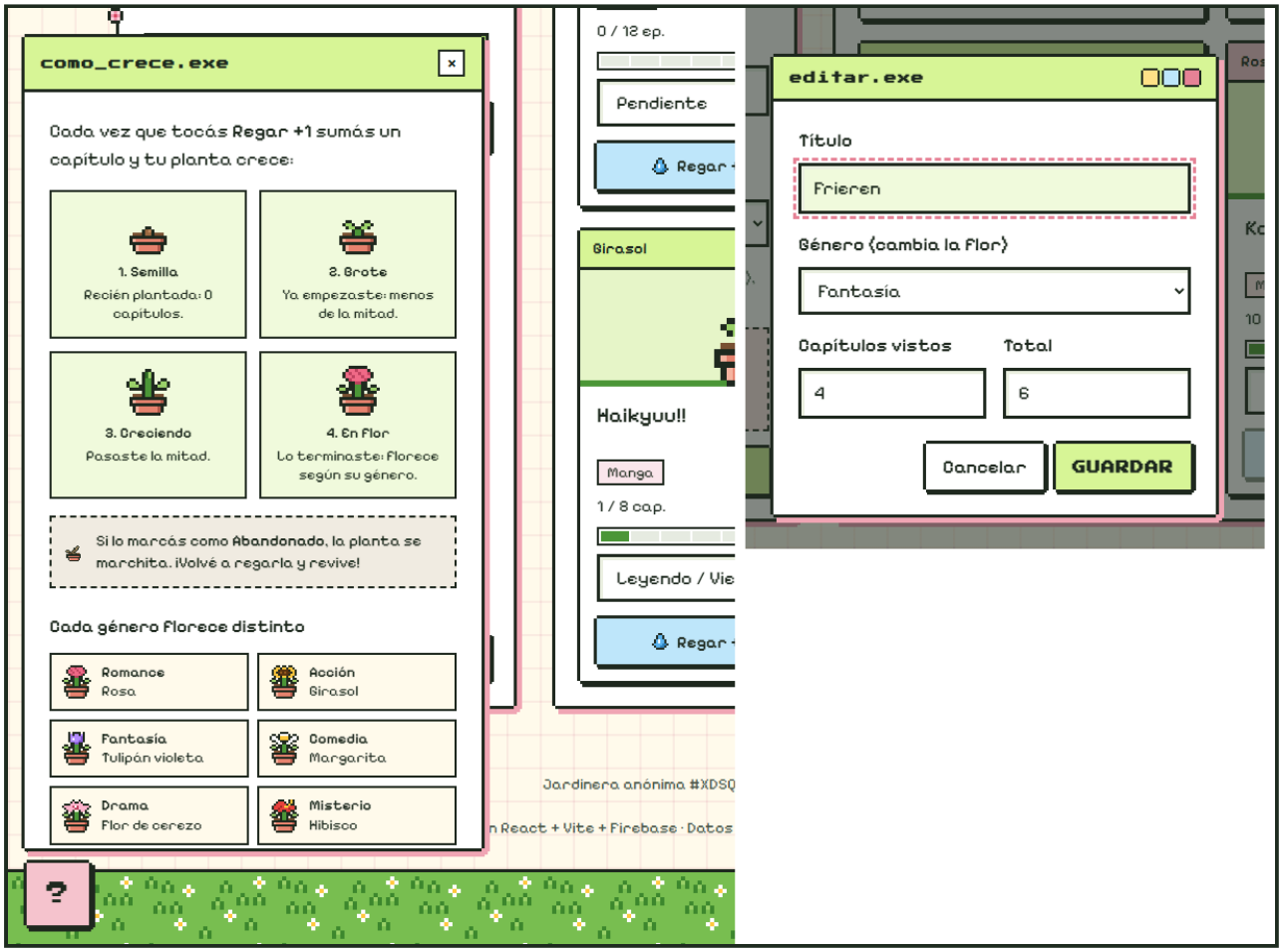
5. Encabezado como cartel: el logo, el nombre y el lema ahora están dentro de un cartel blanco con enredadera, y **las pestañas tienen un iconito**: una rosa (Mi jardín), la abeja (Comunidad) y un tulipán (Herbario).

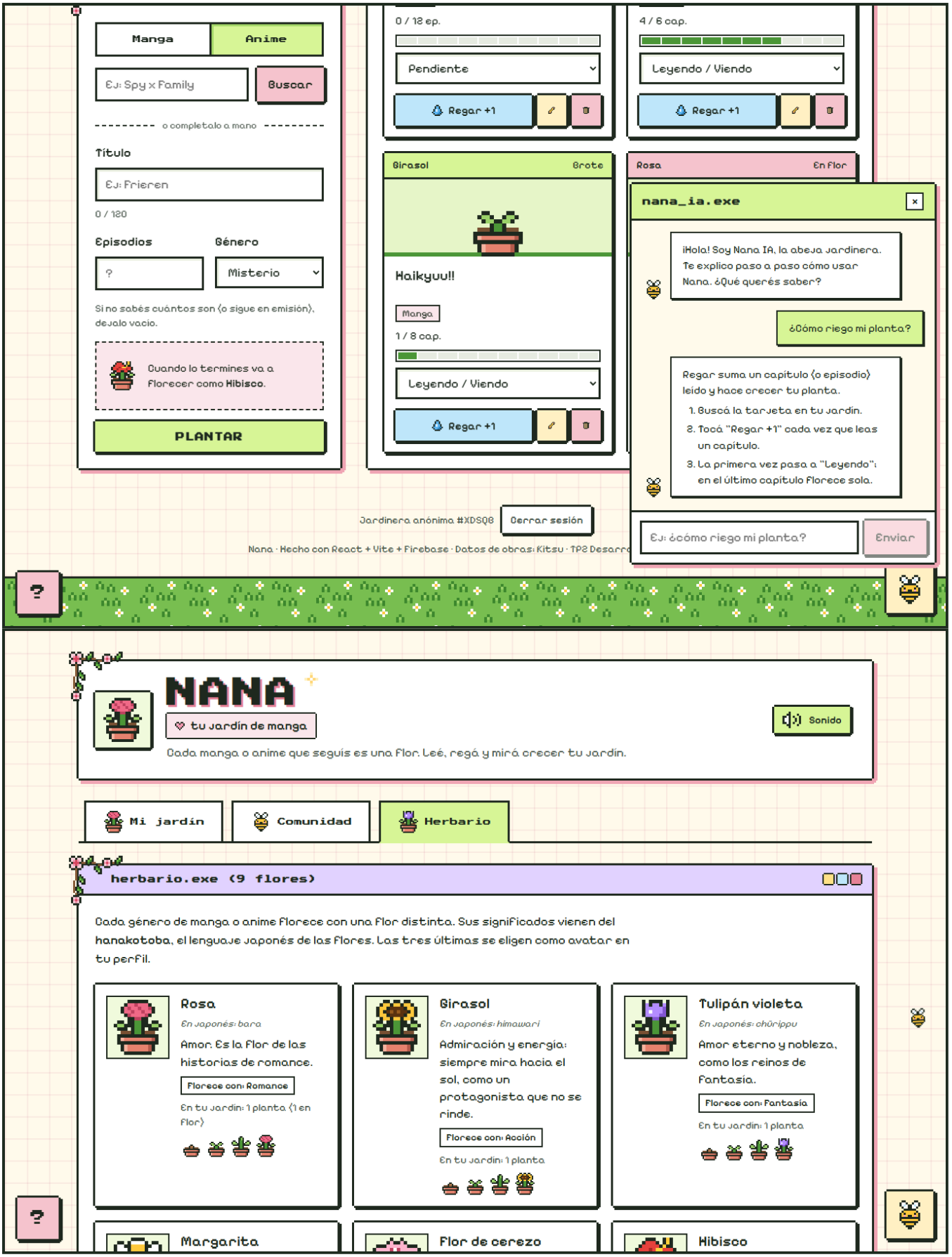
6. El formulario "plantar" acompaña el scroll en la compu: si el jardín es largo, el formulario queda a la vista (`position: sticky`).

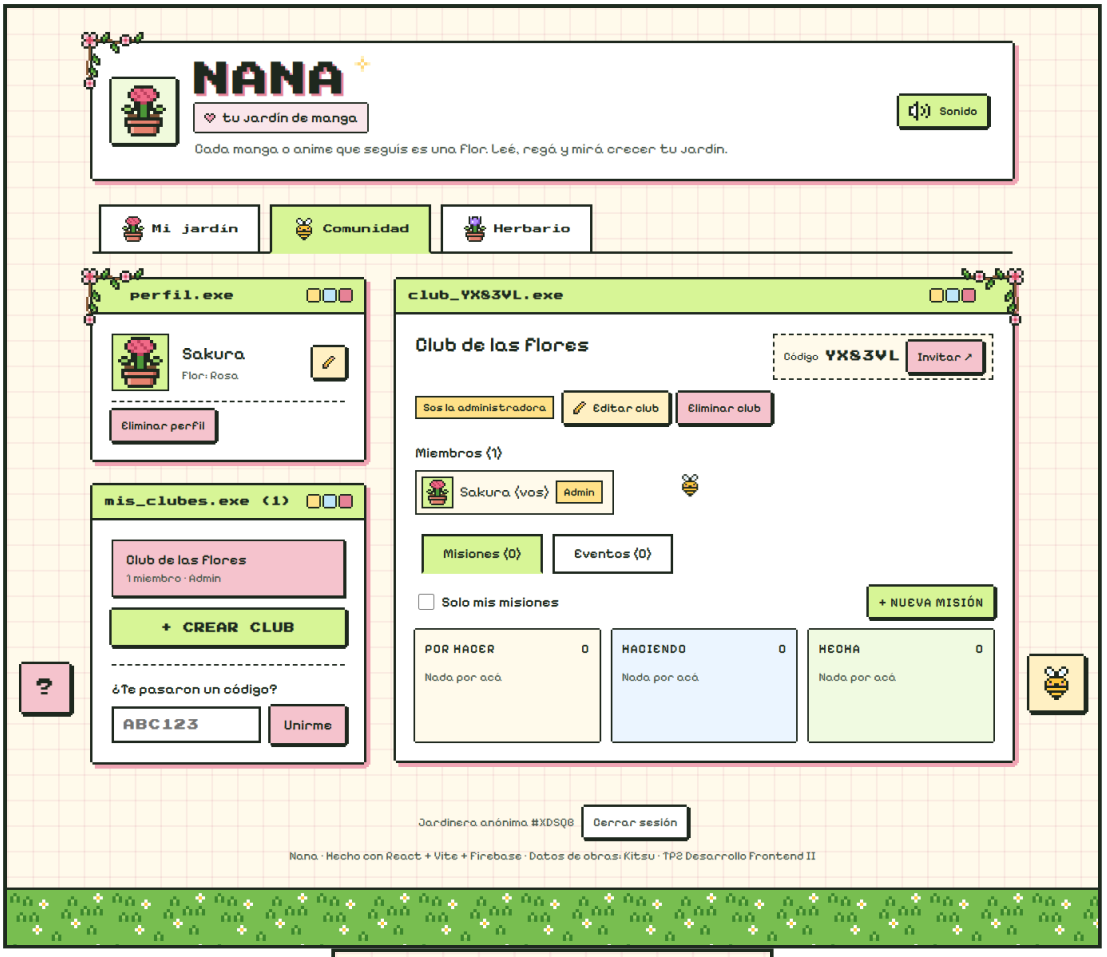
Todos estos cambios están en un archivo nuevo, **`src/Diseno.css`**, que se carga al final y refina los estilos de las versiones anteriores. Así se puede ver en un solo lugar qué cambió en el diseño.



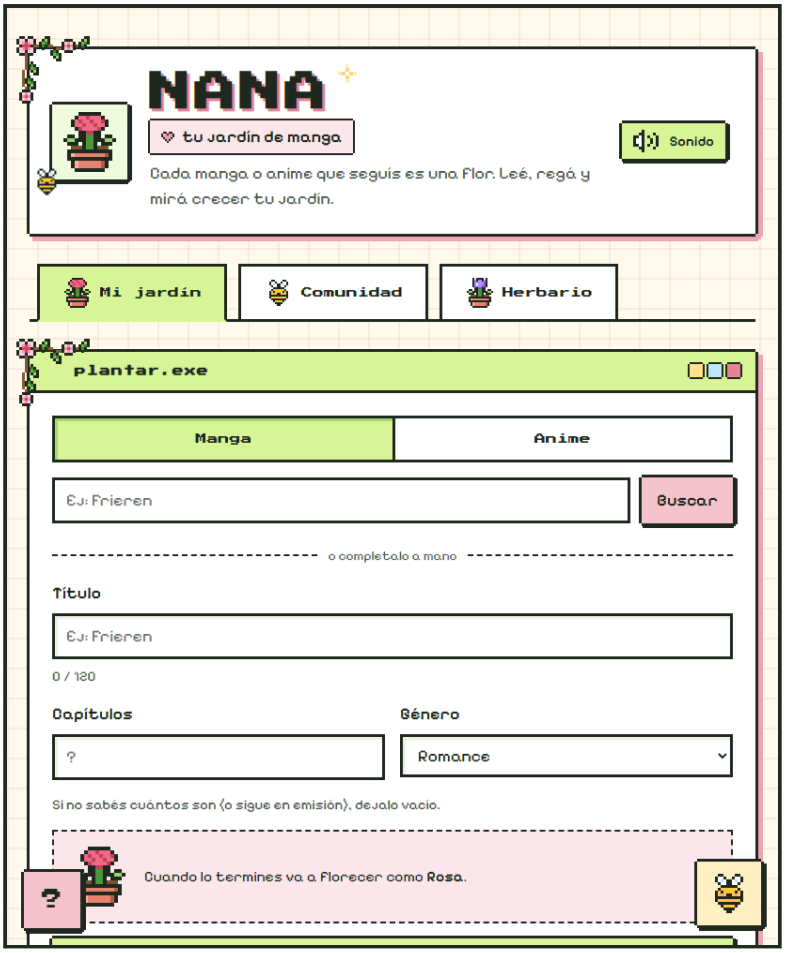
Comparar con la v1.0: más espacio, cartel con enredadera, pestañas con íconos, esquinas pixeladas y sombras rosadas.




Manga
Anime
Ej: Spy x Family
Buscar
o completalo a mano
Título
Ej: Frieren
0 / 120
Episodios
Género
?
Misterio
Si no sabés cuántos son (o sigue en emisión), dejalo vacío.
Cuando lo termines va a florecer como Hibisco.
PLANTAR
0 / 12 ep.
Pendiente
Regar +1
4 / 6 cap.
Leyendo / Viendo
Regar +1
Girasol
Brote
Haikyuu!!
Manga
1 / 8 cap.
Leyendo / Viendo
Regar +1
Rosa
En flor
nana_ia.exe
¡Hola! Soy Nana IA, la abeja jardinera. Te explico paso a paso cómo usar Nana. ¿Qué querés saber?
¿Cómo riego mi planta?
Regar suma un capítulo (o episodio) leído y hace crecer tu planta.
1. Buscá la tarjeta en tu jardín.
2. Tocá "Regar +1" cada vez que leas un capítulo.
3. La primera vez pasa a "Leyendo"; en el último capítulo florece sola.
Ej: ¿cómo riego mi planta?
Enviar
Jardinera anónima #XDSQ8
Cerrar sesión
Nana · Hecho con React + Vite + Firebase · Datos de obras: Kitsu · TP2 Desarr
NANA
tu jardín de manga
Sonido
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Mi jardín
Comunidad
Herbario
herbario.exe (9 flores)
Cada género de manga o anime florece con una flor distinta. Sus significados vienen del hanakotoba, el lenguaje japonés de las flores. Las tres últimas se eligen como avatar en tu perfil.
Rosa
En japonés: bara
Amor. Es la flor de las historias de romance.
Florece con: Romance
En tu jardín: 1 planta (1 en flor)
Girasol
En japonés: himawari
Admiración y energía: siempre mira hacia el sol, como un protagonista que no se rinde.
Florece con: Acción
En tu jardín: 1 planta
Tulipán violeta
En japonés: chūrippu
Amor eterno y nobleza, como los reinos de fantasía.
Florece con: Fantasía
En tu jardín: 1 planta
Margarita
Flor de cerezo
Hibisco


NANA
tu jardín de manga
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Sonido
Mi jardín
Comunidad
Herbario
perfil.exe
Sakura
Flor: Rosa
Eliminar perfil
mis_clubes.exe (1)
Club de las Flores
1 miembro · Admin
+ CREAR CLUB
¿Te pasaron un código?
ABC123
Unirme
club_YX83VL.exe
Club de las Flores
Código YX83VL
Invitar
Sos la administradora
Editar club
Eliminar club
Miembros (1)
Sakura (vos)
Admin
Misiones (0)
Eventos (0)
Solo mis misiones
+ NUEVA MISIÓN
POR HACER 0
Nada por acá
HACIENDO 0
Nada por acá
HECHA 0
Nada por acá
Jardinera anónima #XDSQB
Cerrar sesión
Nana · Hecho con React + Vite + Firebase · Datos de obras: Kitsu · TP2 Desarrollo Frontend II


NANA
tu jardín de manga
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Mi jardín
Comunidad
Herbario
plantar.exe
Manga
Anime
Ej: Frieren
Buscar
o completalo a mano
Título
Ej: Frieren
0 / 120
Capítulos
Género
Romance



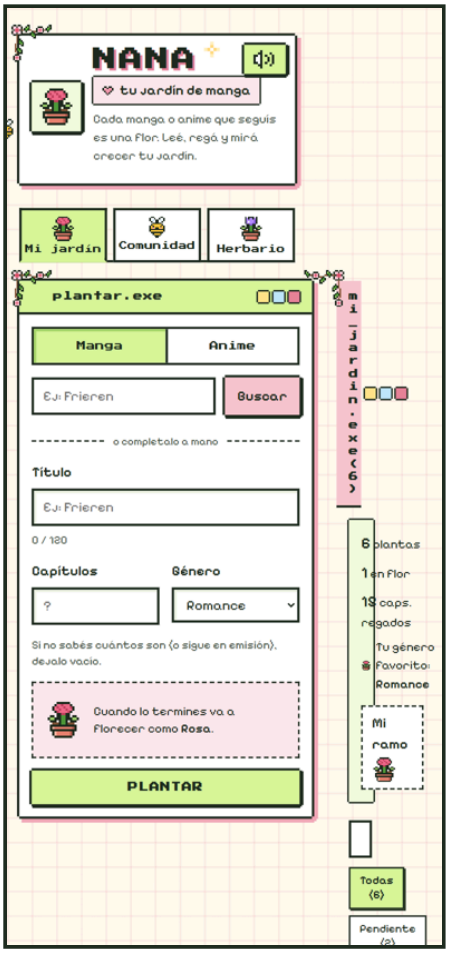
Revisión final contra la consigna y la rúbrica

Se volvió a leer la consigna completa y la tabla de evaluación.

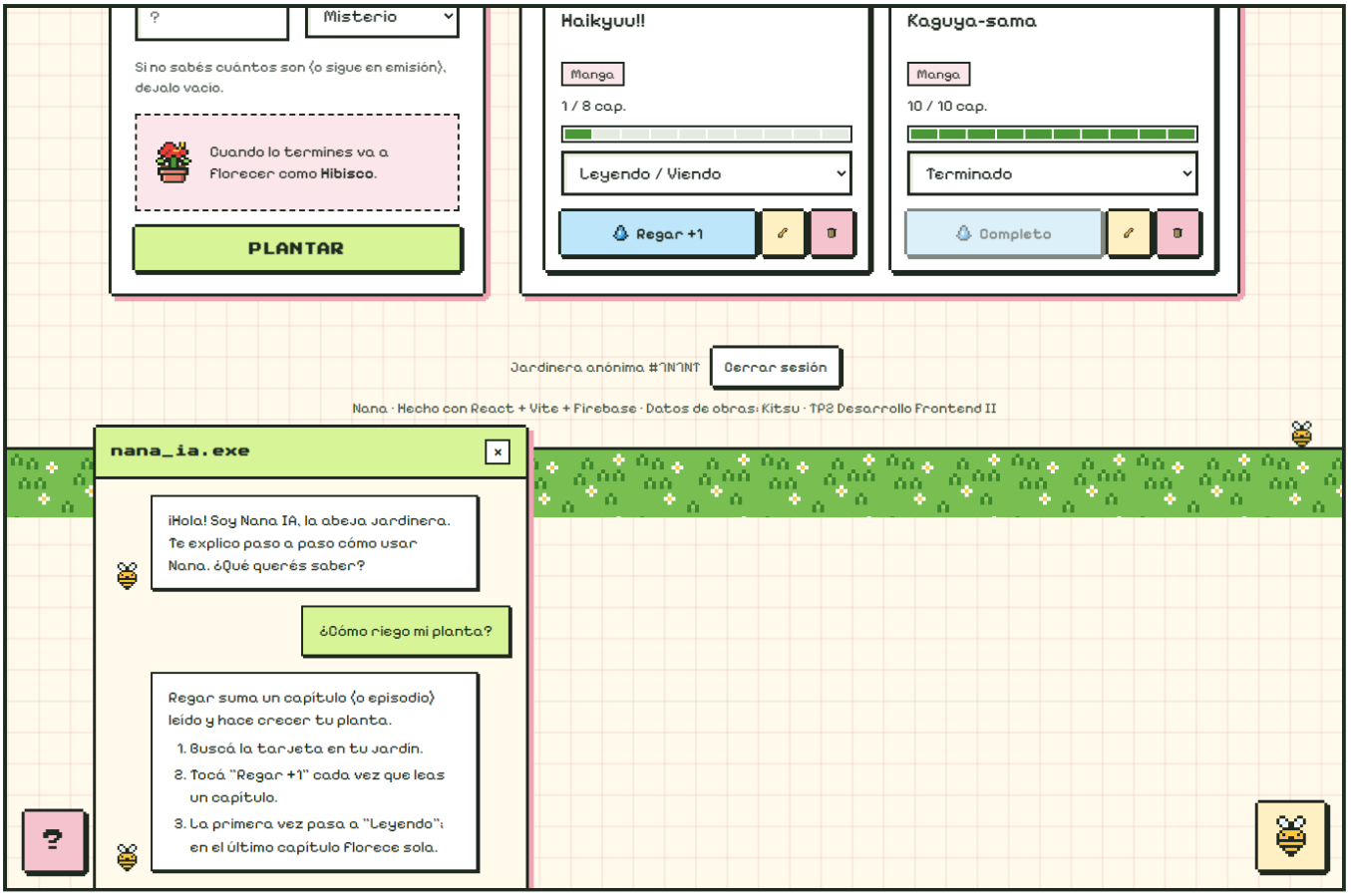
Bloque de la rúbrica	Qué se revisó	Estado
A. React + Vite con las interacciones principales	30 componentes, eventos y estado. Todas las pruebas pasan	✅
A. Firebase: consultar, agregar, modificar, eliminar y recargar	5 CRUD completos, verificados en Firestore y recargando la página	✅
A. GitHub con los archivos para ejecutar	Código, `package.json`, README con pasos	✅
A. Vercel con URL funcional y conexión a Firebase	Despliegue automático, verificado en la URL pública	✅
B. Entrega ordenada con enlaces claros	`docs/entrega.md` con la tabla de enlaces y las versiones	✅
B. README, `.env.example`, reglas; informe con IA y dificultades	Los tres archivos están en el repositorio. `.env` **no** está publicado	✅
C. Originalidad, coherencia visual, interacciones, facilidad de uso, elaboración propia	Diseño refinado en 7 versiones y justificado con referencias en esta bitácora	✅
Mini video	Lo graba la autora (guion en `entrega.md`)	⏳

Dificultades de esta versión

Problema	Cómo se detectó	Solución
En el celular el jardín quedó en dos columnas y todo se aplastaba contra la derecha. La regla nueva de la compu (`380px + resto`) se cargaba después y "pisaba" la regla del celular	Captura en 390 px	Se repitió la regla del celular dentro de `Diseno.css`, para que se cargue después
La guía y Nana IA dejaron de flotar y aparecían al final de la página. Al darle `position: relative` a todas las ventanas (para ubicar la enredadera), también se cambió la de estos paneles, que también son ventanas	Captura con el chat abierto	Se les devolvió `position: fixed` a los dos paneles
La enredadera de la derecha **generaba un scroll horizontal** de 3 píxeles en la tablet	Prueba automática de desborde (mide si la página es más ancha que la pantalla)	Se acercó la enredadera a la esquina y se agregó `overflow-x: clip`
La ventana de edición **recortaba su propio marco** (tiene scroll propio y el marco nuevo se dibuja por fuera)	Captura de la edición	Se le dio un pequeño margen interior para que entren el marco y la sombra



Antes de la corrección: en el celular el jardín quedaba en una columna angosta a la derecha.



Antes de la corrección: el chat de Nana IA aparecía al final de la página en lugar de flotar.

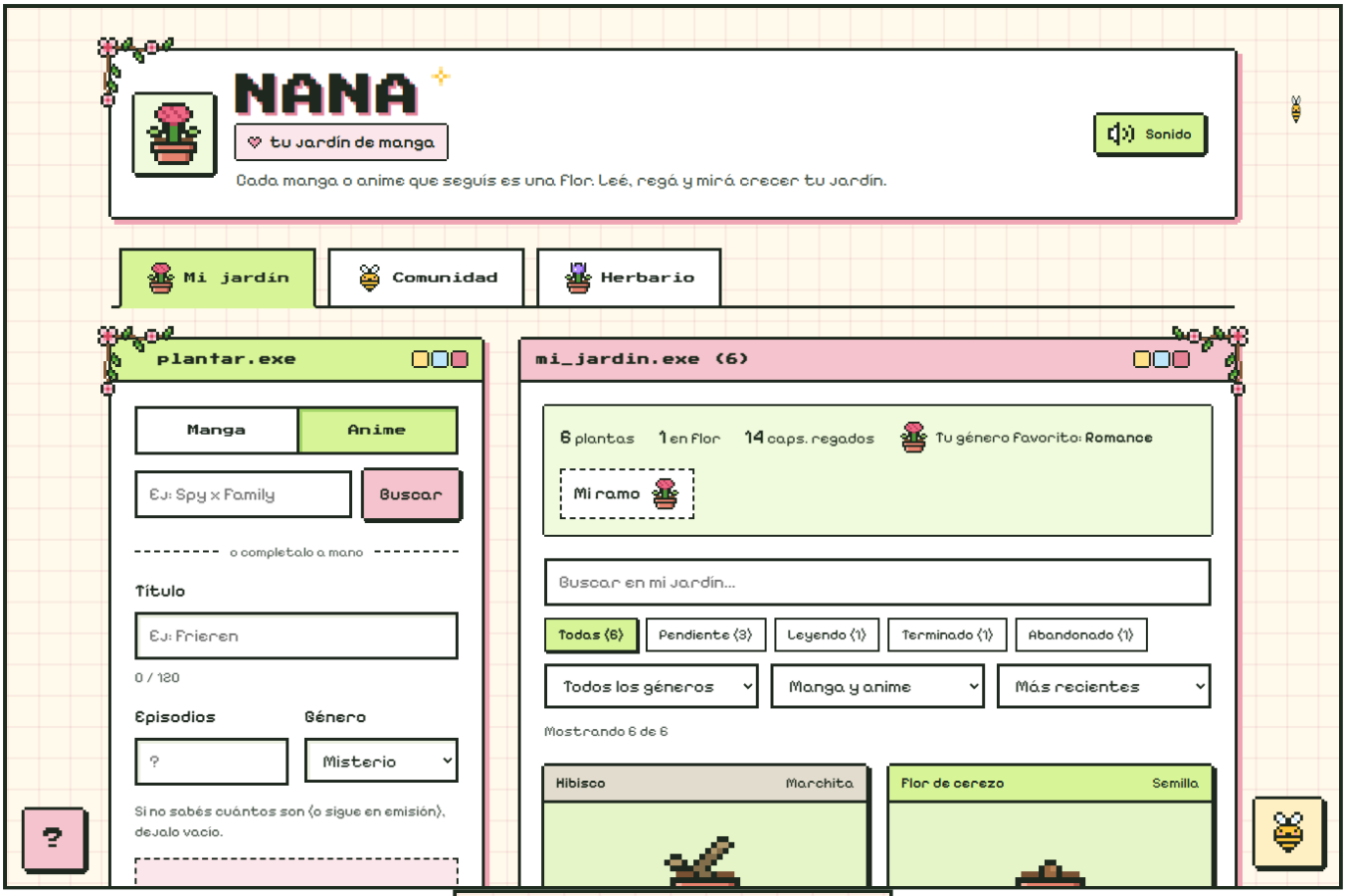
Pruebas

Prueba automática	Resultado
Diseño v1.1: sin scroll horizontal en compu (1366 px), tablet (820 px) y celular (390 px), en las 3 pestañas; sin errores	**7/7** ✅
Regresión v1.0: ramo, filtros, herbario, sonidos, Nana IA	**26/26** ✅
Regresión v0.4.1: flores, guía, Nana IA, perfil, cerrar sesión	**21/21** ✅
Regresión v0.4: comunidad con dos usuarias, roles y seguridad (403)	**19/19** ✅
Regresión v0.3: jardín, API, edición, estados	**17/17** ✅

Es un cambio solo de diseño: **no se tocó Firestore ni las reglas**, por eso no hizo falta volver a publicarlas.

Publicación y verificación en Vercel

1. Se revisó la lista de cambios antes del commit: sin archivos borrados, sin `.env` y sin llaves en el código.
2. Se subió con la etiqueta `v1.1` y Vercel la publicó solo, en segundos.
3. **Sobre la URL pública** se volvieron a correr **todas** las pruebas: diseño (7/7), v1.0 (26/26), v0.4.1 (21/21), comunidad con dos usuarias y seguridad (19/19) y jardín (17/17). En total, **90 de 90 correctas** y ningún error en la consola.


NANA
tu jardín de manga
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Sonido
Mi jardín
Comunidad
Herbario
plantar.exe
Manga
Anime
Ej: Spy x Family
Buscar
o completalo a mano
Título
Ej: Frieren
0 / 120
Episodios
Género
?
Misterio
Si no sabés cuántos son (o sigue en emisión), dejalo vacío.
mi_jardin.exe (6)
6 plantas
1 en flor
14 caps. regados
Tu género favorito: Romance
Mi ramo
Buscar en mi jardín...
Todas (6)
Pendiente (3)
Leyendo (1)
Terminado (1)
Abandonado (1)
Todos los géneros
Manga y anime
Más recientes
Mostrando 6 de 6
Hibisco
Marchita
Flor de cerezo
Semilla


NANA
tu jardín de manga
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Mi jardín
Comunidad
Herbario
plantar.exe
Manga
Anime
Ej: Frieren
Buscar
o completalo a mano
Título
Ej: Frieren
0 / 120
Capítulos
Género
Romance
?

Preparación de la entrega (10/11/2026)

- **Guion del mini video:** se escribió docs/guion-video.md, con 10 escenas. Cada escena dice qué mostrar en pantalla y qué decir. Cubre lo que pide la consigna: la demostración, dónde se configura Firebase y cómo se guarda y se modifica un registro (en la app, en la consola de Firestore y en el código). Avisa además que **no hay que mostrar el archivo** `.env` en el video.
- **"La app debe estar operativa":**
  - Se volvió a probar la URL pública y la prueba del jardín dio **17/17**.
  - Se confirmó que las reglas de Firestore no tienen fecha de vencimiento.
  - Se armó un checklist para el día de la entrega en docs/entrega.md.

Versión 1.2 — Puntuación con sakuras y reseña corta (13/11/2026)

Por qué un sprint más

Al repasar la app completa, la autora sintió que **"le faltaba algo"**. Las ideas iniciales eran dos:

- una sección de **favoritos / "quiero ver"**;
- **puntuar** las obras, como en Letterboxd.

La condición era **no saturar el sitio**, porque lo principal sigue siendo anotar cómo va la lectura. También tenía que ser **coherente** con todo lo anterior y poder **defenderse en el final**.

Decisión: qué se agregó y qué no

Idea	¿Se hizo?	Por qué
Lista "quiero leer / quiero ver"	❌ No hacía falta	**Ya existe:** es el estado **Pendiente**. Plantás la obra, queda como semilla, y con el filtro "Pendiente" ves la lista. Una sección aparte repetiría lo mismo en dos lugares
Sección de favoritos	❌ No como sección	Se resuelve **sin pantallas nuevas**: con el orden **"Mejor puntuadas"** las favoritas quedan arriba
Puntuación de 1 a 5 sakuras + reseña corta	✅ **Sí**	Es lo que le faltaba al ciclo del jardín: hoy plantás, regás y florece… y ahí terminaba. Ahora, **cuando la historia termina para vos** (la terminaste o la abandonaste), podés decir qué te pareció

La **sakura** (flor de cerezo) se eligió en lugar de estrellas porque ya era parte del diseño: las enredaderas de cerezo de la v1.1. Además es un símbolo muy japonés, que encaja con el manga.

Historia de usuario (metodología ágil)

> **Como** lectora, **quiero** puntuar y reseñar lo que termino, **para** recordar qué me pareció y encontrar mis favoritas.

Criterios de aceptación (cada uno se comprobó con una prueba automática):

1. Una planta **pendiente o leyendo** no muestra la puntuación: todavía no la terminaste.
2. Una planta **terminada o abandonada** muestra "¿Qué te pareció?" con 5 sakuras.
3. Al tocar una sakura aparece un aviso ("Le diste 4 sakuras a…") y **sigue ahí al recargar**.
4. Tocar **la misma sakura** otra vez quita la puntuación, y **el dato se borra** de Firestore.
5. La reseña se escribe en la ventana de edición, tiene **hasta 280 caracteres** y, si se pasa, muestra un error y no guarda.

6. "Mi ramo" muestra el **promedio** de las puntuaciones, y el jardín se puede ordenar por **"Mejor puntuadas"**.
7. **Firebase rechaza** una puntuación fuera de 1 a 5 o una reseña demasiado larga, aunque alguien se saltee la interfaz.
8. Funciona en celular, sin scroll horizontal y con botones de al menos 32 px.

Cómo cumple el CRUD

Operación	En la puntuación y la reseña	Código
Crear	Tocar una sakura por primera vez o escribir la reseña	`actualizarPlanta(uid, id, { puntuacion: 4 })` → `updateDoc` agrega el campo
Consultar	Se ve en la tarjeta, en el promedio de "Mi ramo" y en el orden "Mejor puntuadas". Sigue al recargar	`obtenerPlantas` (`getDocs`)
Modificar	Tocar otra sakura o editar el texto	`updateDoc` cambia el campo
Eliminar	Tocar la misma sakura, o vaciar la reseña y guardar	El valor `null` se convierte en `deleteField()` , que **borra el campo** del documento

¿Por qué `deleteField()` y no guardar un 0 o un texto vacío? Porque "sin puntuar" no es lo mismo que "puntuación 0". Si el campo no existe, la base queda limpia, y las reglas pueden exigir que, **si existe**, valga de 1 a 5.

Archivos que cambiaron

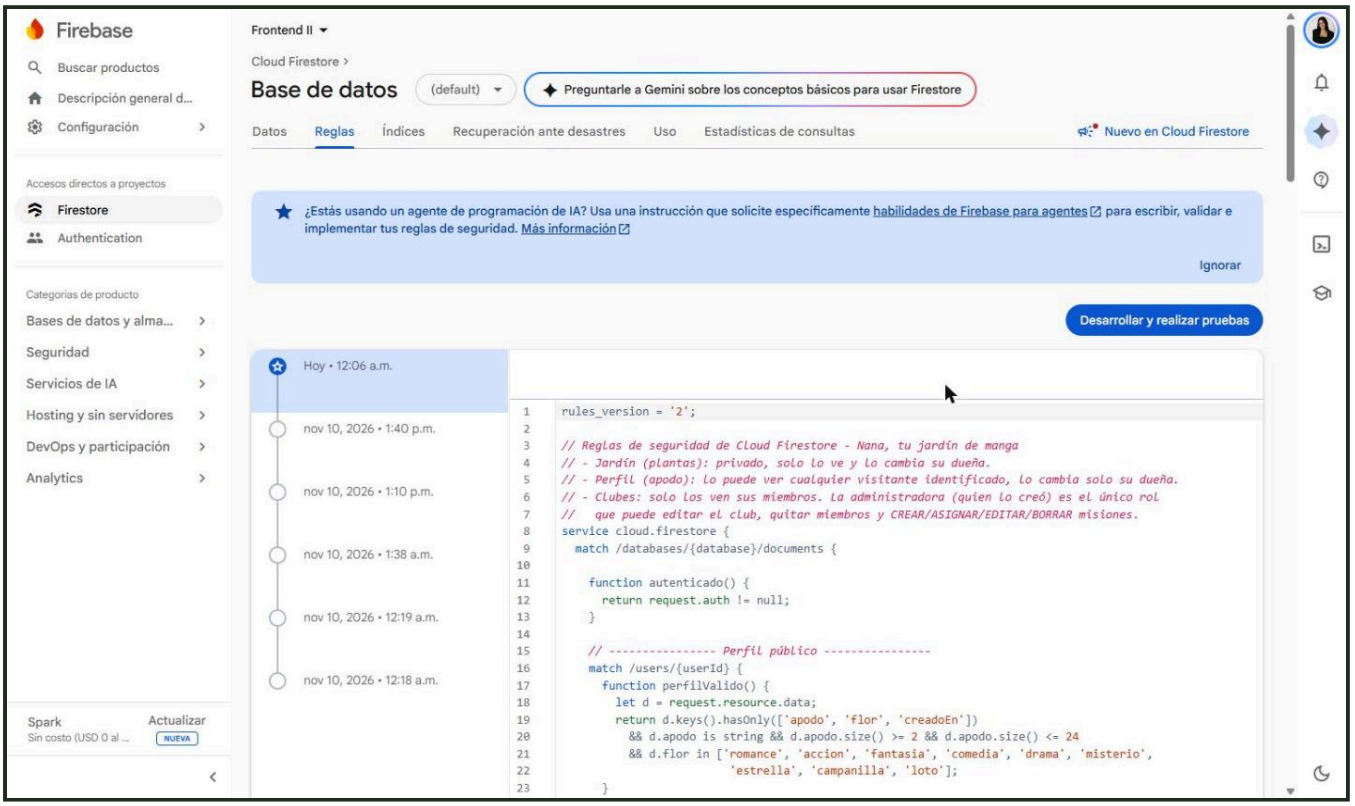
Archivo	Qué se hizo
`src/components/Puntuacion.jsx` (**nuevo**)	Las 5 sakuras. Se pintan al pasar el mouse (estado `previa` con `useState`). Tiene un modo "solo mirar" para el promedio
`src/components/TarjetaPlanta.jsx`	Muestra "¿Qué te pareció?" si la planta terminó (renderizado condicional) y la función `puntuar(n)`
`src/components/EditarPlanta.jsx`	Sección "Tu reseña" con la puntuación y un campo de texto con contador
`src/components/JardinFiltrado.jsx`	Orden "Mejor puntuadas" (`sort`) y promedio en "Mi ramo" (`filter` + `reduce`)
`src/services/plantas.js`	`actualizarPlanta` convierte los `null` en `deleteField()`
`src/data/validaciones.js` y `flores.js`	`validarResena` y el máximo `MAX_RESENA = 280`
`src/data/decoracion.js`	El dibujo pixelado de la sakura (un array de texto, como las demás flores)
`src/data/conocimiento.js`	Nana IA explica cómo puntuar, y que la lista "quiero leer" es el estado Pendiente
`src/Diseno.css`	Estilos de las sakuras, la reseña y el promedio
`firestore.rules`	Acepta los dos campos nuevos **solo si son válidos**

Base de datos: reglas nuevas

Las reglas solo aceptan campos conocidos (`hasOnly`). Por eso, **antes** de subir el código, se agregaron los dos campos nuevos con su validación:

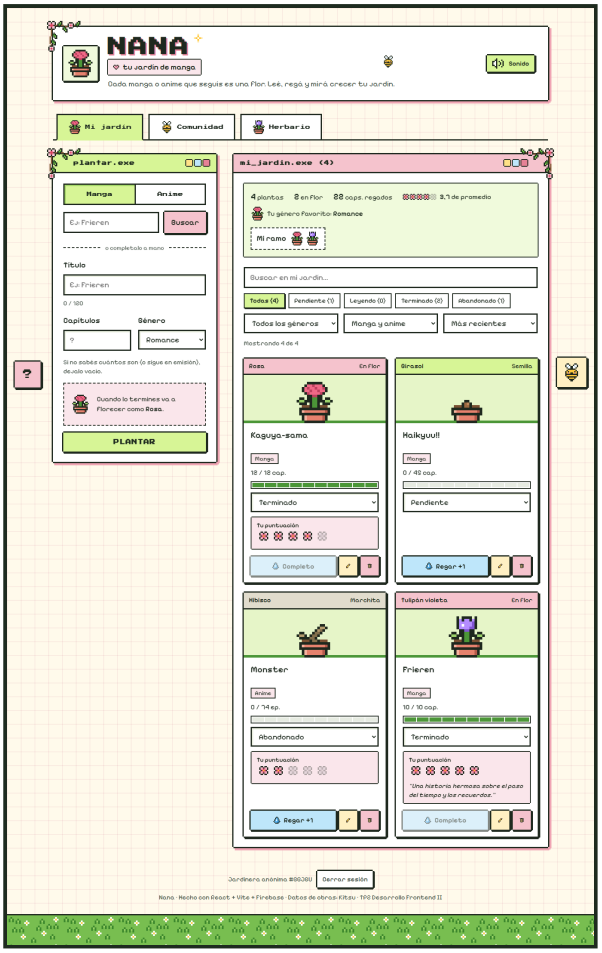
```
'puntuacion', 'resena'   ← sumados a la lista de campos permitidos
&& (!('puntuacion' in d) || (d.puntuacion is int && d.puntuacion >= 1 && d.puntuacion <= 5))
&& (!('resena' in d) || (d.resena is string && d.resena.size() > 0 && d.resena.size() <= 280));
```

Se leen así: "si existe la puntuación, tiene que ser un número entero de 1 a 5", y "si existe la reseña, tiene que ser un texto de 1 a 280 caracteres".

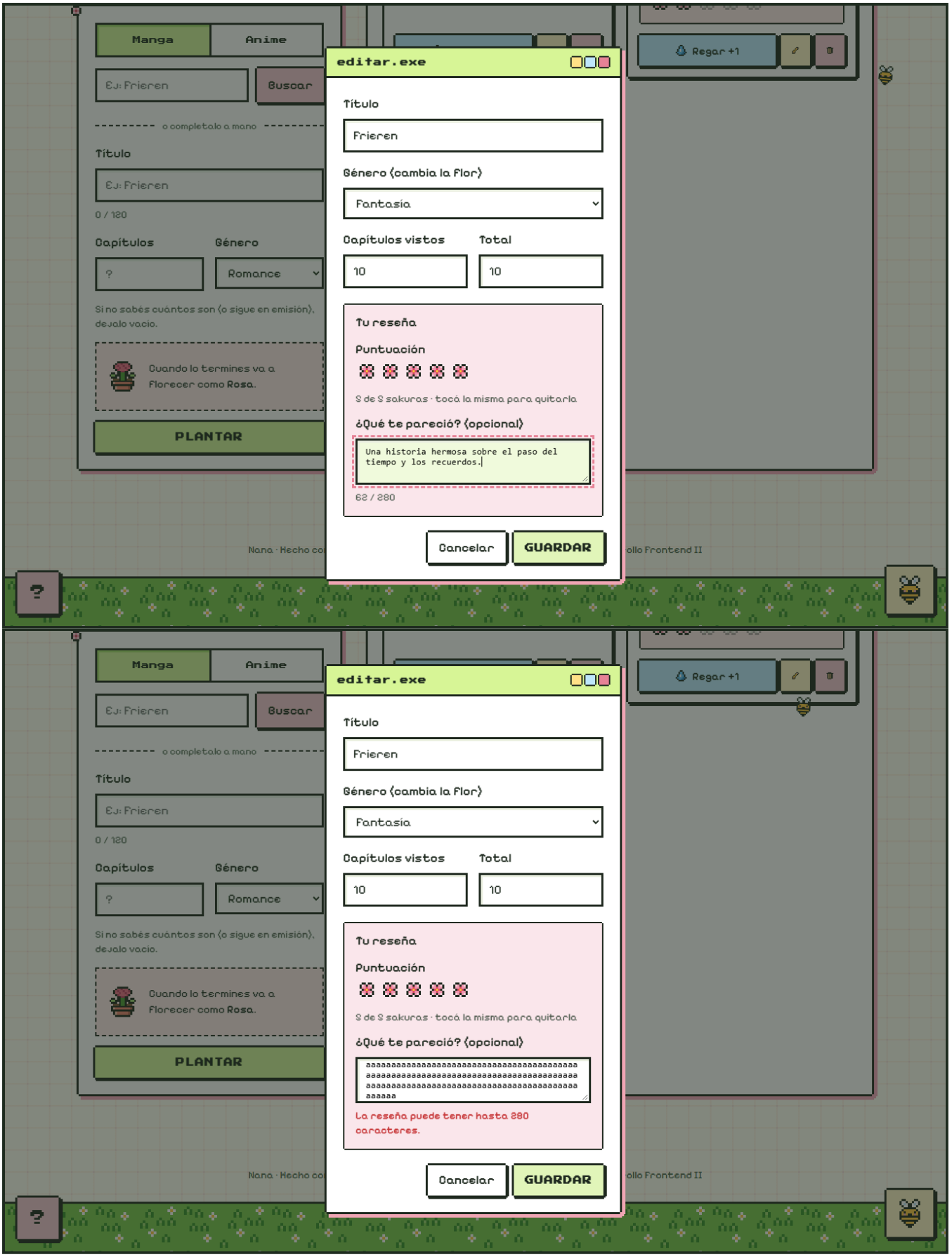


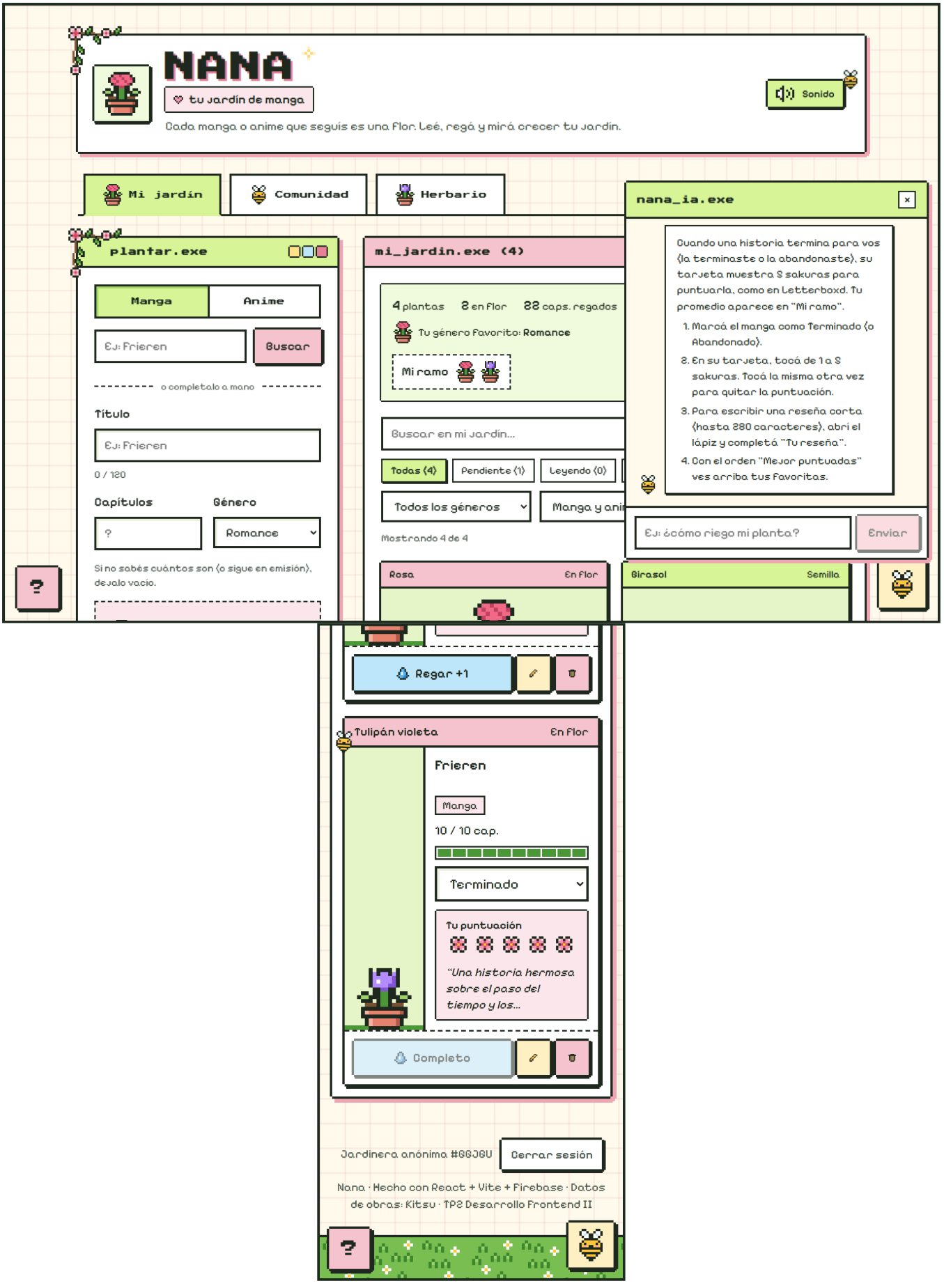
Las reglas nuevas publicadas en la consola de Firebase (Firestore → Reglas), con la versión de hoy arriba en el historial.

Capturas



En "Mi ramo" se ve el promedio (3,7). Haikyuu!! está pendiente, así que no muestra sakuras. Monster está abandonado y también se puede puntuar. Frieren tiene 5 sakuras y su reseña.


Manga
Anime
Ej: Frieren
Buscar
o completalo a mano
Título
Ej: Frieren
0 / 120
Capítulos
Género
?
Romance
Si no sabés cuántos son (o sigue en emisión), dejalo vacío.
Cuando lo termines va a Florecer como Rosa.
PLANTAR
editar.exe
Título
Frieren
Género (cambia la Flor)
Fantasía
Capítulos vistos
Total
10
10
Tu reseña
Puntuación
5 de 5 sakuras · tocá la misma para quitarla
¿Qué te pareció? (opcional)
Una historia hermosa sobre el paso del tiempo y los recuerdos.
62 / 280
Cancelar
GUARDAR
Regar +1
Nana · Hecho co
ollo Frontend II
?
Manga
Anime
Ej: Frieren
Buscar
o completalo a mano
Título
Ej: Frieren
0 / 120
Capítulos
Género
?
Romance
Si no sabés cuántos son (o sigue en emisión), dejalo vacío.
Cuando lo termines va a Florecer como Rosa.
PLANTAR
editar.exe
Título
Frieren
Género (cambia la Flor)
Fantasía
Capítulos vistos
Total
10
10
Tu reseña
Puntuación
5 de 5 sakuras · tocá la misma para quitarla
¿Qué te pareció? (opcional)
aaaa


NANA
tu jardín de manga
Sonido
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Mi jardín
Comunidad
Herbario
plantar.exe
Manga
Anime
Ej: Frieren
Buscar
o completalo a mano
Título
Ej: Frieren
0 / 120
Capítulos
Género
?
Romance
Si no sabés cuántos son (o sigue en emisión), dejalo vacío.
mi_jardin.exe (4)
4 plantas
2 en flor
22 caps. regados
Tu género favorito: Romance
Mi ramo
Buscar en mi jardín...
Todas (4)
Pendiente (1)
Leyendo (0)
Todos los géneros
Manga y ani
Mostrando 4 de 4
Rosa
En flor
Girasol
Semilla
nana_ia.exe
Cuando una historia termina para vos (la terminaste o la abandonaste), su tarjeta muestra 5 sakuras para puntuarla, como en Letterboxd. Tu promedio aparece en "Mi ramo".
1. Marcá el manga como Terminado (o Abandonado).
2. En su tarjeta, tocá de 1 a 5 sakuras. Tocá la misma otra vez para quitar la puntuación.
3. Para escribir una reseña corta (hasta 280 caracteres), abrí el lápiz y completá "Tu reseña".
4. Con el orden "Mejor puntuadas" ves arriba tus favoritas.
Ej: ¿cómo riego mi planta?
Enviar
Regar +1
Tulipán violeta
En flor
Frieren
Manga
10 / 10 cap.
Terminado
Tu puntuación
"Una historia hermosa sobre el paso del tiempo y los...
Completo
Jardinera anónima #8BJGU
Cerrar sesión
Nana · Hecho con React + Vite + Firebase · Datos de obras: Kitsu · TP2 Desarrollo Frontend II

Uso de IA en esta etapa

La autora planteó la idea ("favoritos o puntuar como en Letterboxd, sin saturar el sitio") y pidió que **cada acción respete el CRUD y la metodología ágil**. Claude Code:

- comparó las opciones y notó que "quiero leer" ya existía como estado Pendiente;
- propuso la puntuación con sakuras;
- escribió el código, las reglas y la prueba automática.

La autora revisó y aprobó la idea, y las reglas se publicaron con su autorización.

Dificultades y cómo se resolvieron

Problema	Cómo se detectó	Solución
Firebase rechazaba la puntuación ("Missing or insufficient permissions")	La primera prueba en local dio 7 de 17	Era lo esperado: las reglas viejas no conocían los campos nuevos (`hasOnly`). Se publicaron las reglas nuevas y la prueba pasó completa. **Esto demuestra que las reglas protegen la base:** nadie puede agregar campos que no estén permitidos
¿Cómo "eliminar" una puntuación?	Al pensar el CRUD de la función	Con `deleteField()`, que borra el campo. Guardar un 0 habría roto la regla "de 1 a 5"
El error de la reseña **no se borraba** al corregir el texto, y tapaba el contador	Captura de la ventana de edición	Al escribir se quita el error y vuelve el contador ("62 / 280")
La prueba del celular no encontraba las tarjetas	La prueba automática	No era un error de la app: el navegador automático **recarga la página** al pasar a modo celular. La prueba ahora espera a que el jardín vuelva a cargar

Pruebas

Prueba automática	Resultado
v1.2: puntuación y reseña (los 8 criterios de aceptación, el CRUD completo, recargas, seguridad con 403, compu y celular)	**26/26** ✅
Regresión v1.0: ramo, filtros, herbario, sonidos, Nana IA	**26/26** ✅
Regresión v0.4.1: flores, guía, Nana IA, perfil, cerrar sesión	**21/21** ✅
Regresión v0.4: comunidad con dos usuarias, roles y seguridad (403)	**19/19** ✅
Regresión v0.3: jardín, API, edición, estados	**17/17** ✅
Diseño v1.1: sin scroll horizontal en compu, tablet y celular	**7/7** ✅

En total: **116 de 116 correctas** en local.

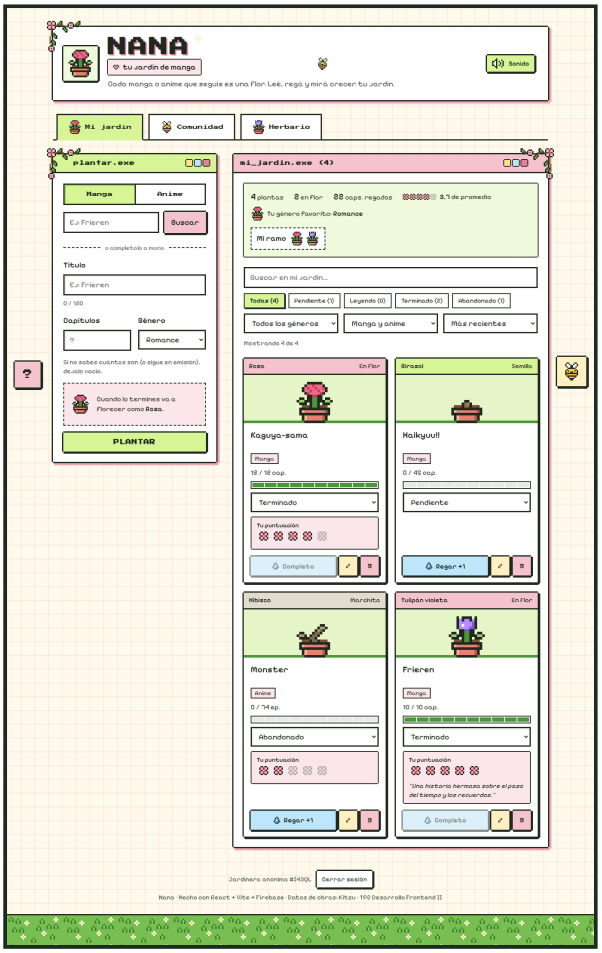
Prueba de seguridad: la prueba llama **directo a Firebase**, sin pasar por la app, e intenta guardar una puntuación de **9 sakuras** y una reseña de **400 caracteres**. Firebase responde **403 (rechazado)** las dos

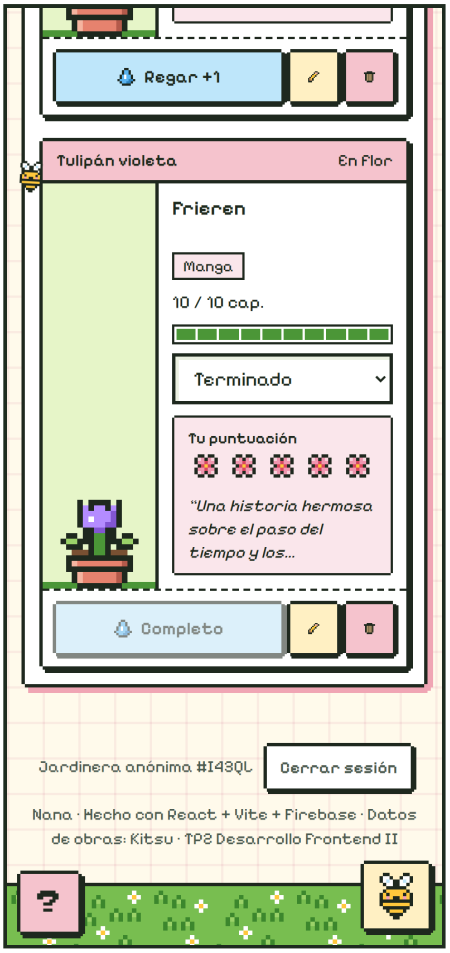
veces. Una puntuación válida responde 200 (aceptada).

Publicación y verificación en Vercel

1. **Primero las reglas:** se publicaron en la consola de Firebase (Firestore → Reglas → Publicar), con la autorización de la autora.
2. Se revisó la lista de cambios antes del commit: sin archivos borrados, sin `.env` (sigue ignorado; solo está `.env.example`) y sin llaves en el código.
3. Se subió con la etiqueta `v1.2`, y Vercel la publicó sola en menos de un minuto.
4. **Sobre la URL pública** se volvieron a correr **todas** las pruebas:

Prueba	Resultado
v1.2: puntuación y reseña	26/26
v1.0	26/26
v0.4.1	21/21
Comunidad con dos usuarias y seguridad	19/19
Jardín	17/17
Diseño	7/7
Total	**116 de 116 correctas**, sin errores en la consola





Revisión del sprint (retrospectiva)

- **Qué salió bien:** la función nueva **no agrega pantallas ni botones sueltos**. Aparece sola cuando la planta termina, así que el sitio no se satura. Se reutilizaron piezas que ya existían: el aviso, los sonidos, el modal de edición, los filtros y el dibujo con arrays.
- **Qué se aprendió:** antes de agregar una función conviene revisar si **ya existe** de otra forma (el "quiero leer" era el estado Pendiente).
- **Orden correcto:** primero las reglas, después el código. Si se sube el código antes, la app publicada falla al puntuar.

Cierre del sprint v1.2: Diario, música y revisión final (13/11/2026)

La autora probó la versión publicada y pidió **sumar todo a la v1.2**, sin abrir una versión nueva, porque es la versión final. Sus pedidos fueron:

1. **Un lugar donde queden guardadas las puntuaciones y lo que leyó**, como el perfil de Letterboxd: una sección de "vistos" o "puntuados".
2. **Más espacio** en la ventana de clubes (el texto quedaba pegado al botón "Crear club"), y revisar el espaciado de toda la app.
3. **Una canción kawaii de fondo**, que se pueda silenciar con el botón Sonido.

4. Cualquier otra acción útil que respete la consigna y el CRUD, y detalles más tiernos, siempre en pixel art y con la misma paleta.
5. Una **revisión general** de GitHub, Firebase, Vercel y el código contra la consigna y la rúbrica.

Historias de usuario de este cierre

#	Historia	Criterios de aceptación (todos probados)
H1	*Como* lectora, *quiero* un diario con todo lo que leí y puntué, *para* ver mi historial como en Letterboxd	Pestaña nueva "Diario". Muestra solo lo terminado o abandonado, agrupado por mes, con la fecha, la portada o la flor, las sakuras y la reseña. Tiene un resumen y un gráfico de puntuaciones
H2	*Como* lectora, *quiero* que quede anotado **cuándo** terminé cada historia y poder corregirlo	Al terminar o abandonar, se guarda la fecha de hoy. Se puede cambiar en la edición. No acepta fechas del futuro. Si la planta vuelve a "Leyendo", la fecha se borra
H3	*Como* lectora, *quiero* puntuar, reseñar y borrar mi reseña desde el diario	Las sakuras funcionan en el diario. "Borrar reseña" pide confirmación y borra los dos campos, pero la lectura sigue en el diario
H4	*Como* visitante, *quiero* música de fondo tierna que pueda apagar	Empieza con el primer toque en la página. El botón Sonido la apaga y la prende
H5	*Como* visitante, *quiero* una interfaz con más aire	El texto de "Mis clubes" ya no queda pegado al botón. Las 4 pestañas entran completas en el celular

Decisión: ¿por qué el Diario no es una colección nueva en Firestore?

Se pensó en crear una colección aparte (`diario/`), pero habría **duplicado** los datos. Si la autora cambiaba la puntuación en el jardín, había que actualizarla en dos lugares. En cambio, el Diario es **otra forma de mirar los mismos documentos** (`users/{uid}/plantas`): filtra los que están terminados o abandonados (`filter`), los ordena (`sort`) y los agrupa por mes (`reduce`).

Solo se agregó **un campo nuevo**, `terminadoEn` (texto con formato AAAA-MM-DD), para saber cuándo terminó cada historia. Es la misma idea que en Letterboxd, donde cada película vista tiene su fecha.

Cómo cumple el CRUD el Diario

Operación	Qué hace la usuaria	Qué pasa en Firebase
Crear	Marca una planta como Terminado o Abandonado (o la riega hasta el último capítulo)	`conFechaDeCierre` agrega `terminadoEn` = hoy en el mismo `updateDoc` , y la lectura **aparece sola** en el diario
Consultar	Abre la pestaña Diario: lecturas por mes, resumen (leídas, puntuadas, reseñas, promedio) y gráfico de barras. Filtros y 4 órdenes	`getDocs` de las plantas + `filter` , `sort` y `reduce` en React
Modificar	Toca sakuras, escribe la reseña o **corrige la fecha** en la edición	`updateDoc` de `puntuacion` , `resena` o `terminadoEn`
Eliminar	"Borrar reseña" (con confirmación) quita sakuras y texto. Si la planta vuelve a "Leyendo", sale del diario. Compostar la planta borra todo	`deleteField()` sobre `puntuacion` y `resena` , o sobre `terminadoEn` . `deleteDoc` al compostar

Música de fondo: original y sin archivos

La canción es **original** (compuesta para Nana, no es de nadie). Tiene 8 compases en do mayor, con melodía de "cajita musical" y un bajo suave. Igual que los sonidos de los clics, se toca **nota por nota con la Web Audio API**, así que no hay archivos de audio ni derechos de autor de por medio. El archivo es `src/services/musica.js`.

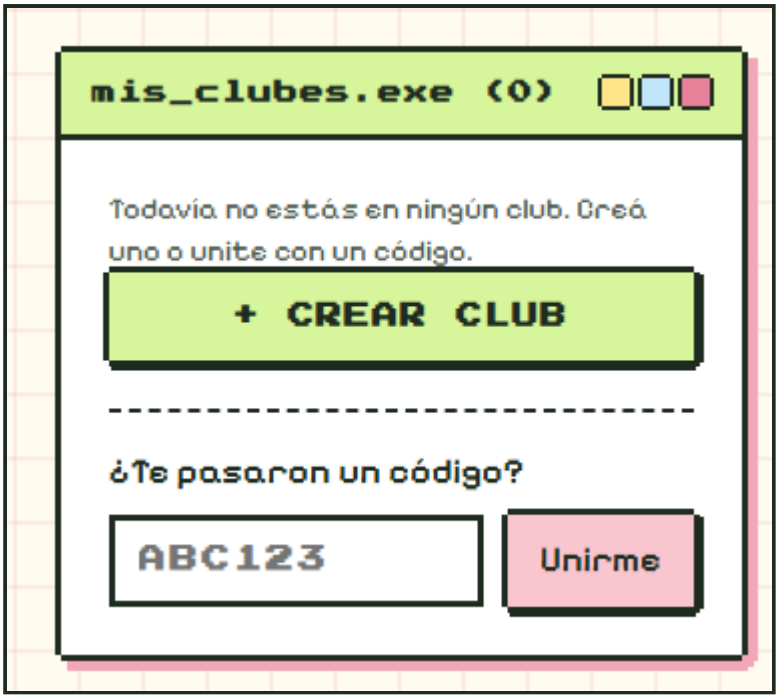
- Un "programador" revisa cada 100 ms (`setInterval`) qué notas tocan en el próximo cuarto de segundo, y las agenda con el reloj del audio. Así la música no se atrasa.
- **Los navegadores no dejan sonar nada hasta que la persona toca la página.** Por eso la música empieza con el primer toque: un `useEffect` en `App.jsx` escucha el primer `pointerdown` y quita el "escuchador" en la limpieza.
- El **botón Sonido** apaga y prende todo (música y efectos). La música entra y sale con un fundido suave, y se pausa si la pestaña queda en segundo plano.
- El volumen es bajo (6 %), para que acompañe y no moleste.

Detalles kawaii nuevos (misma paleta, pixel art)

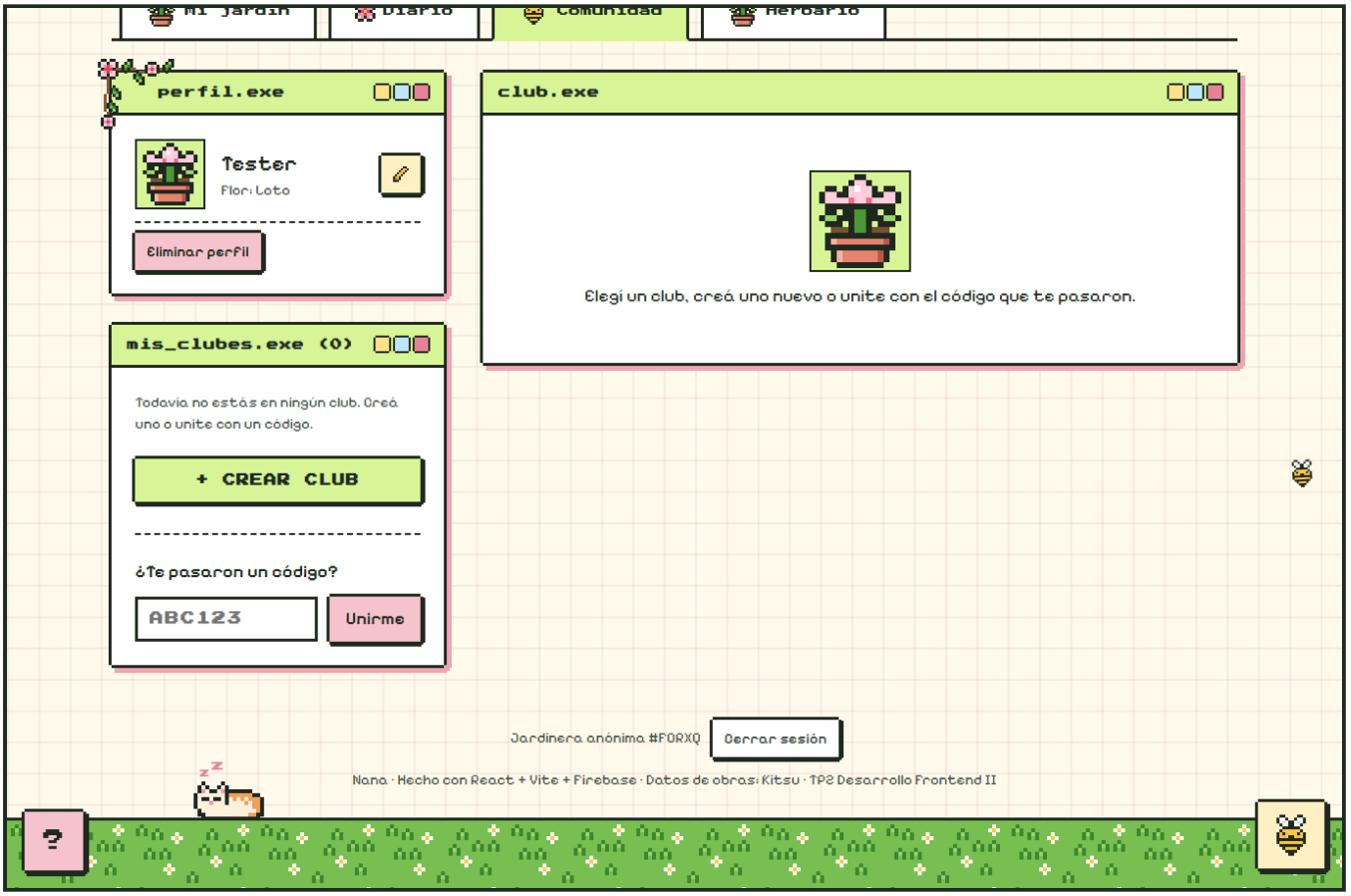
- **Gatito dormilón** sobre el pasto del pie de página, inspirado en las referencias de gatitos pixelados que sumó la autora. Está dibujado con código (un array de texto, como las flores). Duerme con unas "z" que suben y bajan, y **si lo tocás maúlla y aparece un corazón**. El maullido también se genera con Web Audio.
- El mismo gatito acompaña el **diario vacío**, invitando a terminar el primer manga.
- La pestaña Diario usa la **sakura** como ícono, y los meses del diario parecen pestañas de un cuaderno rosa.
- El gráfico de puntuaciones tiene **barras pixeladas a rayas rosas**.

Espaciado

- **"Mis clubes":** el texto "Todavía no estás en ningún club" ahora tiene 24 px de aire antes del botón (antes, unos 4 px). También se separó más la sección "¿Te pasaron un código?".
- **Pestañas en el celular:** con 4 pestañas, "Herbario" quedaba **cortada** en el borde de la pantalla. Ahora son 4 columnas iguales con la fuente pixelada a 8–9 px (está diseñada en múltiplos de 8, así que se ve nítida).
- Se revisaron con capturas el jardín, el diario, la comunidad, el herbario y la edición, en compu y en celular.



Antes (captura de la autora): el texto casi toca el botón "Crear club".



Después: 24 px de separación y más espacio arriba de "¿Te pasaron un código?".

Base de datos: reglas actualizadas

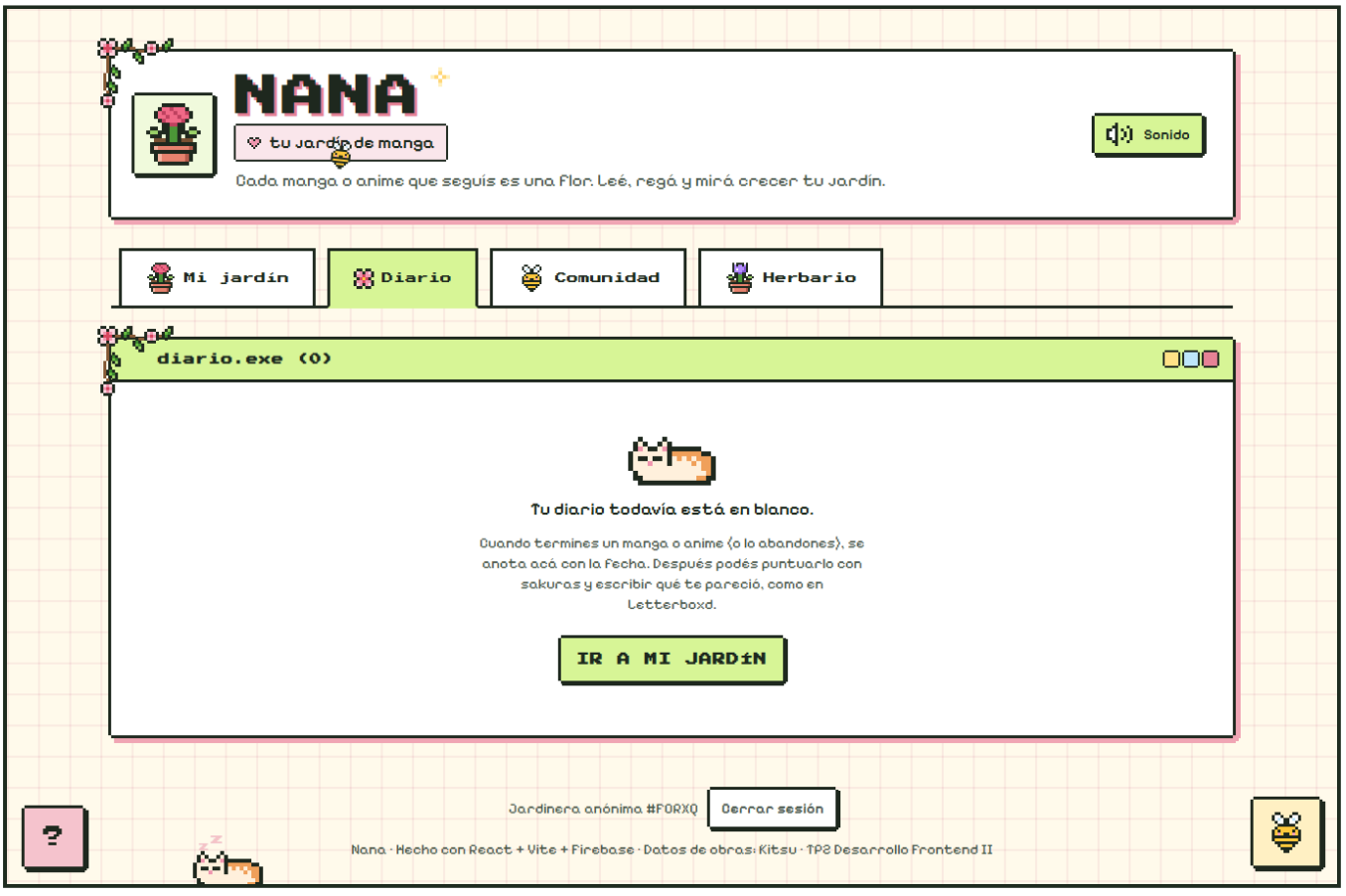
Se agregó `terminadoEn` a la lista de campos permitidos, con su validación:

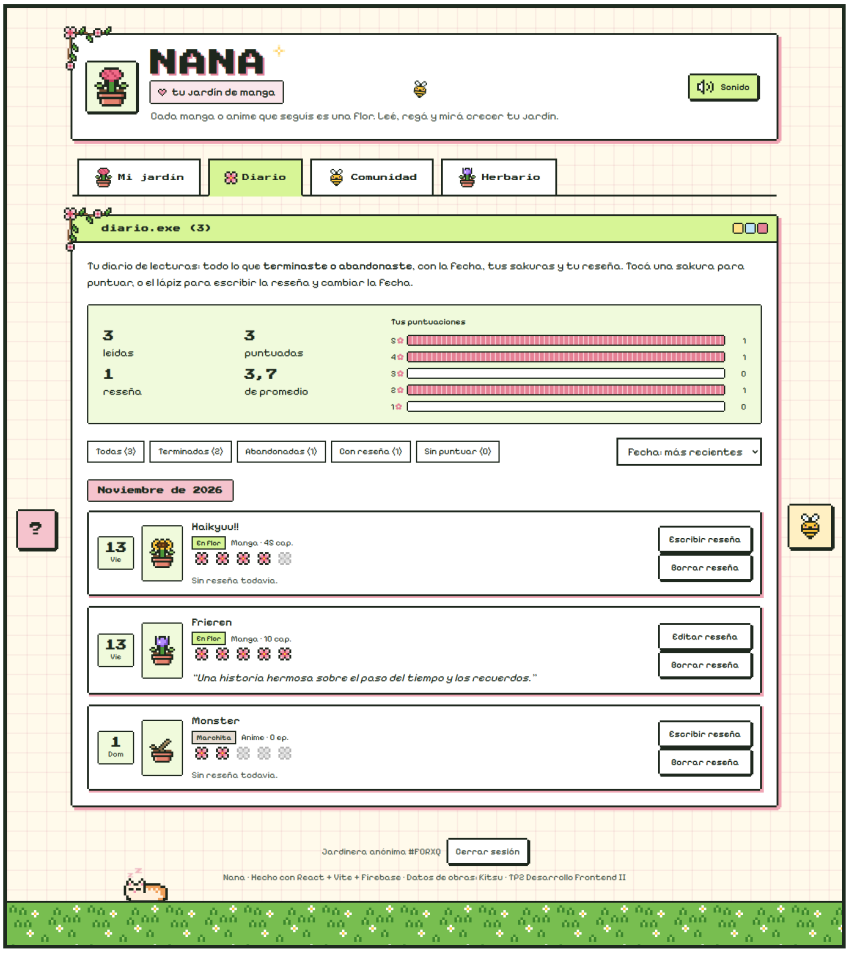
```
&& (!('terminadoEn' in d) || (d.terminadoEn is string
                              && d.terminadoEn.matches('^[0-9]{4}-[0-9]{2}-[0-9]{2}$')));
```

Se lee así: "si existe la fecha, tiene que ser un texto con forma de fecha (cuatro números, guion, dos, guion, dos)". Las reglas se publicaron **antes** de subir el código, con la autorización de la autora.

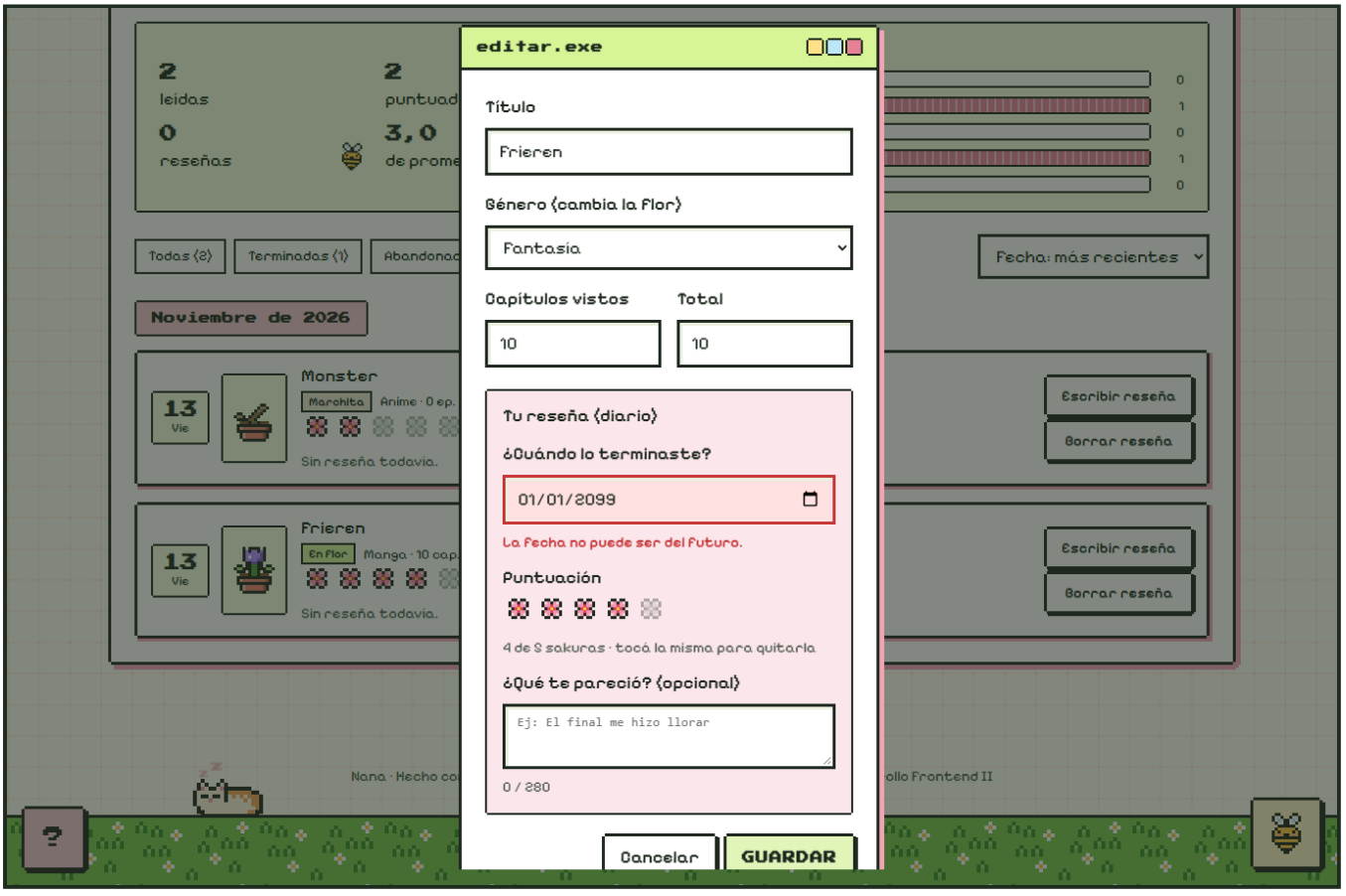


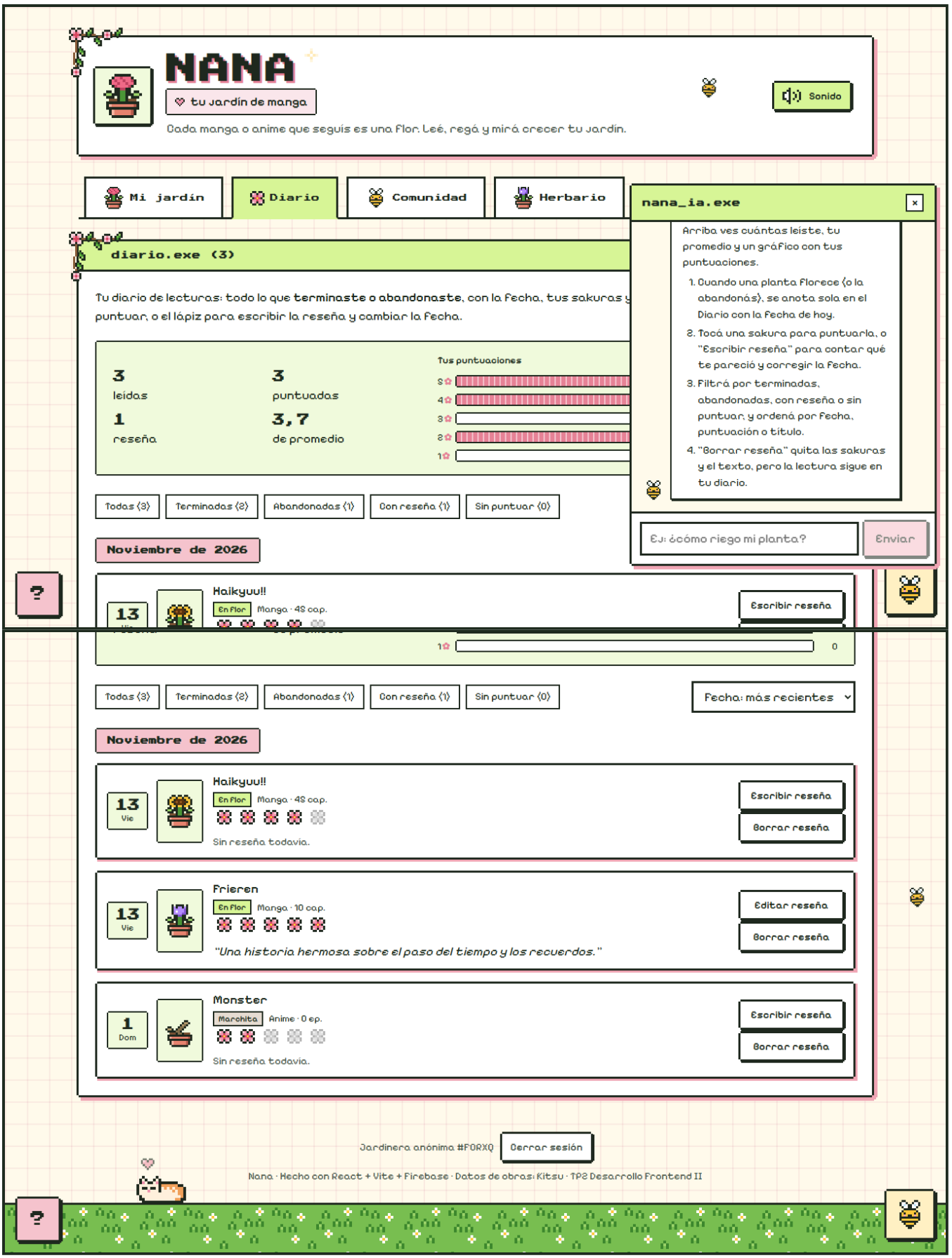
Capturas del Diario

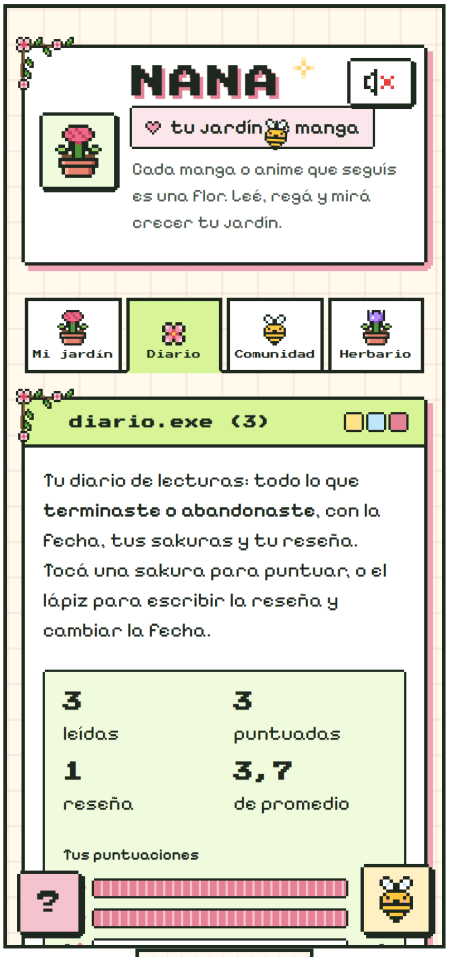




Resumen (3 leídas, 3 puntuadas, 1 reseña, 3,7 de promedio), gráfico de puntuaciones, filtros, el mes como pestaña de cuaderno y cada lectura con su día, su flor, sus sakuras y su reseña.




NANA
tu jardín de manga
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Sonido
Mi jardín
Diario
Comunidad
Herbario
diario.exe (3)
Tu diario de lecturas: todo lo que terminaste o abandonaste, con la fecha, tus sakuras y
puntuar, o el lápiz para escribir la reseña y cambiar la fecha.
3
leídas
3
puntuadas
1
reseña
3,7
de promedio
Tus puntuaciones
5
4
3
2
1
Todas (3)
Terminadas (2)
Abandonadas (1)
Con reseña (1)
Sin puntuar (0)
Noviembre de 2026
13
Haikyuu!!
En Flor
Manga · 45 cap.
Escribir reseña
1
0
Todas (3)
Terminadas (2)
Abandonadas (1)
Con reseña (1)
Sin puntuar (0)
Fecha: más recientes
Noviembre de 2026
13
Vie
Haikyuu!!
En Flor
Manga · 45 cap.
Sin reseña todavía.
Escribir reseña
Borrar reseña
13
Vie
Frieren
En Flor
Manga · 10 cap.
"Una historia hermosa sobre el paso del tiempo y los recuerdos."
Editar reseña
Borrar reseña
1
Dom
Monster
Marchita
Anime · 0 ep.
Sin reseña todavía.
Escribir reseña
Borrar reseña
nana_ia.exe
Arriba ves cuántas leíste, tu promedio y un gráfico con tus puntuaciones.
1. Cuando una planta florece (o la abandonás), se anota sola en el Diario con la fecha de hoy.
2. Tocá una sakura para puntuarla, o "Escribir reseña" para contar qué te pareció y corregir la fecha.
3. Filtrá por terminadas, abandonadas, con reseña o sin puntuar, y ordená por fecha, puntuación o título.
4. "Borrar reseña" quita las sakuras y el texto, pero la lectura sigue en tu diario.
Ej: ¿cómo riego mi planta?
Enviar
Jardinera anónima #F0RXQ
Cerrar sesión
Nana · Hecho con React + Vite + Firebase · Datos de obras: Kitsu · TP2 Desarrollo Frontend II


NANA
♡ tu jardín manga
Cada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Mi jardín
Diario
Comunidad
Herbario
diario.exe (3)
Tu diario de lecturas: todo lo que terminaste o abandonaste, con la fecha, tus sakuras y tu reseña. Tocá una sakura para puntuar, o el lápiz para escribir la reseña y cambiar la fecha.
3
leídas
3
puntuadas
1
reseña
3,7
de promedio
Tus puntuaciones



Dificultades y cómo se resolvieron

Problema	Cómo se detectó	Solución
En el celular, la pestaña **"Herbario" quedaba cortada**	Captura a 390 px. La prueba automática decía "sin scroll horizontal" porque la página **recorta** lo que sobra (`overflow-x: clip`)	Pestañas en 4 columnas iguales y fuente de 8–9 px. Se mejoró la prueba: ahora también comprueba que **ninguna pestaña se corte**
Las pruebas viejas fallaron al buscar Comunidad y Herbario	Regresión: 3 pruebas no encontraban sus pantallas	Las pruebas tocaban las pestañas **por posición** (la 2.ª era Comunidad). Con el Diario en el 2.º lugar se corrieron. Se actualizaron las posiciones
Nana IA decía que el Herbario es la "tercera pestaña"	Prueba de regresión de la v1.0	Ahora es la cuarta: se corrigió el texto y la prueba
¿Cómo hacer sonar música si el navegador bloquea el sonido automático?	Investigando la política de reproducción automática de los navegadores	Empieza con el primer toque de la persona en la página, que es lo que los navegadores permiten
¿Qué pasa con las plantas terminadas **antes** de que existiera la fecha?	Al pensar los datos viejos	El diario usa la fecha en que se plantaron, y se puede corregir con el lápiz

Pruebas finales en local (152 de 152)

Prueba automática	Resultado
Cierre v1.2: diario vacío, entrada automática con la fecha, puntuar desde el diario, resumen y gráfico, editar reseña y fecha, fecha del futuro rechazada, agrupación por mes, filtros y orden, borrar reseña (campos borrados en Firestore), volver a leer (sale del diario y se borra la fecha), **403** para una fecha inválida, Nana IA, música (suena, se apaga y vuelve), gatito, espaciado de Comunidad, celular con 4 pestañas completas	**36/36** ✅
v1.2: puntuación y reseña	**26/26** ✅
v1.0: ramo, filtros, herbario, sonidos, Nana IA	**26/26** ✅
v0.4.1: flores, guía, Nana IA, perfil, cerrar sesión	**21/21** ✅
v0.4: comunidad con dos usuarias, roles y seguridad (403)	**19/19** ✅
v0.3: jardín, API, edición, estados	**17/17** ✅
Diseño: sin scroll horizontal en compu, tablet y celular	**7/7** ✅

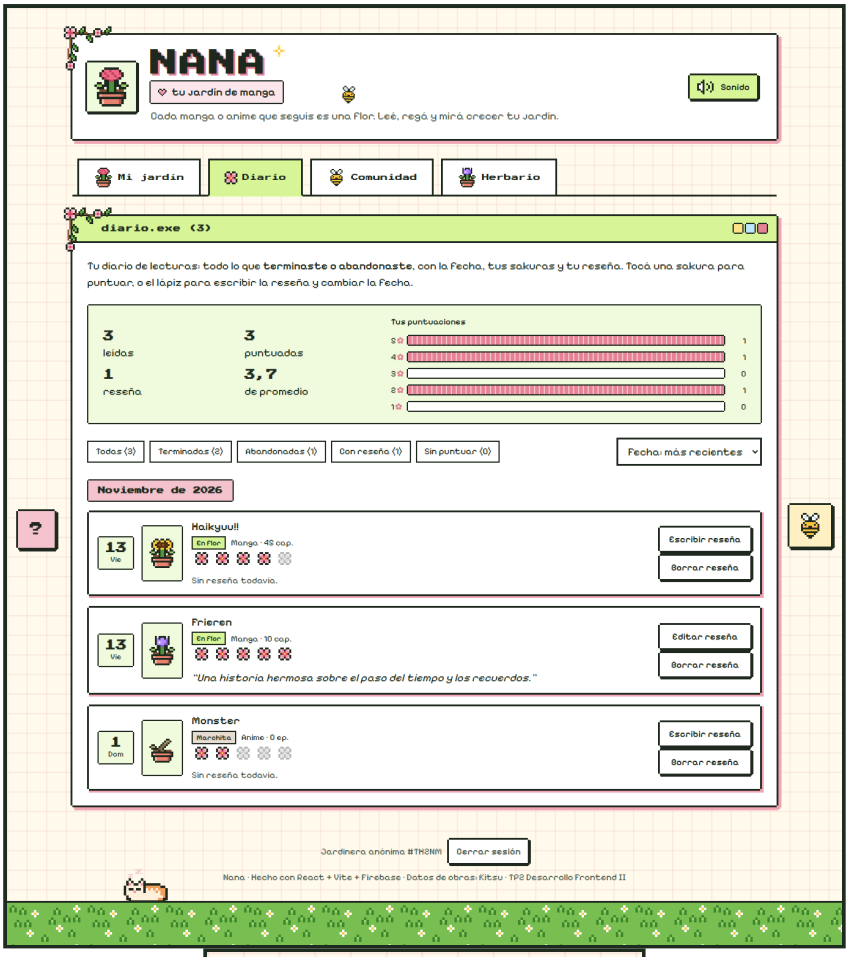
La prueba de la música cuenta cuántas notas crea el navegador: con el sonido activado aparecen unas 12 notas nuevas cada 1,5 segundos. Después de tocar "Sonido", aparecen **0**.

Revisión general contra la consigna y la rúbrica

Qué pide	Dónde se cumple	Verificado
1. React + Vite con componentes, eventos y estado	34 componentes (contando `App`). `useState`, `useEffect` con limpieza, eventos `onClick`, `onChange` y `onSubmit`	✅ lint y build sin errores
2. Firestore: consultar, agregar, modificar y eliminar	Plantas, puntuación, reseña, fecha del diario, perfil, clubes, misiones y eventos	✅ pruebas que leen Firestore directamente
3. Persistencia al recargar (no array ni `localStorage`)	Todo en Firestore. `localStorage` solo guarda la preferencia de sonido	✅ las pruebas recargan y comprueban
4. GitHub con código e instrucciones	Repositorio **público**, README con `npm install` y `npm run dev`, etiquetas v0.1…v1.2	✅ visto sin iniciar sesión
5. Vercel con URL funcional	https://jardin-de-manga.vercel.app, despliegue automático desde `main`	✅ responde y las pruebas pasan sobre la URL
6. Validaciones, carga, éxito/error, compu y celular	Errores por campo (título, capítulos, reseña, fecha), barras de carga, avisos, 390 px sin cortes	✅
7. Reglas de Firestore no abiertas	Jardín privado, roles en clubes, validación de cada campo	✅ varias pruebas de **403**
`.gitignore` con `.env`, `node_modules` y `dist`	Sí. `.env` nunca se subió. Solo está `.env.example`	✅ revisado con `git ls-files`
`.env.example`, reglas y README en el repo	Los tres están	✅
Al menos 3 versiones, con bitácora y verificación	8 versiones (v0.1 → v1.2), cada una probada en Vercel	✅
Informe: idea, público, funciones, datos, IA, instrucciones	`docs/informe.md`	✅
Bitácora con capturas, pruebas, IA y dificultades	Este documento	✅
Diseño justificado y refinado en iteraciones	Pixel art (v0.2), flores únicas (v0.4.1), kawaii (v1.1), diario y gatito (v1.2), con referencias	✅
Mini video	Guion en `docs/guion-video.md`	⏳ lo graba la autora

Publicación y verificación final en Vercel

1. Las reglas con `terminadoEn` se publicaron **antes** del código (Firestore → Reglas → Publicar).
2. Se revisó la lista de cambios: sin archivos borrados y sin `.env`.
3. Se subió a `main`, y Vercel publicó la versión nueva en unos 30 segundos.
4. **Sobre la URL pública** se corrieron **todas** las pruebas: cierre v1.2 (36/36), puntuación (26/26), v1.0 (26/26), v0.4.1 (21/21), comunidad y seguridad (19/19), jardín (17/17) y diseño (7/7). En total, **152 de 152 correctas**, sin errores en la consola.

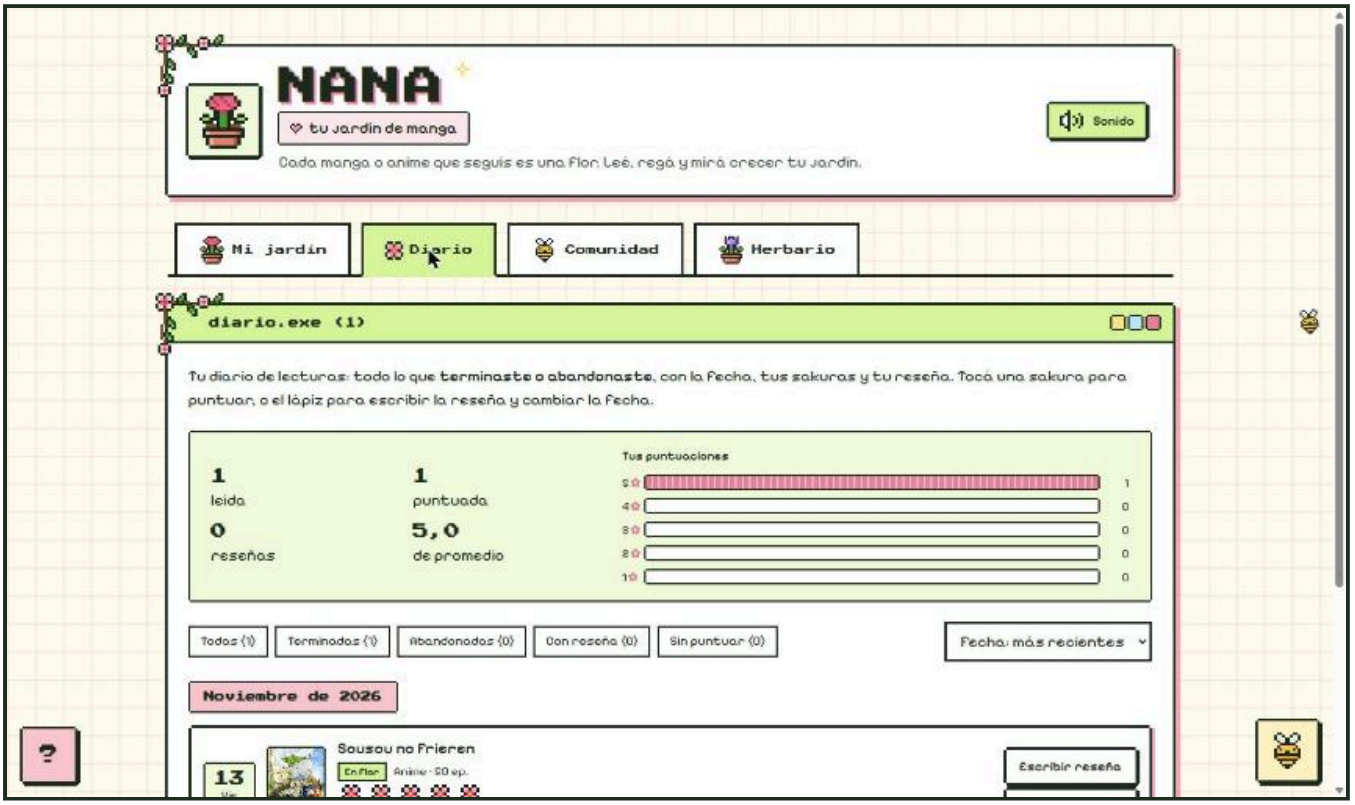

NANA
tu jardín de manga
Sonido
Dada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Mi jardín
Diario
Comunidad
Herbario
diario.exe (3)
Tu diario de lecturas: todo lo que terminaste o abandonaste, con la fecha, tus sakuras y tu reseña. Tocá una sakura para puntuar, o el lápiz para escribir la reseña y cambiar la fecha.
3 leídas
3 puntuadas
1 reseña
3,7 de promedio
Tus puntuaciones
Todas (3)
Terminadas (2)
Abandonadas (1)
Con reseña (1)
Sin puntuar (0)
Fecha: más recientes
Noviembre de 2026
Haikyuu!!
En flor
Manga · 45 cap.
Sin reseña todavía.
Escribir reseña
Borrar reseña
Frieren
En flor
Manga · 10 cap.
"Una historia hermosa sobre el paso del tiempo y los recuerdos."
Editar reseña
Borrar reseña
Monster
Marchita
Anime · 0 ep.
Sin reseña todavía.
Escribir reseña
Borrar reseña
Jardinera anónima #TH2NM
Cerrar sesión
Nana · Hecho con React + Vite + Firebase · Datos de obras: Kitsu · TP2 Desarrollo Frontend II


NANA
tu jardín de manga
Dada manga o anime que seguís es una flor. Leé, regá y mirá crecer tu jardín.
Mi jardín
Diario
Comunidad
Herbario
diario.exe (3)
Tu diario de lecturas: todo lo que terminaste o abandonaste, con la fecha, tus sakuras y tu reseña. Tocá una sakura para puntuar, o el lápiz para escribir la reseña y cambiar la fecha.
3 leídas
3 puntuadas
1 reseña
3,7 de promedio
Tus puntuaciones

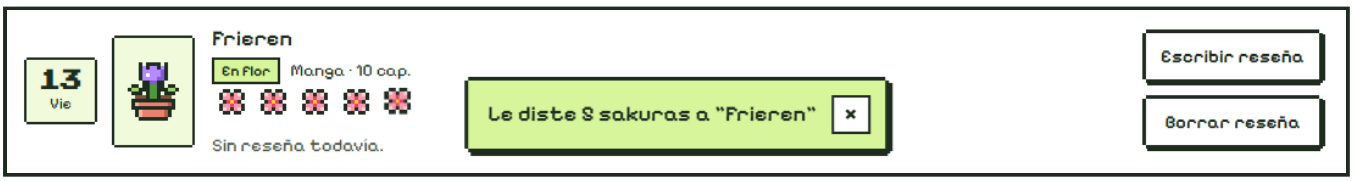
Integración final: una sola v1.2

La autora pidió que **todo lo de este sprint quede en una única versión final**, sin crear una v1.3. La etiqueta `v1.2` se había creado con la primera parte del sprint (puntuación y reseña). Por eso **se movió al último commit**: así, en GitHub, `v1.2` incluye la puntuación, el Diario, la música, el gatito, el espaciado y toda la documentación.

También se comprobó la URL pública con **el jardín real de la autora** (no con un usuario de prueba). Su planta terminada, *Sousou no Frieren*, apareció sola en el Diario, con sus 5 sakuras. Las plantas que se terminaron antes de que existiera la fecha usan la fecha en que se plantaron.



Último ajuste de diseño: la autora notó en el Diario que los botones "Escribir reseña" y "Borrar reseña" estaban **demasiado pegados**. El espacio era de 8 px, pero cada botón tiene una sombra pixelada de unos 5 px hacia abajo, así que a la vista quedaban casi juntos. Se pasó a **16 px** (en compu y en celular) y se dejó lugar para la sombra. La prueba lo mide: 16 px en los dos tamaños, y el Diario sigue en 36/36.



Entrega en PDF: la autora pidió entregar el informe y la bitácora **en PDF, con sus capturas**. Se generaron desde estos mismos documentos (Markdown → HTML → PDF con Chrome), así el contenido es idéntico al del repositorio. Tienen portada, índice con enlaces, números de página y todas las imágenes (11 en el informe y 95 en la bitácora). Además, al informe se le sumó una sección **"7. Capturas"** con la evolución del diseño (v0.1 → v0.2 → v1.1 → v1.2) y las pantallas de la versión final.

- docs/Informe-Nana-TP2.pdf
- docs/Bitacora-Nana-TP2.pdf

> **Si "no se ven los cambios":** Vercel ya publicó la versión nueva, pero una pestaña abierta desde antes sigue mostrando la versión vieja. Se soluciona recargando con `Ctrl + F5` o abriendo la URL en una ventana de incógnito.

Retrospectiva del cierre

- **Qué salió bien:** el Diario suma una sección muy útil **sin duplicar datos**: reutiliza las mismas plantas, el mismo modal de edición, las sakuras y los avisos.
- **Qué se aprendió:** una prueba automática puede dar "OK" y aun así haber un problema visual (la pestaña cortada). Por eso siempre se miran también las capturas.
- **Decisión de alcance:** no se agregaron más secciones. Con 4 pestañas el sitio queda completo y fácil de usar.

Conclusión del proceso

Versión	Qué aportó	Tipo de iteración
v0.1	MVP: CRUD con Firestore, reglas, GitHub y Vercel	Producto mínimo viable
v0.2	Pixel art, flores dibujadas con código, validaciones, avisos, celular	Diseño y experiencia de uso
v0.3	Nombre "Nana", búsqueda con la API, edición, estados coherentes	Funcionalidad
v0.4	Comunidad: clubes, misiones con rol de administradora, eventos, compartir	Módulo nuevo
v0.4.1	9 flores con forma propia, guía, abeja, Nana IA, CRUD del perfil	Ajustes de diseño y ayuda
v1.0	Ramo, filtros, herbario, sonidos, auditoría y documentación final	Estabilización
v1.1	Más espacio, esquinas pixeladas, enredaderas, pasto y revisión final de la rúbrica	Pulido visual
v1.2	Puntuación con sakuras, reseña corta, **Diario de lecturas** con fecha, música de fondo original, gatito, espaciado y revisión final	Incremento funcional y cierre

Lo que más se aprendió:

- **Probar es tan importante como programar:** casi todos los errores aparecieron en las capturas o en las pruebas, no al escribir el código.
- **La seguridad no se resuelve escondiendo botones:** se resuelve en las reglas de la base de datos.
- **Agregar no siempre es mejor:** antes de sumar una función hay que ver si ya existe de otra forma (en la v1.2, "quiero leer" ya era el estado Pendiente).
- **Una app que depende de servicios externos** (una API) tiene que seguir funcionando si ese servicio falla.
- **La IA acelera mucho el trabajo,** pero hace falta revisar, entender y decidir: varias decisiones (el nombre, el pixel art, la comunidad en clubes, Nana IA sin claves) salieron de esa revisión.